

GNPy Implementation Analysis

Complete Technical Reference for
Optical Network Planning Software Development



GNPy

Technical Documentation

Based on GNPy v2.x

Telecom Infra Project

November 25, 2025

Preface

This technical reference provides an in-depth analysis of GNP_y (Gaussian Noise Python), an open-source optical network planning tool. The documentation is heavily based on artificial intelligence (Anthropic’s Claude). Although verification and corrections were done, it could contain errors, please read carefully.

PURPOSE

This document intends to be a bridge between optical networking theory and software implementation, focusing on:

- Understand physical layer impairment model
- Implement accurate quality-of-transmission (QoT) estimation
- Build network planning algorithms
- Create spectrum assignment strategies
- Develop output analysis tools

DOCUMENT STRUCTURE

Part I: Technical Reference (Chapters 1-5)

Chapter 1: Physical Layer Impairments & GNP_y Implementation

Covers theoretical foundations and software architecture for modeling optical impairments including ASE noise, nonlinear interference, chromatic dispersion, and PMD.

Chapter 2: Network Model Building

Analyzes the `auto-design` algorithm for automatic network design, equipment library management, and topology construction using NetworkX.

Chapter 3: Path Computation & Spectrum Assignment

Details constrained path finding algorithms, spectrum allocation strategies, and transponder mode selection with critical insights on topology-based vs. QoT-based optimization.

Chapter 4: Propagation & QoT Computation

Examines element-by-element signal propagation, NLI calculation methods (GN/GGN), and GSNR computation including penalty models.

Chapter 5: Output Analysis & Results Interpretation

Provides practical guidance on interpreting simulation results, troubleshooting strategies, and validation techniques.

Part II: Practical Guide (Chapter 6)

Chapter 6: Practical Guide to Network Simulation

Hands-on tutorial covering installation, topology creation, workflow examples, troubleshooting, and best practices for engineers using GNP_y in production environments.

Recommended Reading Paths:

- **Quick Start:** Chapter 6 → Chapters 1-5 as needed
- **Developer/Implementer:** Chapters 1-5 → Chapter 6
- **Network Planner:** Chapter 6 + Chapter 3 + Chapter 5

HOW TO USE THIS DOCUMENT

- **Sequential Reading:** Follow chapters 1-5 for comprehensive understanding
- **Reference Mode:** Use table of contents and hyperlinks for specific topics
- **Implementation Guide:** Code examples include actual GNPpy line numbers
- **Validation:** Each chapter includes testing and validation sections

CONVENTIONS

Code Blocks: Python examples with syntax highlighting

Equations: Mathematical formulations with variable definitions

Tables: Parameter references with units and typical ranges

Critical Notes: Implementation warnings and best practices

ACKNOWLEDGMENTS

This documentation is based on GNPpy developed by the Telecom Infra Project and contributors. Special recognition to the core development team for creating robust, production-grade open-source optical network planning software.

Contents

1	Physical Layer Impairments & GNPpy Implementation	1
1.1	Introduction to Optical Network Impairments	1
1.1.1	Scope and Motivation	1
1.1.2	Impairment Classification	1
1.1.3	GNPy Architecture Overview	3
1.2	Linear Fiber Impairments	3
1.2.1	Attenuation	3
1.2.2	Chromatic Dispersion	5
1.2.3	Polarization Mode Dispersion	7
1.2.4	Polarization Dependent Loss	8
1.3	Nonlinear Fiber Impairments	9
1.3.1	Theoretical Foundation: Nonlinear Schrödinger Equation	9
1.3.2	Self-Phase Modulation (SPM)	10
1.3.3	Cross-Phase Modulation (XPM)	10
1.3.4	Four-Wave Mixing (FWM)	10
1.3.5	The Gaussian Noise Model	11
1.3.6	Stimulated Raman Scattering	14
1.4	Amplifier Impairments	16
1.4.1	Amplified Spontaneous Emission (ASE) Noise	16
1.4.2	Noise Power in Different Bandwidths	20
1.4.3	Gain Spectral Tilt	20
1.5	ROADM Impairments	21
1.5.1	Optical Filtering	21
1.5.2	Inter-Channel Crosstalk	22
1.6	Implementation Workflow	23
1.6.1	Signal Propagation Architecture	23
1.6.2	SpectralInformation Class	24
1.6.3	Element Propagation Chain	25
1.6.4	Typical Propagation Sequence	25
1.6.5	QoT Estimation at Receiver	26
1.7	Validation and Calibration	27
1.7.1	Model Validation Approaches	27
1.7.2	Typical Validation Metrics	27
1.7.3	Calibration Parameters	27
1.8	Summary	27
1.8.1	Implementation Architecture	27
1.8.2	Critical Implementation Details	28
1.8.3	Software Development Considerations	28
1.8.4	Next Steps	28
2	Network Model Building	29

2.1	Introduction to Network Model Building	30
2.1.1	Overview and Motivation	30
2.1.2	Three-Function Architecture	30
2.1.3	Design Philosophy	30
2.1.4	Chapter Organization	31
2.2	Stage 1: Equipment Library Loading	31
2.2.1	Function Specification	31
2.2.2	Equipment Configuration Structure	32
2.2.3	Implementation Details	32
2.2.4	Validation Functions	34
2.2.5	Equipment Dictionary Structure	39
2.3	Stage 2: Network Graph Construction	40
2.3.1	Function Specification	40
2.3.2	Topology JSON Structure	41
2.3.3	Graph Construction Algorithm	41
2.3.4	Network Validation	44
2.3.5	NetworkX DiGraph Structure	47
2.4	Stage 3: Network Auto-Design	48
2.4.1	auto-design Algorithm Overview	48
2.4.2	Function Specification	48
2.4.3	Step 1: Add Missing Elements	49
2.4.4	Step 2: Set Fiber Attributes	53
2.4.5	Step 3: Power Optimization (auto-design Algorithm)	54
2.4.6	Output VOA Configuration	60
2.5	OpenROADM Alignment	61
2.5.1	Equipment Library Mapping	61
2.5.2	Network Topology Mapping	61
2.5.3	Design Output Mapping	61
2.6	Implementation Considerations	61
2.6.1	Performance Optimization	61
2.6.2	Testing Strategies	62
2.6.3	Error Handling Best Practices	62
2.7	Summary	63
2.7.1	Key Takeaways	63
2.7.2	Next Chapter Preview	63
3	Path Computation & Spectrum Assignment	65
3.1	Overview	66
3.2	Path Computation	66
3.2.1	Algorithm Foundation	66
3.2.2	Core Function: <code>compute_constrained_path</code>	66
3.2.3	Path Selection vs. QoT Validation	67
3.2.4	Routing Constraints	68
3.2.5	Disjoint Path Computation	68
3.3	Spectrum Assignment	68
3.3.1	Optical Multiplex Section (OMS)	68
3.3.2	Spectrum Slot Model	69
3.3.3	Path Spectrum Assignment	69
3.3.4	Bidirectional Spectrum Assignment	71
3.4	Mode Selection	71
3.4.1	Transponder Modes	71

3.4.2	Mode Selection Algorithm	71
3.4.3	GSNR Calculation with Penalties	73
3.5	Integration: Complete Workflow	73
3.5.1	Service Request Processing	73
3.5.2	Blocking Reasons	75
3.6	Performance Considerations	75
3.6.1	Computational Complexity	75
3.6.2	Optimization Opportunities	75
3.7	Practical Examples	76
3.7.1	Example 1: Simple Path Request	76
3.7.2	Example 2: Constrained Routing	76
3.7.3	Example 3: Spectrum Exhaustion	77
3.7.4	Example 4: Bidirectional Service	77
3.8	Summary	77
3.8.1	Key Takeaways	77
3.8.2	Implementation Guidance	77
3.8.3	Extensions	78
4	Propagation & QoT Computation	79
4.1	Overview	80
4.2	SpectralInformation Data Structure	80
4.2.1	Core Attributes	80
4.2.2	Key Methods	80
4.2.3	Power Representation	81
4.3	Fiber Propagation	81
4.3.1	Fiber Model Overview	81
4.3.2	Propagation Algorithm	81
4.3.3	Chromatic Dispersion Model	82
4.3.4	PMD Model	83
4.3.5	Nonlinear Interference (NLI)	83
4.3.6	Raman Scattering	85
4.4	EDFA Propagation	86
4.4.1	EDFA Model Overview	86
4.4.2	Amplifier Types	86
4.4.3	Propagation Implementation	86
4.4.4	ASE Noise Calculation	87
4.4.5	Noise Figure Models	88
4.4.6	Gain Saturation	88
4.5	ROADM Propagation	89
4.5.1	ROADM Model Overview	89
4.5.2	Propagation Implementation	89
4.5.3	Channel Equalization Strategy	90
4.5.4	Add/Drop OSNR Model	90
4.6	Transceiver (Receiver) Computation	91
4.6.1	Transceiver Model Overview	91
4.6.2	SNR Calculation	91
4.6.3	Penalty Calculation	92
4.6.4	Effective GSNR with Penalties	93
4.6.5	Transmitter OSNR	94
4.7	Complete Propagation Example	94
4.7.1	End-to-End Signal Flow	94

4.7.2	Power Budget Example	96
4.8	Performance Considerations	96
4.8.1	Computational Bottlenecks	96
4.8.2	Accuracy vs. Speed Trade-offs	96
4.8.3	Optimization Strategies	96
4.9	Validation and Testing	97
4.9.1	Unit Test Strategy	97
4.9.2	End-to-End Validation	98
4.10	Summary	98
4.10.1	Key Takeaways	98
4.10.2	Implementation Checklist	98
4.10.3	Extensions	99
5	Output Analysis & Results Interpretation	101
5.1	Overview: GNPY's Analytical Capabilities	102
5.1.1	GNPY coverage	102
5.1.2	Output Hierarchy	102
5.1.3	Command Reference	102
5.2	Core Metrics: Physical Interpretation	102
5.2.1	Metric Definitions and Physical Meaning	102
5.2.2	Regime Identification	103
5.2.3	Decision Thresholds	104
5.3	Output Formats & Access Methods	104
5.3.1	JSON Response Structure	104
5.3.2	Successful Path Response	104
5.3.3	Path Route Objects	105
5.3.4	Blocked Path Response	106
5.4	CSV Output Format	107
5.4.1	Generation	107
5.4.2	Console Output: Element-Level Diagnostics	107
5.4.3	CSV Column Interpretation	109
5.4.4	Example CSV Output	109
5.5	Console Output	109
5.5.1	Path Request Summary	109
5.5.2	Transmission Example Output	110
5.6	Auto-Designed Network Output	110
5.6.1	Purpose	110
5.6.2	Generation	110
5.6.3	Structure	110
5.7	Programmatic Access for Research	111
5.7.1	Python API Integration	111
5.7.2	CSV Batch Analysis	112
5.8	Metrics Interpretation	112
5.8.1	GSNR vs. OSNR	112
5.8.2	GSNR Margins	112
5.8.3	Per-Channel Analysis	113
5.9	Troubleshooting	113
5.9.1	Low GSNR Issues	113
5.9.2	Spectrum Exhaustion	114
5.9.3	Amplifier Saturation	115
5.9.4	CD/PMD Penalties	115

5.10	Validation Techniques	116
5.10.1	Sanity Checks	116
5.10.2	Regression Testing	117
5.11	Performance Analysis	118
5.11.1	Comparing Alternatives	118
5.11.2	Sensitivity Analysis	120
5.12	Practical Workflows	121
5.12.1	Network Planning	121
5.12.2	Troubleshooting Workflow	121
5.13	Best Practices	121
5.13.1	Simulation Setup	121
5.13.2	Output Analysis	122
5.13.3	Common Pitfalls	122
5.14	Summary	122
5.14.1	Key Metrics Hierarchy	122
5.14.2	Analysis Checklist	122
5.14.3	Next Steps	122
6	Practical Guide to Network Simulation	123
6.1	Getting Started	123
6.1.1	Installation	123
6.1.2	GNPy Files	125
6.1.3	Command Line Tools	125
6.1.4	Quick Decision Tree	126
6.2	A First Simulation	127
6.2.1	Step 1: Create Topology	127
6.2.2	Step 2: Run Simulation	128
6.2.3	Step 3: Output	128
6.2.4	Step 5: Save Auto-Designed Network	130
6.2.5	Step 6: Experiment	130
6.3	Creating A Network Model	131
6.3.1	Network Information Gathering	131
6.3.2	Step-by-Step Topology Creation	131
6.3.3	Common Topology Patterns	133
6.3.4	Equipment Configuration	135
6.3.5	Validation Checklist	136
6.4	Practical Workflows	137
6.4.1	Workflow 1: Network Feasibility Study	137
6.4.2	Workflow 2: Path Request Validation	138
6.4.3	Workflow 3: Network Design Optimization	142
6.4.4	Workflow 4: Troubleshooting Low GSNR	142
6.4.5	Workflow 5: Comparing Design Alternatives	143
6.5	Common Issues and Solutions	144
6.5.1	File and Configuration Issues	144
6.5.2	Physical Layer Issues	145
6.5.3	Service Request Issues	147
6.5.4	Performance Issues	148
6.6	Parameter Reference Guide	149
6.6.1	Finding Realistic Values	149
6.6.2	When You Don't Have Exact Values	150
6.7	Complete Worked Examples	151

6.7.1	Example 1: Metro Ring Network	151
6.7.2	Example 2: Long-Haul Point-to-Point	153
6.7.3	Example 3: Service Request with Constraints	154
6.8	Best Practices Summary	155
6.8.1	Modeling Checklist	155
6.8.2	Performance Guidelines	156
6.8.3	Common Pitfalls to Avoid	156
6.8.4	When to Use What	156
6.9	Next Steps	156
6.9.1	From Here	156
6.9.2	Additional Resources	157
6.9.3	Getting Help	157
6.10	Summary	158
A	Quick Reference	159
A.1	Common Parameters	159
A.2	Unit Conversions	159
A.3	Command Reference	159
B	Glossary	161
C	Bibliography	163

Chapter 1

Physical Layer Impairments & GNPpy Implementation

CHAPTER OVERVIEW

This chapter provides a comprehensive analysis of physical layer impairments in optical networks and their implementation in GNPpy. It covers the theoretical foundations, mathematical models, and software architecture required to understand and implement quality-of-transmission (QoT) estimation tools for wavelength division multiplexed (WDM) networks. This document serves as a technical reference for developers implementing optical network planning software.

1.1. INTRODUCTION TO OPTICAL NETWORK IMPAIRMENTS

1.1.1 Scope and Motivation

In wavelength division multiplexed (WDM) optical networks, accurate prediction of signal quality is essential for network planning, capacity optimization, and ensuring service reliability. The quality of transmission (QoT) depends on accumulated physical impairments along the optical path, including linear and nonlinear fiber effects, amplifier noise, and filtering in reconfigurable optical add-drop multiplexers (ROADMs).

GNPpy (Gaussian Noise model Python) implements a comprehensive physical layer model based on the Gaussian Noise (GN) model and its extensions. This chapter analyzes each impairment category and details how GNPpy's architecture models these effects.

1.1.2 Impairment Classification

Physical impairments are classified into four primary categories:

1. **Linear Fiber Impairments:** Signal attenuation, chromatic dispersion (CD), polarization mode dispersion (PMD), polarization dependent loss (PDL)
2. **Nonlinear Fiber Effects:** Self-phase modulation (SPM), cross-phase modulation (XPM), four-wave mixing (FWM), stimulated Raman scattering (SRS)
3. **Amplifier Impairments:** Amplified spontaneous emission (ASE) noise, gain spectral tilt, transient effects
4. **ROADM Impairments:** Optical filtering, inter-channel crosstalk, add/drop OSNR degradation

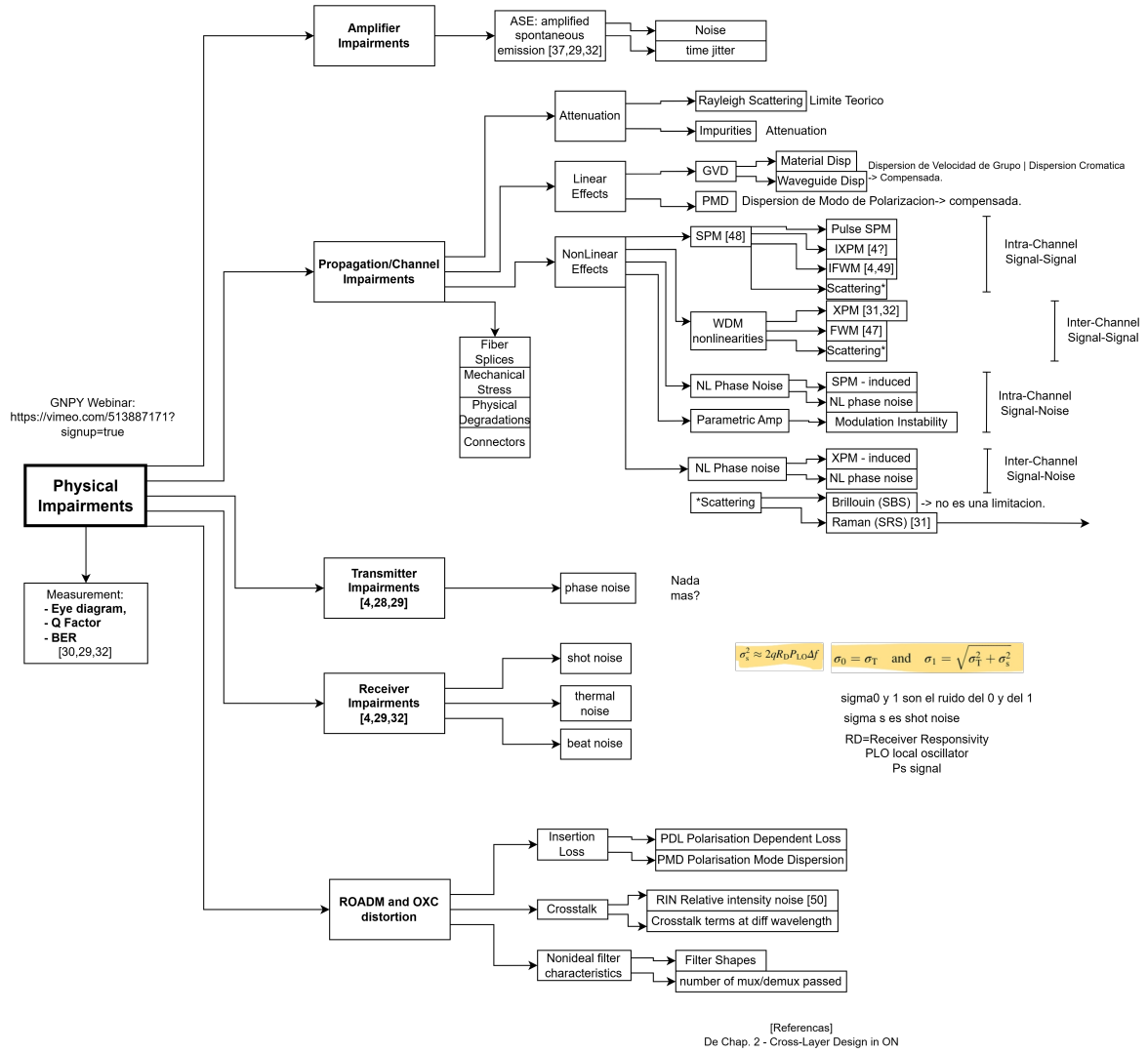


Figure 1.1: Physical impairments taxonomy in optical networks. Impairments are categorized by network element (transmitter, fiber, amplifier, ROADM, receiver) and physical phenomenon (linear, nonlinear, noise).

1.1.3 GNPpy Architecture Overview

GNPpy's implementation is organized into modular components:

- **elements.py**: Network element classes (Fiber, Edfa, Roadm, Transceiver)
- **science_utils.py**: Physics solvers (NliSolver, RamanSolver)
- **info.py**: Spectral information propagation container
- **network.py**: Network topology and path computation

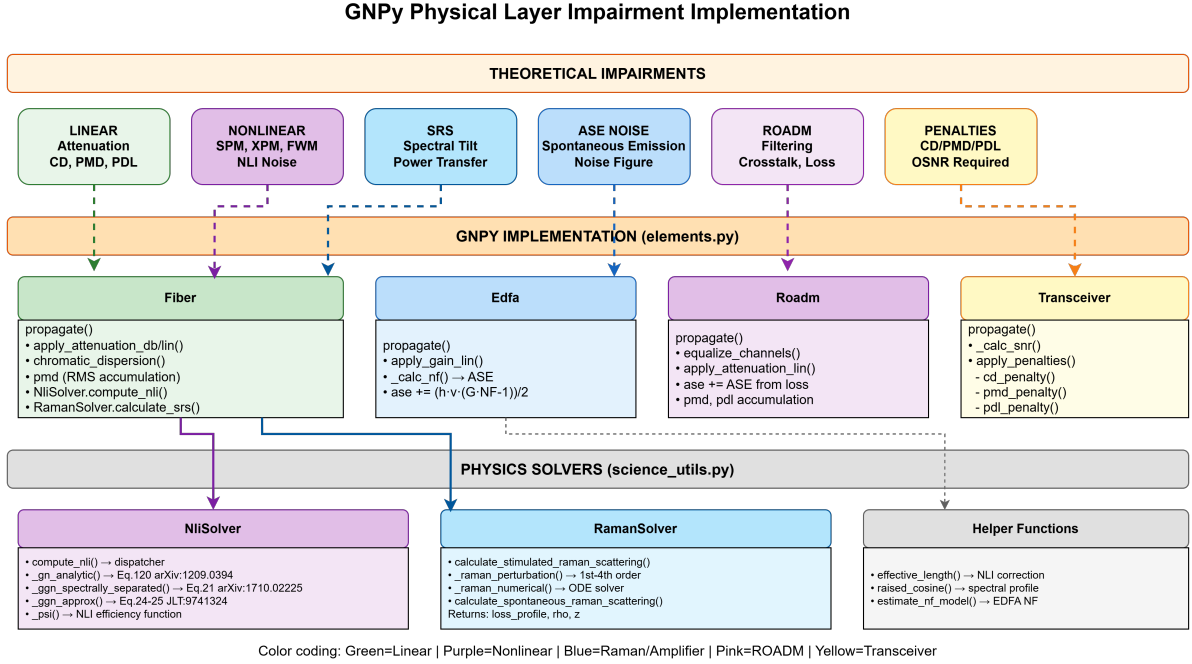


Figure 1.2: GNPpy implementation architecture showing mapping between theoretical impairments and software modules. Color coding: Green=Linear, Purple=Nonlinear, Blue=Amplifier, Pink=ROADM, Yellow=Transceiver.

1.2. LINEAR FIBER IMPAIRMENTS

Linear impairments are power-independent effects that accumulate deterministically along the fiber path. These impairments can be compensated (CD) or mitigated through system design (PMD, PDL).

1.2.1 Attenuation

Physical Phenomenon

Optical attenuation results from Rayleigh scattering and material absorption in silica fiber. Rayleigh scattering dominates at 1550 nm with typical loss coefficients of 0.18-0.22 dB/km for standard single-mode fiber (SSMF).

Physical Origin:

- **Rayleigh scattering**: Microscopic density fluctuations frozen during fiber manufacture cause scattering proportional to λ^{-4}
- **Infrared absorption**: Molecular vibrations in Si-O bonds
- **UV absorption**: Electronic transitions

Mathematical Model

The optical power along a fiber of length L evolves according to Beer's law:

$$P(z) = P_0 \exp(-\alpha z) \quad (1.1)$$

where:

- P_0 = input power (W)
- α = attenuation coefficient (Np/m)
- z = propagation distance (m)

In decibel units:

$$P_{dB}(z) = P_{0,dB} - \alpha_{dB} \cdot z \quad (1.2)$$

where $\alpha_{dB} = 10 \log_{10}(e) \cdot \alpha \approx 4.343\alpha$ (dB/m).

Typical Values:

- SSMF at 1550 nm: $\alpha = 0.2$ dB/km = 4.6×10^{-5} Np/m
- Non-zero dispersion-shifted fiber (NZDSF): $\alpha = 0.22$ dB/km
- Low-loss fibers: $\alpha < 0.17$ dB/km

GNPy Implementation

Class Structure:

The `Fiber` class in `elements.py` implements attenuation through the `propagate()` method:

```

1 class Fiber(Element):
2     def propagate(self, spectral_info: SpectralInformation):
3         """Apply fiber propagation including attenuation"""
4
5         # 1. Input connector loss
6         attenuation_in_db = self.params.con_in + self.params.att_in
7         spectral_info.apply_attenuation_db(attenuation_in_db)
8
9         # 2. Compute Raman scattering profile
10        if sim_params.raman_params.flag:
11            srs = RamanSolver.calculate_stimulated_raman_scattering(
12                spectral_info, self)
13            # Fiber attenuation modified by SRS
14            attenuation_fiber = srs.loss_profile[:, -1]
15            spectral_info.apply_attenuation_lin(attenuation_fiber)
16        else:
17            # Simple exponential attenuation
18            fiber_loss_db = self.params.loss_coef * self.params.length
19                / 1000
20            spectral_info.apply_attenuation_db(fiber_loss_db)
21
22        # 3. Output connector loss
23        attenuation_out_db = self.params.con_out
24        spectral_info.apply_attenuation_db(attenuation_out_db)

```

Key Implementation Details:

1. **Frequency-dependent attenuation:** The `alpha()` method computes wavelength-dependent loss:

```

1 def alpha(self, frequency=None):
2     """Attenuation coefficient in Np/m"""
3     ref_freq = self.params.ref_frequency
4     ref_wavelength = c / ref_freq # Reference wavelength
5
6     if frequency is None:
7         frequency = ref_freq
8
9     wavelength = c / frequency

```

```

10
11     # Rayleigh scattering: alpha proportional to lambda^-4
12     alpha_ref = self.params.loss_coef / (10 * log10(e)) / 1000
13     alpha_f = alpha_ref * (wavelength / ref_wavelength)**(-4)
14
15     return alpha_f

```

2. **Connector losses:** Separate accounting for input/output connector imperfections (`con_in`, `con_out`)
3. **Additional attenuation:** `att_in` parameter allows modeling splice losses or other discrete losses

Configuration Parameters:

Parameter	Default	Description
<code>loss_coef</code>	0.2 dB/km	Fiber attenuation coefficient
<code>con_in</code>	0.5 dB	Input connector loss
<code>con_out</code>	0.5 dB	Output connector loss
<code>att_in</code>	0 dB	Additional input attenuation
<code>ref_frequency</code>	193.1 THz	Reference frequency for loss

Table 1.1: Attenuation configuration parameters in GNPpy

1.2.2 Chromatic Dispersion

Physical Phenomenon

Chromatic dispersion (CD) arises from the frequency dependence of the refractive index in optical fiber. Different spectral components of a modulated signal travel at different group velocities, causing pulse spreading.

Physical Origin:

- **Material dispersion:** Frequency-dependent refractive index of silica
- **Waveguide dispersion:** Frequency-dependent mode confinement
- **Total dispersion:** $D(\lambda) = D_{\text{material}} + D_{\text{waveguide}}$

Mathematical Model

The group delay per unit wavelength is characterized by the dispersion parameter:

$$D(\lambda) = -\frac{\lambda}{c} \frac{d^2 n}{d\lambda^2} \quad [\text{ps}/(\text{nm} \cdot \text{km})] \quad (1.3)$$

The accumulated chromatic dispersion over length L is:

$$CD_{\text{total}} = D(\lambda) \cdot L \quad [\text{ps}/\text{nm}] \quad (1.4)$$

Taylor Series Expansion:

CD is commonly expressed using group velocity dispersion (GVD) parameters:

$$\beta(\omega) = \beta_0 + \beta_1(\omega - \omega_0) + \frac{1}{2}\beta_2(\omega - \omega_0)^2 + \frac{1}{6}\beta_3(\omega - \omega_0)^3 + \dots \quad (1.5)$$

where:

- β_0 = propagation constant (rad/m)
- $\beta_1 = 1/v_g$ = inverse group velocity (s/m)
- β_2 = group velocity dispersion (s^2/m)
- β_3 = dispersion slope parameter (s^3/m)

Relationship between D and β_2 :

$$\beta_2 = -\frac{D\lambda^2}{2\pi c} \quad (1.6)$$

Typical Values:

- SSMF: $D = 16 - 17$ ps/(nm·km) at 1550 nm
- NZDSF: $D = 2 - 6$ ps/(nm·km) at 1550 nm
- Dispersion-shifted fiber (DSF): $D \approx 0$ at 1550 nm (zero-dispersion wavelength)

GNPy Implementation

Dispersion Accumulation:

```

1 class Fiber(Element):
2     def chromatic_dispersion(self, freq=None):
3         """Accumulated chromatic dispersion [s/m]
4
5         Returns the total accumulated CD at the fiber output
6         """
7         freq = self.params.ref_frequency if freq is None else freq
8         beta2 = self.beta2(freq)
9         beta3 = self.beta3(freq)
10        ref_f = self.params.ref_frequency
11        length = self.params.length
12
13        # Taylor expansion around reference frequency
14        beta = beta2 + 2 * pi * beta3 * (freq - ref_f)
15
16        # Convert to dispersion parameter
17        dispersion = -beta * 2 * pi * ref_f**2 / c
18
19        return dispersion * length
20
21    def beta2(self, frequency):
22        """Group velocity dispersion parameter [s^2/m]"""
23        if self.params.beta2 is not None:
24            # Beta2 specified directly
25            return self.params.beta2
26        else:
27            # Compute from dispersion parameter D
28            wavelength = c / frequency
29            D = self.params.dispersion # ps/(nm.km)
30            beta2 = -D * wavelength**2 / (2 * pi * c)
31            return beta2 * 1e-6 # Convert to s^2/m
32
33    def propagate(self, spectral_info: SpectralInformation):
34        # ... attenuation code ...
35
36        # Accumulate chromatic dispersion
37        spectral_info.chromatic_dispersion += \
38            self.chromatic_dispersion(spectral_info.frequency)

```

Key Implementation Features:

1. **Dual specification:** CD can be specified using either:
 - Dispersion parameter D (ps/(nm·km))
 - GVD parameter β_2 (s²/m)
2. **Dispersion slope:** Third-order dispersion β_3 accounts for wavelength-dependent dispersion
3. **Per-channel tracking:** CD accumulates separately for each wavelength channel in `SpectralInformation`
4. **Penalty computation:** The `Transceiver` class applies CD-induced OSNR penalties at the receiver

Configuration Parameters:

Parameter	Default (SSMF)	Description
dispersion	1.67×10^{-5} or $16.7 \text{ ps}/(\text{nm}\cdot\text{km})$	CD parameter (s/(m·m))
dispersion_slope	1.0×10^{-7}	Dispersion slope (s/(m·m·m))
beta2	-	Alternative: GVD (s ² /m)
beta3	-	Alternative: TOD (s ³ /m)
ref_frequency	193.1 THz	Reference for Taylor expansion

Table 1.2: Chromatic dispersion parameters in GNPpy

1.2.3 Polarization Mode Dispersion*Physical Phenomenon*

Polarization mode dispersion (PMD) arises from birefringence in optical fiber. Ideally circular fiber has random birefringence due to:

- Core ellipticity from manufacturing imperfections
- Asymmetric stress from cable design
- Environmental factors (temperature, mechanical stress)

The two orthogonal polarization modes travel at slightly different velocities, causing pulse spreading. PMD is a stochastic phenomenon with random polarization coupling along the fiber.

Mathematical Model

PMD is characterized statistically. For fiber length L , the mean differential group delay (DGD) is:

$$\langle \Delta\tau \rangle = D_{PMD} \sqrt{L} \quad (1.7)$$

where D_{PMD} is the PMD coefficient (ps/ $\sqrt{\text{km}}$).

The DGD probability distribution follows a Maxwellian distribution:

$$p(\Delta\tau) = \frac{32}{\pi^2} \frac{\Delta\tau^2}{\langle \Delta\tau \rangle^3} \exp\left(-\frac{4\Delta\tau^2}{\pi \langle \Delta\tau \rangle^2}\right) \quad (1.8)$$

OSNR Penalty:

For systems without PMD compensation, the OSNR penalty is approximately:

$$\text{Penalty}_{PMD} \approx 10 \log_{10} \left(1 + \left(\frac{\pi \Delta\tau}{2T_s} \right)^2 \right) \quad [\text{dB}] \quad (1.9)$$

where $T_s = 1/B$ is the symbol period.

Typical Values:

- Modern fiber: $D_{PMD} = 0.05 - 0.1 \text{ ps}/\sqrt{\text{km}}$
- Older fiber: $D_{PMD} = 0.5 - 2 \text{ ps}/\sqrt{\text{km}}$
- ITU-T G.652 specification: $D_{PMD} < 0.5 \text{ ps}/\sqrt{\text{km}}$

GNPpy Implementation

```

1 class Fiber(Element):
2     def propagate(self, spectral_info: SpectralInformation):
3         # ... attenuation and CD code ...
4
5         # Accumulate PMD (RMS addition)

```

```

6         pmd_fiber = self.params.pmd_coef * sqrt(self.params.length /
7           1000)
8         spectral_info.pmd = sqrt(spectral_info.pmd**2 + pmd_fiber**2)
9
10    class Transceiver(Element):
11        def _calc_snr(self, spectral_info: SpectralInformation):
12            """Calculate penalties and required OSNR"""
13
14            # PMD penalty
15            symbol_rate = spectral_info.baud_rate
16            pmd_value = spectral_info.pmd
17            penalty_pmd = self.pmd_penalty(pmd_value, symbol_rate)
18
19            # Total penalty applied to required OSNR
20            osnr_required = self.mode.osnr + penalty_pmd + ...
21
22        def pmd_penalty(self, pmd, symbol_rate):
23            """Compute PMD-induced penalty [dB]"""
24            if pmd == 0:
25                return 0
26
27            symbol_period = 1 / symbol_rate
28            normalized_pmd = pi * pmd / (2 * symbol_period)
29
30            penalty = 10 * log10(1 + normalized_pmd**2)
31            return penalty

```

Key Implementation Details:

1. **Statistical accumulation:** PMD accumulates as root-sum-square (RSS) across fiber spans
2. **Transceiver penalty:** PMD affects receiver sensitivity, applied as OSNR penalty
3. **ROADM contribution:** ROADMs can add PMD through filters and switches

1.2.4 Polarization Dependent Loss

Physical Phenomenon

Polarization dependent loss (PDL) occurs when optical components exhibit different transmission/reflection for different polarization states. Sources include:

- Optical isolators and circulators
- Thin-film filters in ROADMs
- Fiber connectors and splices
- Wavelength selective switches (WSS)

Mathematical Model

PDL is defined as the ratio of maximum to minimum transmission:

$$\text{PDL} = 10 \log_{10} \left(\frac{T_{\max}}{T_{\min}} \right) \quad [\text{dB}] \quad (1.10)$$

For cascaded elements with PDL values $\text{PDL}_1, \text{PDL}_2, \dots, \text{PDL}_N$, the total PDL is:

$$\text{PDL}_{\text{total}} = \sqrt{\text{PDL}_1^2 + \text{PDL}_2^2 + \dots + \text{PDL}_N^2} \quad (1.11)$$

OSNR Penalty:

The statistical OSNR penalty due to PDL is approximately:

$$\text{Penalty}_{\text{PDL}} \approx \frac{\text{PDL}^2}{8 \ln(2)} \quad [\text{dB}] \quad (1.12)$$

Typical Values:

- Fiber span: PDL < 0.2 dB
- ROADM node: PDL = 0.3-0.8 dB
- Optical amplifier: PDL < 0.5 dB

GNPy Implementation

```

1 class Fiber(Element):
2     def propagate(self, spectral_info: SpectralInformation):
3         # PDL accumulates as RSS
4         pdl_fiber = self.params.pdl
5         spectral_info.pdl = sqrt(spectral_info.pdl**2 + pdl_fiber**2)
6
7 class Roadm(Element):
8     def propagate(self, spectral_info: SpectralInformation):
9         # ROADM PDL contribution
10        pdl_roadm = self.params.pdl
11        spectral_info.pdl = sqrt(spectral_info.pdl**2 + pdl_roadm**2)
12
13 class Transceiver(Element):
14     def pdl_penalty(self, pdl):
15         """PDL-induced OSNR penalty [dB]"""
16         if pdl == 0:
17             return 0
18         penalty = pdl**2 / (8 * log(2))
19         return penalty

```

1.3. NONLINEAR FIBER IMPAIRMENTS

Nonlinear effects become significant at high optical powers and cause signal-dependent distortions. Unlike linear impairments, nonlinear effects increase with launched power and channel count, fundamentally limiting system capacity.

1.3.1 Theoretical Foundation: Nonlinear Schrödinger Equation

Signal propagation in optical fiber is governed by the nonlinear Schrödinger equation (NLSE):

$$\frac{\partial A}{\partial z} = -\frac{\alpha}{2}A - i\frac{\beta_2}{2}\frac{\partial^2 A}{\partial t^2} + i\gamma|A|^2A \quad (1.13)$$

where:

- $A(z, t)$ = slowly-varying envelope of the electric field
- α = fiber attenuation (Np/m)
- β_2 = group velocity dispersion (s^2/m)
- γ = nonlinear coefficient ($\text{W}^{-1}\cdot\text{m}^{-1}$)

The nonlinear coefficient is:

$$\gamma = \frac{2\pi n_2}{\lambda A_{\text{eff}}} \quad (1.14)$$

where:

- $n_2 = 2.6 \times 10^{-20} \text{ m}^2/\text{W}$ = nonlinear refractive index of silica
- A_{eff} = effective mode area (typically $80 \mu\text{m}^2$ for SSMF)
- λ = wavelength

Typical Value: $\gamma \approx 1.2 \text{ W}^{-1}\cdot\text{km}^{-1}$ for SSMF at 1550 nm

1.3.2 Self-Phase Modulation (SPM)

Physical Phenomenon

SPM arises from the intensity-dependent refractive index. A channel's own power modulates its phase, causing spectral broadening and interaction with chromatic dispersion.

The instantaneous phase shift is:

$$\phi_{SPM}(z, t) = \gamma |A(t)|^2 L_{eff} \quad (1.15)$$

where $L_{eff} = \frac{1-e^{-\alpha L}}{\alpha}$ is the effective fiber length.

Impact on System Performance

SPM alone does not directly degrade signal quality (it's a deterministic effect), but its interaction with chromatic dispersion causes:

- Pulse distortion
- Timing jitter
- Effective spectral broadening

1.3.3 Cross-Phase Modulation (XPM)

Physical Phenomenon

XPM occurs when one channel's intensity modulates the phase of another channel. For channel i affected by channel j :

$$\phi_{XPM,i}(z, t) = 2\gamma |A_j(t)|^2 L_{eff} \quad (1.16)$$

The factor of 2 arises from the overlap of orthogonal polarizations.

Key Characteristics:

- Depends on walk-off between channels (due to dispersion)
- More significant for closely spaced channels
- Reduced by channel randomization (different modulation formats)

1.3.4 Four-Wave Mixing (FWM)

Physical Phenomenon

FWM is a parametric process where three waves at frequencies f_i, f_j, f_k generate a fourth wave at:

$$f_{FWM} = f_i + f_j - f_k \quad (1.17)$$

Phase-Matching Condition:

FWM efficiency depends on the phase mismatch:

$$\Delta\beta = \beta(f_i) + \beta(f_j) - \beta(f_k) - \beta(f_{FWM}) \quad (1.18)$$

FWM is most efficient when:

- Channels are equally spaced
- Low dispersion (small $\Delta\beta$)
- High power

Mitigation:

- Use fibers with moderate dispersion (NZDSF)
- Unequal channel spacing
- Reduce launched power

1.3.5 The Gaussian Noise Model

Model Foundation

The Gaussian Noise (GN) model treats nonlinear interference (NLI) as additive Gaussian noise. This is valid when:

- Many interfering channels
- Channels have random phases
- System is dispersion-managed or has sufficient dispersion

Core Equation:

The NLI power spectral density for channel i at frequency f_i is:

$$G_{NLI}(f_i) = \frac{16}{27} \gamma^2 L_{eff}^2 \iiint G_i(f_1) G_j(f_2) G_k(f_3) \frac{\rho(f_1, f_2, f_3)}{\eta(f_1, f_2, f_3)} df_1 df_2 df_3 \quad (1.19)$$

where:

- $G_i(f)$ = power spectral density of channel i
- ρ = efficiency factor (depends on phase matching)
- η = coherence factor (depends on dispersion)

Analytical Approximation (GN Model)

Poggiolini derived an analytical closed-form solution (equation 120 from arXiv:1209.0394):

$$P_{NLI,i} = \eta_i P_i \sum_j P_j^2 B_i \quad (1.20)$$

where:

- η_i = nonlinear efficiency matrix
- P_i = power of channel i
- P_j = power of interfering channel j
- B_i = bandwidth of channel i

The efficiency η accounts for:

- Fiber nonlinearity (γ^2)
- Effective length (L_{eff}^2)
- Phase-matching conditions (via arcsinh terms)

GNPy Implementation: NliSolver Class

```

1 class NliSolver:
2     """Nonlinear Interference Solver
3
4     Implements GN model and its extensions (GGN) for computing
5     nonlinear interference noise power.
6     """
7
8     @staticmethod
9     def compute_nli(spectral_info, fiber, srs):
10         """Main entry point for NLI calculation
11
12         Args:
13             spectral_info: Channel parameters (power, frequency, baud
14                             rate)
15             fiber: Fiber parameters (gamma, beta2, length, ...)
16             srs: Raman scattering profile
17
18         Returns:
19             nli: NLI power per channel [W]
20         """

```

```

20     method = sim_params.nli_params.method
21
22     if 'gn_model_analytic' in method:
23         # Closed-form GN model (fast, less accurate)
24         eta = NliSolver._gn_analytic(spectral_info, fiber)
25
26     elif 'ggn_spectrally_separated' in method:
27         # GGN model with spectral separation (accurate)
28         # Compute only for representative channels
29         cut_indices = sim_params.nli_params.computed_channels
30         eta = NliSolver._ggn_spectrally_separated(
31             cut_indices, spectral_info, fiber, srs)
32
33     # Apply power to eta matrix
34     nli = self._apply_power_to_eta(eta, spectral_info)
35
36     return nli
37
38     @staticmethod
39     def _gn_analytic(spectral_info, fiber,
40                     spm_weight=1.0, xpm_weight=1.0):
41         """Analytical GN model (equation 120, arXiv:1209.0394)
42
43         Computes eta matrix using closed-form expressions
44         """
45         # Extract spectral parameters
46         nch = spectral_info.number_of_channels
47         frequency = spectral_info.frequency
48         baud_rate = spectral_info.baud_rate
49         delta_freq = spectral_info.df # Channel spacing
50
51         # Fiber parameters
52         alpha = fiber.alpha(frequency)
53         beta2 = fiber.beta2(frequency)
54         gamma = outer(fiber.gamma(frequency), ones(nch))
55         length = fiber.params.length
56
57         # Effective and asymptotic lengths
58         effective_length = (1 - exp(-alpha * length)) / alpha
59         asymptotic_length = 1 / alpha
60
61         # Build weight matrix: SPM on diagonal, XPM off-diagonal
62         identity = diag(ones(nch))
63         weight = spm_weight * identity + \
64             xpm_weight * (ones([nch, nch]) - identity)
65
66         # Compute psi function (phase-matching integral)
67         psi = NliSolver._psi(delta_freq, baud_rate, beta2,
68                             effective_length, asymptotic_length)
69
70         # Nonlinear efficiency matrix
71         cut_baud_rate = outer(baud_rate, ones(nch))
72         pump_baud_rate = outer(ones(nch), baud_rate)
73
74         eta = gamma**2 * weight * psi / \
75             (cut_baud_rate * pump_baud_rate**2)
76
77     return eta

```

```

78
79 @staticmethod
80 def _psi(df, baud_rate, beta2, L_eff, L_a):
81     """Psi function: phase-matching efficiency
82
83     Implements equation 123 from arXiv:1209.0394
84
85     Args:
86         df: Frequency separation matrix
87         baud_rate: Channel baud rates
88         beta2: GVD parameter
89         L_eff: Effective fiber length
90         L_a: Asymptotic fiber length (1/alpha)
91
92     Returns:
93         psi: Phase-matching efficiency matrix
94     """
95     # Build frequency and baud rate matrices
96     cut_baud = outer(baud_rate, ones(baud_rate.size))
97     pump_baud = outer(ones(baud_rate.size), baud_rate)
98
99     cut_beta = outer(beta2, ones(baud_rate.size))
100    pump_beta = outer(ones(baud_rate.size), beta2)
101    beta2_avg = (cut_beta + pump_beta) / 2
102
103    # Integration limits for pump channel
104    right_extreme = df + pump_baud / 2
105    left_extreme = df - pump_baud / 2
106
107    # arcsinh integration (handles phase matching)
108    arg_right = pi**2 * L_a * abs(beta2_avg) * \
109                cut_baud * right_extreme
110    arg_left = pi**2 * L_a * abs(beta2_avg) * \
111                cut_baud * left_extreme
112
113    psi = (arcsinh(arg_right) - arcsinh(arg_left)) / 2
114    psi *= L_eff**2 / (2 * pi * abs(beta2_avg) * L_a)
115
116    return psi
117
118 @staticmethod
119 def _ggn_spectrally_separated(cut_indices, spectral_info,
120                                fiber, srs):
121     """Enhanced GGN model with spectral separation
122
123     More accurate than GN model, accounts for:
124     - Detailed spectral profile of channels
125     - Raman effect on power distribution
126     - Frequency-dependent fiber parameters
127
128     Computes eta only for representative channels (cut_indices)
129     and interpolates for others.
130     """
131     # Implementation details in science_utils.py
132     # Uses equation 21 from arXiv:1710.02225
133     ...

```

Key Implementation Aspects:

1. **Method selection:** Multiple NLI models available
 - `gn_model_analytic`: Fast, closed-form GN model
 - `ggn_spectrally_separated`: Accurate GGN with spectral profiles
 - `ggn_approx`: Approximate GGN (balance of speed/accuracy)
2. **Computational optimization:**
 - Eta matrix is $N_{ch} \times N_{ch}$ (computationally expensive)
 - For large systems, compute only for representative channels
 - Interpolate results for other channels
3. **SPM/XPM weighting:**
 - SPM weight = 1.0 (diagonal terms)
 - XPM weight = 1.0 (off-diagonal terms)
 - Can be adjusted for different modulation formats
4. **Integration with Raman:** SRS affects power profile, which affects NLI calculation

1.3.6 Stimulated Raman Scattering

Physical Phenomenon

Stimulated Raman scattering (SRS) is an inelastic scattering process where photons transfer energy to molecular vibrations in silica. This causes:

- Power transfer from shorter (higher frequency) to longer (lower frequency) wavelengths
- Gain tilt across the WDM spectrum
- Modified effective fiber loss for each channel

Raman Gain:

The Raman gain coefficient peaks at 13 THz frequency offset:

$$g_R(\Delta f) = g_{R,peak} \cdot g_{R,shape}(\Delta f) \quad (1.21)$$

where $g_{R,peak} \approx 1 \times 10^{-13}$ m/W for silica.

Coupled Power Equations

For N channels, SRS is described by coupled equations:

$$\frac{dP_i}{dz} = -\alpha_i P_i + \sum_{j \neq i} g_R(f_i - f_j) P_i P_j \quad (1.22)$$

GNPy Implementation: RamanSolver Class

```

1 class RamanSolver:
2     """Raman scattering solver
3
4     Computes power evolution due to stimulated Raman scattering
5     """
6
7     @staticmethod
8     def calculate_stimulated_raman_scattering(spectral_info, fiber):
9         """Main SRS calculation
10
11         Returns:
12             SRS profile containing:
13             - Power evolution along fiber
14             - Loss profile per channel
15             - Gain/loss due to Raman
16         """
17         # Initialize spatial grid
18         z_array = linspace(0, fiber.params.length,
```

```

19         sim_params.raman_params.
20             solver_spatial_resolution)
21
22     # Initialize power profiles
23     power_profile = zeros((spectral_info.number_of_channels,
24                           len(z_array)))
25     power_profile[:, 0] = spectral_info.signal # Initial powers
26
27     # Raman gain profile
28     raman_gain = RamanSolver._raman_gain_profile(
29         spectral_info.frequency)
30
31     # Solve coupled equations
32     if sim_params.raman_params.method == 'perturbative':
33         # First-order perturbation (fast)
34         power_profile = RamanSolver._raman_perturbation(
35             spectral_info, fiber, z_array, raman_gain)
36     else:
37         # Full numerical solution (accurate)
38         power_profile = RamanSolver._raman_numerical(
39             spectral_info, fiber, z_array, raman_gain)
40
41     # Compute effective loss per channel
42     loss_profile = -power_profile / \
43         power_profile[:, 0][:, newaxis]
44
45     return SRSPProfile(power_profile, loss_profile, z_array)
46
47 @staticmethod
48 def _raman_gain_profile(frequency):
49     """Compute Raman gain spectrum
50
51     Returns matrix g_R[i,j] = gain at freq_i due to freq_j
52     """
53     nch = len(frequency)
54     g_R = zeros((nch, nch))
55
56     for i in range(nch):
57         for j in range(nch):
58             delta_f = frequency[i] - frequency[j]
59
60             if delta_f > 0: # i receives gain from j
61                 # Lorentzian-like Raman gain shape
62                 g_R[i, j] = RamanSolver._lorentzian_raman(
63                     delta_f, peak_freq=13e12, width=6e12)
64
65     return g_R
66
67 @staticmethod
68 def _raman_perturbation(spectral_info, fiber, z, g_R):
69     """First-order perturbative solution to Raman equations
70
71     Assumes: Raman effect << attenuation
72     Fast, suitable for typical C-band systems
73     """
74     nch = spectral_info.number_of_channels
75     alpha = fiber.alpha(spectral_info.frequency)
76     length = fiber.params.length

```

```

76
77     # Attenuation profile
78     power = outer(spectral_info.signal, exp(-alpha * z))
79
80     # Raman correction (first order)
81     for i in range(nch):
82         raman_correction = 0
83         for j in range(nch):
84             if i != j:
85                 # Raman interaction i <- j
86                 L_eff_raman = (1 - exp(-(alpha[i] + alpha[j]) * z))
87                     / \
88                         (alpha[i] + alpha[j])
89                 raman_correction += g_R[i,j] * \
90                     spectral_info.signal[j] * \
91                     L_eff_raman
92
93         power[i, :] *= exp(raman_correction)
94
95     return power

```

Integration with NLI Calculation:

The Raman power profile affects NLI computation:

```

1 class Fiber(Element):
2     def propagate(self, spectral_info: SpectralInformation):
3         # Compute Raman profile
4         if sim_params.raman_params.flag:
5             srs = RamanSolver.calculate_stimulated_raman_scattering(
6                 spectral_info, self)
7         else:
8             srs = None
9
10        # Compute NLI using Raman-modified powers
11        nli = NliSolver.compute_nli(spectral_info, self, srs)
12
13        # Apply attenuation from Raman profile
14        if srs is not None:
15            spectral_info.apply_attenuation_lin(srs.loss_profile[:,
16                -1])
17        else:
18            # Simple exponential attenuation
19            loss_db = self.params.loss_coef * self.params.length / 1000
20            spectral_info.apply_attenuation_db(loss_db)
21
22        # Add NLI noise
23        spectral_info.nli += nli

```

1.4. AMPLIFIER IMPAIRMENTS

1.4.1 Amplified Spontaneous Emission (ASE) Noise

Physical Phenomenon

Erbium-doped fiber amplifiers (EDFAs) provide gain through stimulated emission. However, spontaneous emission also occurs, generating broadband optical noise that accumulates through the network.

Noise Figure:

The EDFA noise figure characterizes noise addition:

$$NF = \frac{SNR_{in}}{SNR_{out}} = \frac{2n_{sp}(G-1)}{G} \quad (1.23)$$

where:

- G = amplifier gain (linear)
 - n_{sp} = spontaneous emission factor ($n_{sp} \geq 1$)
- For ideal amplifier, $n_{sp} = 1$ giving $NF_{ideal} = 3$ dB.

ASE Power Spectral Density

The ASE power in bandwidth Δf is:

$$P_{ASE} = n_{sp}h\nu(G-1)\Delta f \quad (1.24)$$

where:

- h = Planck's constant
- ν = optical frequency
- Δf = noise bandwidth (typically 12.5 GHz for OSNR measurement)

GNPy Implementation: EDFA Class

```

1 class Edfa(Element):
2     """Erbium-Doped Fiber Amplifier
3
4     Supports multiple amplifier models:
5     - variable_gain: NF depends on gain
6     - fixed_gain: Constant gain
7     - advanced_model: Detailed spectral characteristics
8     - openroadm: OpenROADM MSA compliant
9     """
10
11     def propagate(self, spectral_info: SpectralInformation):
12         """Apply amplifier gain and add ASE noise"""
13
14         # 1. Determine target gain
15         if self.operational.gain_target:
16             gain = self.operational.gain_target
17         else:
18             gain = self.compute_gain_target(spectral_info)
19
20         # 2. Compute noise figure
21         nf = self._calc_nf(spectral_info.frequency, gain)
22
23         # 3. Apply gain
24         spectral_info.apply_gain_lin(db2lin(gain))
25
26         # 4. Add ASE noise
27         ase_power = self._calc_ase(gain, nf, spectral_info.frequency)
28         spectral_info.ase += ase_power
29
30     def _calc_nf(self, frequency, gain):
31         """Calculate noise figure
32
33         Depends on amplifier type and configuration
34         """
35         if self.params.type_def == 'variable_gain':
36             # Polynomial NF model: NF = a + b*G + c*G^2
37             nf_min = self.params.nf_min

```

```

38         nf_max = self.params.nf_max
39         gain_flatmax = self.params.gain_flatmax
40
41         # Interpolate NF based on gain
42         nf = nf_min + (nf_max - nf_min) * \
43             (gain / gain_flatmax)**2
44
45     elif self.params.type_def == 'fixed_gain':
46         # Constant NF
47         nf = self.params.nf0
48
49     elif self.params.type_def == 'advanced_model':
50         # Load detailed NF ripple from file
51         nf = self._nf_ripple(frequency, gain)
52
53     return nf
54
55 def _calc_ase(self, gain, nf, frequency):
56     """Calculate ASE power per channel
57
58     Args:
59         gain: Amplifier gain [dB]
60         nf: Noise figure [dB]
61         frequency: Channel frequencies [Hz]
62
63     Returns:
64         ASE power per channel [W]
65     """
66     # Convert to linear
67     gain_lin = db2lin(gain)
68     nf_lin = db2lin(nf)
69
70     # ASE formula
71     #  $P_{ASE} = (G - 1) * NF * h * \nu * \Delta_f / 2$ 
72     # Factor of 2: ASE in both polarizations
73
74     h = 6.62607015e-34 # Planck's constant
75     delta_f = 12.5e9 # Reference bandwidth (12.5 GHz)
76
77     # Per-channel ASE
78     ase = (gain_lin - 1) * nf_lin * h * frequency * delta_f / 2
79
80     return ase
81
82 def compute_gain_target(self, spectral_info):
83     """Auto-design: determine required gain
84
85     For amplifier auto-design, target gain compensates:
86     - Previous span loss
87     - Target output power
88     """
89     # Get previous element loss
90     prev_loss = spectral_info.path_loss[-1] # dB
91
92     # Target output power
93     p_target = self.operational.power_target
94
95     # Current power

```

```

96     p_current = watt2dbm(mean(spectral_info.signal))
97
98     # Required gain
99     gain_target = prev_loss + (p_target - p_current)
100
101     # Clip to amplifier limits
102     gain_target = clip(gain_target,
103                       self.params.gain_min,
104                       self.params.gain_flatmax)
105
106     return gain_target

```

Advanced Model Features:

For high-accuracy simulations, GNPy supports vendor-specific amplifier models:

```

1 class Edfa(Element):
2     def _nf_ripple(self, frequency, gain):
3         """Load NF ripple from advanced configuration
4
5         Advanced config includes:
6         - NF ripple vs. wavelength (96 channels)
7         - Gain ripple vs. wavelength
8         - Dynamic gain tilt
9         - Temperature dependence
10        """
11        # Load from advanced_config.json
12        config = self.advanced_config
13
14        # Interpolate NF at channel frequencies
15        nf_base = config['nf_coef'](gain)
16        nf_ripple = interp(frequency,
17                          config['nf_ripple_freq'],
18                          config['nf_ripple_values'])
19
20        return nf_base + nf_ripple
21
22    def _gain_ripple(self, frequency):
23        """Gain flatness vs. wavelength"""
24        config = self.advanced_config
25        return interp(frequency,
26                    config['gain_ripple_freq'],
27                    config['gain_ripple_values'])
28
29    def _dynamic_gain_tilt(self, frequency, delta_power):
30        """Dynamic gain tilt
31
32        When input power deviates from nominal,
33        gain spectrum tilts
34
35        Args:
36            delta_power: Power deviation from nominal [dB]
37        """
38        config = self.advanced_config
39        dgt = config['dgt'] # dB/dB
40
41        # Tilt = DGT * delta_power
42        tilt = dgt * delta_power
43
44        return tilt

```

1.4.2 Noise Power in Different Bandwidths

For QoT assessment and comparison with measurements, noise powers must be calculated in specific reference bandwidths.

Standard Reference Bandwidths

0.1 nm Reference Bandwidth:

The industry-standard OSNR measurement uses 0.1 nm (12.5 GHz at 1550 nm):

$$B_{0.1nm} = \frac{c}{\lambda^2} \cdot 0.1 \times 10^{-9} \approx 12.5 \text{ GHz} \quad (1.25)$$

Signal Bandwidth:

The actual signal bandwidth depends on baud rate and modulation format:

$$B_{sig} = R_s \text{ [Hz]} \quad (1.26)$$

where R_s is the symbol rate (baud rate).

ASE Power in Different Bandwidths

From the ASE power spectral density ρ_{ASE} (W/Hz):

$$P_{ASE,0.1nm} = \rho_{ASE} \cdot B_{0.1nm} \quad (1.27)$$

$$P_{ASE,sig} = \rho_{ASE} \cdot B_{sig} \quad (1.28)$$

NLI Power in Different Bandwidths

Similarly, from NLI power spectral density ρ_{NLI} :

$$P_{NLI,0.1nm} = \rho_{NLI} \cdot B_{0.1nm} \quad (1.29)$$

$$P_{NLI,sig} = \rho_{NLI} \cdot B_{sig} \quad (1.30)$$

Combined Noise Power

Total noise combines ASE and NLI contributions:

$$P_{noise,0.1nm} = P_{ASE,0.1nm} + P_{NLI,0.1nm} \quad (1.31)$$

$$P_{noise,sig} = P_{ASE,sig} + P_{NLI,sig} \quad (1.32)$$

These bandwidth-specific noise powers are used in Chapter 5 to calculate OSNR and SNR metrics for different reference bandwidths.

1.4.3 Gain Spectral Tilt

EDFAs exhibit wavelength-dependent gain. Without proper equalization, this causes power non-uniformity across channels.

Causes:

- Erbium emission spectrum shape
- Input power variations
- Temperature effects

GNPy Modeling:

```

1 def apply_gain_ripple(self, spectral_info):
2     """Apply frequency-dependent gain"""
3     gain_ripple = self._gain_ripple(spectral_info.frequency)

```

```

4
5     # Apply gain + ripple
6     total_gain = self.operational.gain_target + gain_ripple
7     spectral_info.apply_gain_db(total_gain)

```

1.5. ROADM IMPAIRMENTS

1.5.1 Optical Filtering

Physical Phenomenon

ROADMs use wavelength selective switches (WSS) based on liquid crystal or MEMS technology. These devices apply:

- Optical filtering (typically Gaussian or rectangular)
- Concatenated filtering through multiple ROADM nodes
- Filter narrowing effect (cascaded filters → narrower effective bandwidth)

Mathematical Model

For Gaussian filters, the transfer function is:

$$H(f) = \exp\left(-\frac{(f - f_0)^2}{2\sigma^2}\right) \quad (1.33)$$

Cascaded through N ROADMs:

$$H_{\text{total}}(f) = \prod_{i=1}^N H_i(f) = \exp\left(-\frac{(f - f_0)^2}{2\sigma_{\text{eff}}^2}\right) \quad (1.34)$$

where $\sigma_{\text{eff}} = \sigma/\sqrt{N}$ (filter narrows with cascading).

GNPy Implementation

```

1 class Roadm(Element):
2     """Reconfigurable Optical Add-Drop Multiplexer
3
4     Models:
5     - Pass-through loss
6     - Add/drop OSNR degradation
7     - Optical filtering
8     - Crosstalk
9     """
10
11     def propagate(self, spectral_info: SpectralInformation):
12         """Apply ROADM effects"""
13
14         # 1. Determine path type (express/add/drop)
15         path_type = self.get_path_type(spectral_info)
16
17         # 2. Apply path loss
18         if path_type == 'express':
19             loss = self.params.loss_express
20         elif path_type == 'add':
21             loss = self.params.loss_add
22         else: # drop
23             loss = self.params.loss_drop
24
25         spectral_info.apply_attenuation_db(loss)
26
27         # 3. Add path OSNR degradation

```

```

28     if path_type in ['add', 'drop']:
29         self._add_roadm_ase(spectral_info)
30
31     # 4. PDL and PMD accumulation
32     spectral_info.pdl = sqrt(spectral_info.pdl**2 +
33                             self.params.pdl**2)
34     spectral_info.pmd = sqrt(spectral_info.pmd**2 +
35                             self.params.pmd**2)
36
37     def _add_roadm_ase(self, spectral_info):
38         """Add OSNR degradation from ROADM add/drop
39
40         ROADMs degrade OSNR through:
41         - Imperfect filtering (ASE leakage)
42         - Amplifier stages in add/drop modules
43         """
44         # Add/drop OSNR specification
45         osnr_penalty = self.params.add_drop_osnr # dB
46
47         # Convert to equivalent ASE power
48         # This ASE degrades the signal OSNR
49         signal_power = spectral_info.signal
50         ase_equivalent = signal_power / db2lin(osnr_penalty)
51
52         spectral_info.ase += ase_equivalent
53
54     def equalize_channels(self, spectral_info):
55         """ROADM channel power equalization
56
57         ROADMs can adjust per-channel attenuation
58         to maintain uniform power
59         """
60         if self.params.equalization_enabled:
61             target_power = self.params.target_pch_out_db
62             current_power = watt2dbm(spectral_info.signal)
63
64             # Per-channel VOA settings
65             delta_power = target_power - current_power
66             spectral_info.apply_gain_db(delta_power)

```

1.5.2 Inter-Channel Crosstalk

Crosstalk occurs when adjacent channel power leaks through imperfect filtering in WSS.

Model:

Crosstalk power from channel j to channel i :

$$P_{\text{XT},i} = \sum_{j \neq i} X_{ij} P_j \quad (1.35)$$

where X_{ij} is the crosstalk coefficient (typically -30 to -40 dB).

GNPy Implementation:

Crosstalk is typically lumped into the ROADM OSNR penalty rather than modeled explicitly for computational efficiency.

1.6. IMPLEMENTATION WORKFLOW

1.6.1 Signal Propagation Architecture

GNPy propagates a **SpectralInformation** object through the network, accumulating impairments at each element.

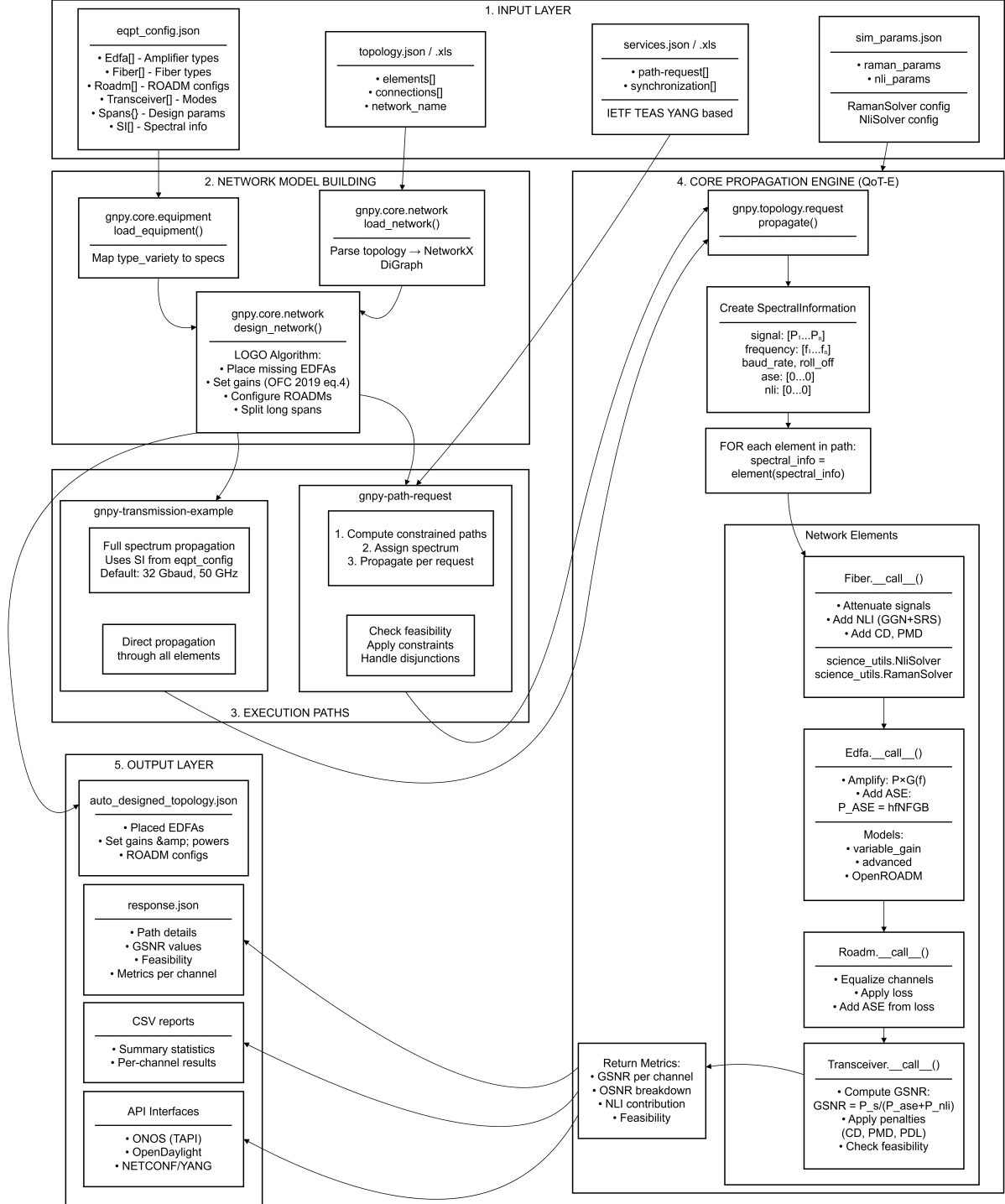


Figure 1.3: GNPy processing workflow from input configuration to QoT estimation. Shows the five-layer architecture: input layer, network model building, execution paths, core propagation engine, and output layer.

1.6.2 SpectralInformation Class

```

1 class SpectralInformation:
2     """Container for WDM spectrum parameters
3
4     Tracks per-channel:
5     - Signal power
6     - ASE noise
7     - NLI noise
8     - Chromatic dispersion
9     - PMD, PDL
10    - Latency
11    """
12
13    def __init__(self, frequency, baud_rate, power, ...):
14        self.frequency = frequency # Hz, array
15        self.baud_rate = baud_rate # Baud, array
16
17        # Power components [W]
18        self.signal = power
19        self.ase = zeros_like(frequency)
20        self.nli = zeros_like(frequency)
21
22        # Accumulated impairments
23        self.chromatic_dispersion = zeros_like(frequency) # s/m
24        self.pmd = 0 # s (RSS accumulation)
25        self.pdl = 0 # dB (RSS accumulation)
26
27        # Path tracking
28        self.path_loss = [] # List of span losses
29        self.visited_elements = []
30
31    def apply_attenuation_db(self, loss_db):
32        """Apply loss to all power components"""
33        loss_lin = db2lin(loss_db)
34        self.signal /= loss_lin
35        self.ase /= loss_lin
36        self.nli /= loss_lin
37
38    def apply_gain_lin(self, gain_lin):
39        """Apply gain (linear)"""
40        self.signal *= gain_lin
41        self.ase *= gain_lin
42        self.nli *= gain_lin
43
44    @property
45    def osnr(self):
46        """Calculate OSNR per channel [dB]"""
47        signal_power = self.signal
48        noise_power = self.ase + self.nli
49
50        # OSNR in 0.1 nm (12.5 GHz) bandwidth
51        osnr_lin = signal_power / noise_power
52        osnr_db = 10 * log10(osnr_lin)
53
54        return osnr_db
55
56    @property

```

```

57 def snr(self):
58     """Calculate SNR per channel [dB]
59
60     SNR in signal bandwidth (different from OSNR)
61     """
62     signal_power = self.signal
63     noise_bw = self.baud_rate # Signal bandwidth
64     noise_ref_bw = 12.5e9 # OSNR reference bandwidth
65
66     noise_power = (self.ase + self.nli) * \
67                 (noise_bw / noise_ref_bw)
68
69     snr_lin = signal_power / noise_power
70     snr_db = 10 * log10(snr_lin)
71
72     return snr_db

```

1.6.3 Element Propagation Chain

```

1 class Element:
2     """Base class for all network elements"""
3
4     def propagate(self, spectral_info: SpectralInformation):
5         """Propagate spectral information through element
6
7         Each element type overrides this method
8         """
9         raise NotImplementedError
10
11 # Network propagation
12 def propagate_path(network, path, spectral_info):
13     """Propagate through a list of network elements
14
15     Args:
16         network: Network object
17         path: List of element UIDs
18         spectral_info: Initial spectral information
19
20     Returns:
21         Final spectral_info with accumulated impairments
22     """
23     for element_uid in path:
24         element = network.get_element(element_uid)
25
26         # Propagate through this element
27         element.propagate(spectral_info)
28
29         # Track element in path
30         spectral_info.visited_elements.append(element_uid)
31
32     return spectral_info

```

1.6.4 Typical Propagation Sequence

```

1 # Example: Propagation through 3-span system
2 # Tx -> Fiber -> EDFA -> Fiber -> EDFA -> Fiber -> EDFA -> Rx
3
4 # 1. Initialize at transmitter

```

```

5 spectral_info = SpectralInformation(
6     frequency=freq_array,
7     baud_rate=baud_array,
8     power=tx_power,
9     tx_osnr=40.0 # dB, transmitter OSNR
10 )
11
12 # 2. Span 1: Fiber
13 fiber1.propagate(spectral_info)
14 # - Applies attenuation (signal, ASE, NLI all reduced)
15 # - Accumulates CD, PMD, PDL
16 # - Computes and adds NLI noise
17 # - Applies Raman tilt (if enabled)
18
19 # 3. EDFA 1
20 edfa1.propagate(spectral_info)
21 # - Applies gain (signal, ASE, NLI all amplified)
22 # - Adds new ASE noise
23 # - Does NOT add new NLI (NLI added in fiber only)
24
25 # 4-9. Repeat for spans 2 and 3
26
27 # 10. Receiver
28 transceiver_rx.propagate(spectral_info)
29 # - Calculates penalties (CD, PMD, PDL)
30 # - Computes required OSNR
31 # - Determines feasibility

```

1.6.5 QoT Estimation at Receiver

```

1 class Transceiver(Element):
2     def propagate(self, spectral_info: SpectralInformation):
3         """Receiver: compute QoT and feasibility"""
4
5         # 1. Calculate OSNR from accumulated noise
6         osnr_actual = spectral_info.osnr # dB per channel
7
8         # 2. Compute penalties
9         penalty_cd = self.cd_penalty(
10             spectral_info.chromatic_dispersion,
11             spectral_info.baud_rate)
12
13         penalty_pmd = self.pmd_penalty(
14             spectral_info.pmd,
15             spectral_info.baud_rate)
16
17         penalty_pdl = self.pdl_penalty(spectral_info.pdl)
18
19         # 3. Required OSNR with penalties
20         osnr_required = self.mode.osnr + \
21             penalty_cd + penalty_pmd + penalty_pdl
22
23         # 4. Margin
24         margin = osnr_actual - osnr_required
25
26         # 5. Feasibility
27         feasible = (margin > 0)
28

```

```

29     # Store results
30     spectral_info.osnr_required = osnr_required
31     spectral_info.margin = margin
32     spectral_info.feasible = feasible
33
34     return spectral_info
35
36 def cd_penalty(self, cd, baud_rate):
37     """Chromatic dispersion penalty [dB]
38
39     Based on transceiver mode capability
40     """
41     if abs(cd) <= self.mode.chromatic_dispersion:
42         # Within tolerance, no penalty
43         return 0
44     else:
45         # Simplified penalty model
46         # Real implementations use detailed DSP models
47         excess_cd = abs(cd) - self.mode.chromatic_dispersion
48         penalty = (excess_cd / 1000)**2 * 0.1 # Example formula
49         return penalty

```

1.7. VALIDATION AND CALIBRATION

1.7.1 Model Validation Approaches

GNPy's physical layer models should be validated against:

1. **Experimental measurements:** Lab testbeds with controlled conditions
2. **Field deployments:** Live network measurements
3. **Other simulation tools:** Cross-validation with VPItransmissionMaker, OptiSystem

1.7.2 Typical Validation Metrics

- **OSNR prediction accuracy:** Target ± 1 dB
- **NLI estimation:** Within 20% of measured values
- **Reach prediction:** Within 10% of actual feasible distance

1.7.3 Calibration Parameters

Key parameters for calibration:

Parameter	Typical Range	Calibration Method
EDFA noise figure	4-11 dB	Measure with OSA
Fiber nonlinear coeff.	0.8-1.4 W ⁻¹ ·km ⁻¹	Four-wave mixing test
ROADM loss	15-20 dB	Direct measurement
ROADM add/drop OSNR	30-38 dB	Back-to-back test
SPM/XPM weights	0.5-1.5	Fit to system tests

Table 1.3: Calibration parameters and methods

1.8. SUMMARY

This chapter provided a comprehensive analysis of physical layer impairments in optical networks and their implementation in GNPy. Key takeaways:

1.8.1 Implementation Architecture

- **Modular design:** Separation of network elements (`elements.py`) and physics solvers (`science_utils.py`)

- **Spectral information propagation:** `SpectralInformation` class accumulates impairments through network
- **Per-channel tracking:** All impairments tracked individually per wavelength channel

1.8.2 Critical Implementation Details

1. **Linear impairments:** Deterministic, accumulate predictably
 - Attenuation: Exponential decay with optional Raman modification
 - CD: Linear accumulation, penalty at receiver
 - PMD, PDL: RSS accumulation, quadratic penalties
2. **Nonlinear impairments:** Power-dependent, complex interactions
 - NLI: Gaussian noise model with analytical (GN) or numerical (GGN) solutions
 - SRS: Coupled power equations, modifies effective fiber loss
 - Computational optimization via representative channel sampling
3. **Amplifier noise:** ASE accumulation determines OSNR floor
 - Noise figure models: polynomial, ripple, advanced vendor models
 - Gain/tilt management essential for multi-span systems
4. **ROADM effects:** Filtering and OSNR degradation
 - Path-dependent losses (express/add/drop)
 - Add/drop OSNR specification
 - Channel equalization

1.8.3 Software Development Considerations

For implementing similar optical network planning software:

1. **Class hierarchy:** Base `Element` class with specialized propagation methods
2. **Spectral information container:** Efficient storage of per-channel parameters
3. **Physics solvers:** Separate complex calculations (NLI, Raman) from element logic
4. **Computational optimization:**
 - Matrix operations for multi-channel calculations
 - Representative channel sampling for large systems
 - Caching of expensive computations
5. **Extensibility:** Easy addition of new element types and impairment models
6. **Configuration management:** JSON-based equipment libraries
7. **Validation framework:** Built-in comparison with measurements

1.8.4 Next Steps

The following chapters will cover:

- Chapter 2: Network Model Building and Topology Management
- Chapter 3: Path Computation and Spectrum Assignment
- Chapter 4: Configuration File Specifications
- Chapter 5: Output Analysis and Interpretation
- Chapter 6: Practical Usage Examples

Chapter 2

Network Model Building

CHAPTER OVERVIEW

This chapter provides comprehensive analysis of GNPpy’s network model building phase, focusing on the three core functions that transform JSON configurations into simulation-ready network graphs. We examine `load_equipment()`, `load_network()`, and `design_network()` in detail, including validation strategies, graph construction algorithms, and the **auto-design** (Local Optimization, Global Optimization) auto-design algorithm. This material is essential for implementing optical network planning tools with automated amplifier placement and power optimization.

2.1. INTRODUCTION TO NETWORK MODEL BUILDING

2.1.1 Overview and Motivation

Network model building is the critical phase that bridges static configuration files and dynamic simulation. The process transforms JSON descriptions into operational data structures that support:

- Path computation algorithms
- Signal propagation simulation
- Quality of transmission estimation
- Network optimization

Key Challenge: Network descriptions are often incomplete:

- Amplifiers may not be explicitly placed
- Power levels may not be specified
- Equipment parameters may need validation
- Topology may require modification for feasibility

2.1.2 Three-Function Architecture

GNPy's network building follows a three-stage pipeline:

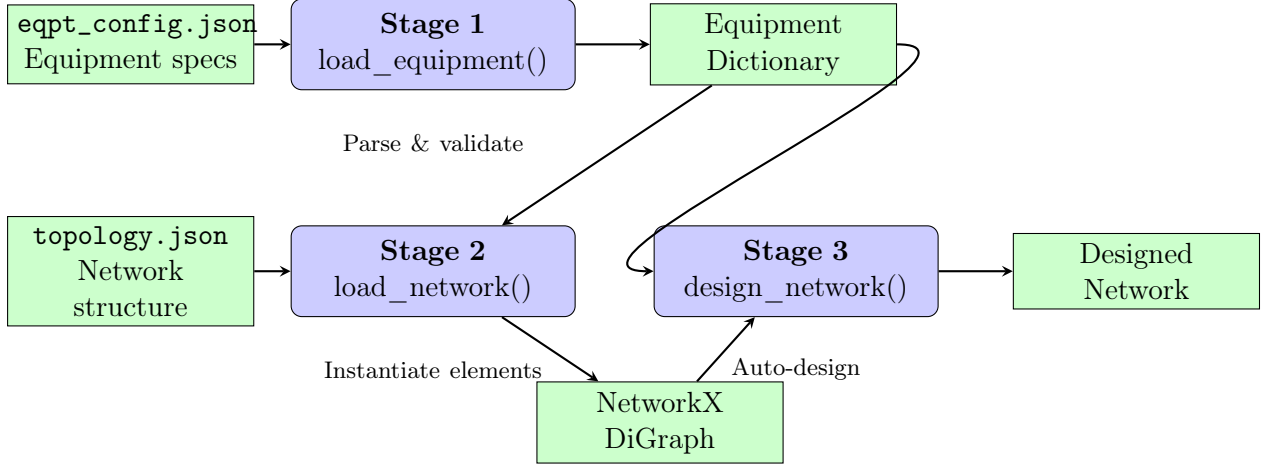


Figure 2.1: Three-stage network building pipeline

Data Flow:

$$\text{JSON Files} \xrightarrow{\text{parse}} \text{Python Objects} \xrightarrow{\text{connect}} \text{Graph} \xrightarrow{\text{optimize}} \text{Designed Network} \quad (2.1)$$

2.1.3 Design Philosophy

The three-function architecture embodies several design principles:

1. **Separation of concerns:**
 - Equipment specs separate from topology
 - Raw network separate from designed network
 - Validation at each stage
2. **Reusability:**
 - Equipment library shared across topologies
 - Design algorithms independent of specific networks
 - Modular validation functions
3. **Extensibility:**
 - Easy to add new element types

- Pluggable validation rules
 - Customizable design algorithms
4. **Standards compliance:**
- OpenROADM model alignment
 - IETF YANG compatibility
 - Industry-standard equipment specs

2.1.4 Chapter Organization

This chapter is organized as follows:

- **Section 2:** `load_equipment()` - Equipment library construction
- **Section 3:** `load_network()` - Graph building and validation
- **Section 4:** `design_network()` - auto-design algorithm and optimization
- **Section 5:** OpenROADM alignment and standards compliance
- **Section 6:** Implementation considerations and best practices

2.2. STAGE 1: EQUIPMENT LIBRARY LOADING

2.2.1 Function Specification

Location and Signature

```

1  # gnpy/core/equipment.py
2
3  def load_equipment(filename, extra_configs=None):
4      """Load equipment library from JSON configuration
5
6      Args:
7          filename (str or Path): Path to eqpt_config.json
8          extra_configs (dict): Optional advanced EDFA configs
9              Format: {config_file: config_data}
10
11      Returns:
12          dict: Equipment library organized by category
13              {
14                  'Edfa': {type_variety: Amp, ...},
15                  'Fiber': {type_variety: FiberParams, ...},
16                  'RamanFiber': {type_variety: RamanFiberParams, ...},
17                  'Span': {default: SpanParams},
18                  'Roadm': {type_variety: RoadmParams, ...},
19                  'SI': {default: SI, ...},
20                  'Transceiver': {type_variety: Transceiver, ...}
21              }
22
23      Raises:
24          EquipmentConfigError: Invalid configuration
25          FileNotFoundError: Config file not found
26          JSONDecodeError: Malformed JSON
27
28      Side Effects:
29          - Validates all parameters
30          - Logs warnings for deprecated fields
31          - Applies unit conversions
32      """
33      # Load and parse JSON
34      with open(filename, encoding='utf-8') as f:
35          json_data = json.load(f)
36

```

```

37 # Build equipment dictionary
38 equipment = _equipment_from_json(json_data, extra_configs)
39
40 return equipment

```

2.2.2 Equipment Configuration Structure

JSON Schema

The equipment configuration file has seven main sections:

```

{
  "Edfa": [
    {
      "type_variety": "std_medium_gain",
      "type_def": "variable_gain",
      "gain_flatmax": 26,
      "gain_min": 15,
      "p_max": 23,
      "nf_min": 6,
      "nf_max": 10,
      "out_voa_auto": false,
      "allowed_for_design": true
    }
  ],
  "Fiber": [...],
  "RamanFiber": [...],
  "Span": {...},
  "Roadm": [...],
  "SI": [...],
  "Transceiver": [...]
}

```

Equipment Categories

Category	Cardinality	Purpose
Edfa	Multiple	Amplifier types and models
Fiber	Multiple	Fiber types and characteristics
RamanFiber	Multiple	Raman-enabled fiber types
Span	Single (default)	Span design parameters
Roadm	Multiple	ROADM configurations
SI	Multiple (must have 'default')	Spectral information templates
Transceiver	Multiple	Transceiver modes and capabilities

Table 2.1: Equipment configuration categories

2.2.3 Implementation Details

Processing Workflow

```

1 def _equipment_from_json(json_data, extra_configs):
2     """Convert JSON to equipment dictionary with validation"""
3
4     equipment = {}
5
6     # 1. Process EDFAs
7     if 'Edfa' in json_data:

```

```

8     equipment['Edfa'] = {}
9     for amp_config in json_data['Edfa']:
10         variety = amp_config['type_variety']
11
12         # Load advanced config if specified
13         if 'advanced_config_from_json' in amp_config:
14             config_file = amp_config['advanced_config_from_json']
15             if extra_configs and config_file in extra_configs:
16                 # Merge advanced configuration
17                 advanced = extra_configs[config_file]
18                 amp_config = {**amp_config, **advanced}
19
20         # Validate EDFA parameters
21         _validate_edfa(amp_config)
22
23         # Create Amp object
24         amp = Amp(**amp_config)
25         equipment['Edfa'][variety] = amp
26
27     # 2. Process Fibers
28     if 'Fiber' in json_data:
29         equipment['Fiber'] = {}
30         for fiber_config in json_data['Fiber']:
31             variety = fiber_config['type_variety']
32
33             # Unit conversions
34             _convert_fiber_units(fiber_config)
35
36             # Validate
37             _validate_fiber(fiber_config)
38
39             # Create FiberParams
40             fiber = FiberParams(**fiber_config)
41             equipment['Fiber'][variety] = fiber
42
43     # 3. Process RamanFiber
44     if 'RamanFiber' in json_data:
45         equipment['RamanFiber'] = {}
46         for raman_config in json_data['RamanFiber']:
47             variety = raman_config['type_variety']
48
49             # Inherit from Fiber if specified
50             if 'fiber_type_variety' in raman_config:
51                 base_fiber = equipment['Fiber'][
52                     raman_config['fiber_type_variety']]
53                 raman_config = {**base_fiber.__dict__, **raman_config}
54
55             _validate_raman_fiber(raman_config)
56
57             raman_fiber = RamanFiberParams(**raman_config)
58             equipment['RamanFiber'][variety] = raman_fiber
59
60     # 4. Process Span (singleton)
61     if 'Span' in json_data:
62         span_config = json_data['Span']
63         _validate_span(span_config)
64         equipment['Span'] = {'default': SpanParams(**span_config)}
65     else:

```

```

66     # Use default span parameters
67     equipment['Span'] = {'default': SpanParams()}
68
69     # 5. Process ROADMs
70     if 'Roadm' in json_data:
71         equipment['Roadm'] = {}
72         for roadm_config in json_data['Roadm']:
73             variety = roadm_config.get('type_variety', 'default')
74             _validate_roadm(roadm_config)
75             roadm = RoadmParams(**roadm_config)
76             equipment['Roadm'][variety] = roadm
77
78     # 6. Process SI (must have 'default')
79     if 'SI' in json_data:
80         equipment['SI'] = {}
81         has_default = False
82         for si_config in json_data['SI']:
83             variety = si_config.get('f_min', 'default')
84             if variety == 'default':
85                 has_default = True
86             _validate_si(si_config)
87             si = SI(**si_config)
88             equipment['SI'][variety] = si
89
90         if not has_default:
91             raise EquipmentConfigError(
92                 "SI must contain a 'default' entry")
93     else:
94         raise EquipmentConfigError("Missing SI configuration")
95
96     # 7. Process Transceivers
97     if 'Transceiver' in json_data:
98         equipment['Transceiver'] = {}
99         for trx_config in json_data['Transceiver']:
100             variety = trx_config['type_variety']
101             _validate_transceiver(trx_config)
102             trx = Transceiver(**trx_config)
103             equipment['Transceiver'][variety] = trx
104
105     # 8. Cross-validate
106     _cross_validate_equipment(equipment)
107
108     return equipment

```

2.2.4 Validation Functions

EDFA Validation

```

1 def _validate_edfa(config):
2     """Validate EDFA configuration parameters
3
4     Checks:
5     - Gain range consistency
6     - Power limits
7     - Noise figure range
8     - Type-specific requirements
9     """
10    variety = config['type_variety']

```

```

11     type_def = config['type_def']
12
13     # 1. Check type_def validity
14     valid_types = ['variable_gain', 'fixed_gain', 'openroadm',
15                   'advanced_model', 'dual_stage']
16     if type_def not in valid_types:
17         raise EquipmentConfigError(
18             f"{variety}: Invalid type_def '{type_def}'. "
19             f"Must be one of {valid_types}")
20
21     # 2. Validate gain range
22     if config['gain_min'] >= config['gain_flatmax']:
23         raise EquipmentConfigError(
24             f"{variety}: gain_min ({config['gain_min']}) must be < "
25             f"gain_flatmax ({config['gain_flatmax']})")
26
27     if config['gain_min'] < 0:
28         raise EquipmentConfigError(
29             f"{variety}: gain_min must be non-negative")
30
31     if config['gain_flatmax'] > 50:
32         logger.warning(
33             f"{variety}: gain_flatmax ({config['gain_flatmax']} dB) "
34             f"is unusually high")
35
36     # 3. Validate power limits
37     if config['p_max'] <= 0:
38         raise EquipmentConfigError(
39             f"{variety}: p_max must be positive")
40
41     if config['p_max'] > 30:
42         logger.warning(
43             f"{variety}: p_max ({config['p_max']} dBm) "
44             f"is very high")
45
46     # 4. Validate noise figure
47     if 'nf_min' in config and 'nf_max' in config:
48         if config['nf_min'] >= config['nf_max']:
49             raise EquipmentConfigError(
50                 f"{variety}: nf_min must be < nf_max")
51
52         if config['nf_min'] < 3:
53             raise EquipmentConfigError(
54                 f"{variety}: nf_min ({config['nf_min']} dB) "
55                 f"violates quantum limit (3 dB)")
56
57         if config['nf_max'] > 15:
58             logger.warning(
59                 f"{variety}: nf_max ({config['nf_max']} dB) "
60                 f"is unusually high")
61
62     # 5. Type-specific validation
63     if type_def == 'dual_stage':
64         if 'preamp_variety' not in config:
65             raise EquipmentConfigError(
66                 f"{variety}: dual_stage requires preamp_variety")
67         if 'booster_variety' not in config:
68             raise EquipmentConfigError(

```

```

69         f"{variety}: dual_stage requires booster_variety")
70
71     if type_def == 'openroadm':
72         # Check OpenROADM-specific fields
73         required = ['gain_flatmax', 'gain_min', 'nf_coef']
74         for field in required:
75             if field not in config:
76                 raise EquipmentConfigError(
77                     f"{variety}: OpenROADM type requires '{field}'")
78
79     if type_def == 'advanced_model':
80         if 'advanced_config_from_json' not in config:
81             logger.warning(
82                 f"{variety}: advanced_model should specify "
83                 f"advanced_config_from_json")
84
85     # 6. Validate VOA settings
86     if 'out_voa_auto' in config:
87         if config['out_voa_auto']:
88             # Check for VOA margin and step
89             if 'voa_margin' in config and config['voa_margin'] < 0:
90                 raise EquipmentConfigError(
91                     f"{variety}: voa_margin must be non-negative")

```

Fiber Validation

```

1  def _validate_fiber(config):
2      """Validate fiber configuration parameters"""
3      variety = config['type_variety']
4
5      # 1. Dispersion parameter
6      if 'dispersion' in config:
7          D = config['dispersion'] # s/(m*m)
8          D_psnmkm = D * 1e6 # Convert to ps/(nm*km)
9
10         if abs(D_psnmkm) > 30:
11             logger.warning(
12                 f"{variety}: Dispersion ({D_psnmkm:.1f} ps/(nm*km)) "
13                 f"is very high")
14
15         if D_psnmkm < 0:
16             logger.info(f"{variety}: Negative dispersion (DCF?)")
17
18     # 2. Nonlinear coefficient
19     if 'gamma' in config:
20         gamma = config['gamma'] # 1/(W*m)
21         gamma_wkm = gamma * 1000 # Convert to 1/(W*km)
22
23         if gamma_wkm < 0.5 or gamma_wkm > 3:
24             logger.warning(
25                 f"{variety}: Gamma ({gamma_wkm:.2f} 1/(W*km)) "
26                 f"is outside typical range [0.5, 3]")
27
28     # 3. Loss coefficient
29     if 'loss_coef' in config:
30         alpha = config['loss_coef'] # dB/km
31
32         if alpha < 0.15 or alpha > 0.5:

```

```

33         logger.warning(
34             f"{variety}: Loss coefficient ({alpha} dB/km) "
35             f"is unusual")
36
37     # 4. PMD coefficient
38     if 'pmd_coef' in config:
39         pmd = config['pmd_coef'] # s/sqrt(m)
40         pmd_ps_sqrtkm = pmd * 1e12 * sqrt(1000)
41
42         if pmd_ps_sqrtkm > 2:
43             logger.warning(
44                 f"{variety}: PMD coefficient "
45                 f"({pmd_ps_sqrtkm:.2f} ps/sqrt(km)) is high")
46
47     # 5. Effective area
48     if 'effective_area' in config:
49         aeff = config['effective_area'] # m^2
50         aeff_um2 = aeff * 1e12 # Convert to um^2
51
52         if aeff_um2 < 50 or aeff_um2 > 150:
53             logger.warning(
54                 f"{variety}: Effective area ({aeff_um2:.1f} um^2) "
55                 f"is unusual")
56
57     # 6. Check consistency between gamma and aeff
58     if 'gamma' in config and 'effective_area' in config:
59         # gamma = 2*pi*n2 / (lambda * Aeff)
60         n2 = 2.6e-20 # m^2/W
61         wavelength = 1.55e-6 # m
62         gamma_calc = 2 * pi * n2 / (wavelength * config['effective_area'])
63
64         gamma_ratio = config['gamma'] / gamma_calc
65         if gamma_ratio < 0.8 or gamma_ratio > 1.2:
66             logger.warning(
67                 f"{variety}: Gamma ({config['gamma']:.3e}) "
68                 f"inconsistent with Aeff ({config['effective_area']:.3e}) "
69                 f"Expected gamma equivalent to {gamma_calc:.3e}")
70
71 def _convert_fiber_units(config):
72     """Convert fiber parameters to internal units"""
73     # Dispersion: store in s/(m*m), display in ps/(nm*km)
74     if 'dispersion' in config:
75         # JSON should provide s/(m*m)
76         config['_dispersion_display'] = config['dispersion'] * 1e6
77
78     # Nonlinear coef: store in 1/(W*m), display in 1/(W*km)
79     if 'gamma' in config:
80         config['_gamma_display'] = config['gamma'] * 1000
81
82     # PMD: store in s/sqrt(m), display in ps/sqrt(km)
83     if 'pmd_coef' in config:
84         config['_pmd_display'] = config['pmd_coef'] * 1e12 * sqrt(1000)

```

Span Validation

```

1 def _validate_span(config):

```

```

2      """Validate span configuration
3
4      Span config controls automatic network design
5      """
6      # 1. Power mode
7      if 'power_mode' in config:
8          if config['power_mode']:
9              # Check delta_power_range_db
10             if 'delta_power_range_db' not in config:
11                 raise EquipmentConfigError(
12                     "power_mode=True requires delta_power_range_db")
13
14             dp_range = config['delta_power_range_db']
15             if len(dp_range) != 3:
16                 raise EquipmentConfigError(
17                     "delta_power_range_db must be [min, max, step]")
18
19             dp_min, dp_max, dp_step = dp_range
20             if dp_min >= dp_max:
21                 raise EquipmentConfigError(
22                     "delta_power_range_db: min must be < max")
23
24             if dp_step <= 0:
25                 raise EquipmentConfigError(
26                     "delta_power_range_db: step must be positive")
27
28      # 2. Max length and loss
29      if 'max_length' in config:
30          if config['max_length'] <= 0:
31              raise EquipmentConfigError(
32                  "max_length must be positive")
33
34          if config['max_length'] > 200:
35              logger.warning(
36                  f"max_length ({config['max_length']} km) is very long")
37
38      if 'max_loss' in config:
39          if config['max_loss'] <= 0:
40              raise EquipmentConfigError(
41                  "max_loss must be positive")
42
43          if config['max_loss'] > 35:
44              logger.warning(
45                  f"max_loss ({config['max_loss']} dB) is very high")
46
47      # 3. Padding
48      if 'padding' in config:
49          if config['padding'] < 0:
50              raise EquipmentConfigError(
51                  "padding must be non-negative")
52
53      # 4. EOL (end-of-life margin)
54      if 'EOL' in config:
55          if config['EOL'] < 0:
56              raise EquipmentConfigError(
57                  "EOL must be non-negative")
58
59          if config['EOL'] > 5:

```

```

60         logger.warning(
61             f"EOL ({config['EOL']} dB) is unusually high")

```

2.2.5 Equipment Dictionary Structure

The output equipment dictionary has this structure:

```

1 equipment = {
2     'Edfa': {
3         'std_medium_gain': Amp(
4             type_variety='std_medium_gain',
5             type_def='variable_gain',
6             gain_flatmax=26,
7             gain_min=15,
8             p_max=23,
9             nf_min=6,
10            nf_max=10,
11            ...
12        ),
13        'std_high_gain': Amp(...),
14        ...
15    },
16    'Fiber': {
17        'SSMF': FiberParams(
18            type_variety='SSMF',
19            dispersion=1.67e-05, # s/(m*m)
20            gamma=1.27e-03, # 1/(W*m)
21            effective_area=83e-12,
22            pmd_coef=1.265e-15,
23            ...
24        ),
25        'LEAF': FiberParams(...),
26        ...
27    },
28    'Span': {
29        'default': SpanParams(
30            power_mode=True,
31            delta_power_range_db=[-2, 3, 0.5],
32            max_length=150,
33            max_loss=28,
34            padding=10,
35            EOL=0
36        )
37    },
38    'Roadm': {
39        'default': RoadmParams(
40            target_pch_out_db=-20,
41            add_drop_osnr=38,
42            pmd=0,
43            pdl=0
44        )
45    },
46    'SI': {
47        'default': SI(
48            f_min=191.3e12,
49            f_max=195.1e12,
50            baud_rate=32e9,
51            spacing=50e9,
52            power_dbm=0,

```

```

53         roll_off=0.15,
54         tx_osnr=40
55     )
56 },
57 'Transceiver': {
58     'default': Transceiver(
59         type_variety='default',
60         frequency={'min': 191.35e12, 'max': 196.1e12},
61         mode=[
62             TransceiverMode(
63                 format='DP-QPSK',
64                 baud_rate=32e9,
65                 OSNR=12,
66                 bit_rate=100e9,
67                 ...
68             ),
69             ...
70         ]
71     )
72 }
73 }

```

2.3. STAGE 2: NETWORK GRAPH CONSTRUCTION

2.3.1 Function Specification

Location and Signature

```

1  # gnpy/core/network.py
2
3  def load_network(topology, equipment):
4      """Build NetworkX directed graph from topology
5
6      Args:
7          topology (dict or str): Topology JSON data or filename
8          equipment (dict): Equipment library from load_equipment()
9
10     Returns:
11         nx.DiGraph: Network graph with:
12             - Nodes = network elements (Fiber, Edfa, etc.)
13             - Edges = directed connections
14             - Node attributes = element objects and metadata
15
16     Raises:
17         NetworkTopologyError: Invalid topology structure
18         EquipmentConfigError: Unknown equipment reference
19
20     Process:
21         1. Parse topology JSON (if filename provided)
22         2. Create element objects using equipment specs
23         3. Build directed graph
24         4. Validate connectivity
25         5. Assign default parameters
26     """
27     # Load JSON if needed
28     if isinstance(topology, str):
29         with open(topology, encoding='utf-8') as f:
30             topology = json.load(f)
31

```

```

32 # Build network graph
33 network = _network_from_json(topology, equipment)
34
35 # Validate
36 _validate_network_topology(network)
37
38 return network

```

2.3.2 Topology JSON Structure

Elements and Connections

```

{
  "elements": [
    {
      "uid": "Paris",
      "type": "Transceiver",
      "metadata": {
        "location": {
          "city": "Paris",
          "region": "IDF",
          "latitude": 48.8566,
          "longitude": 2.3522
        }
      }
    },
    {
      "uid": "Fiber_Paris_Lyon",
      "type": "Fiber",
      "type_variety": "SSMF",
      "params": {
        "length": 450,
        "length_units": "km"
      }
    },
    {
      "uid": "Amp_post_Paris",
      "type": "Edfa",
      "type_variety": "std_medium_gain",
      "operational": {
        "gain_target": null,
        "power_target": null,
        "tilt_target": 0
      }
    }
  ],
  "connections": [
    {"from_node": "Paris", "to_node": "Fiber_Paris_Lyon"},
    {"from_node": "Fiber_Paris_Lyon", "to_node": "Amp_post_Paris"}
  ]
}

```

2.3.3 Graph Construction Algorithm

```

1 def _network_from_json(topology, equipment):
2     """Convert topology JSON to NetworkX DiGraph"""
3
4     # Initialize directed graph

```

```

5     network = nx.DiGraph()
6
7     # ===== STEP 1: Create nodes =====
8     logger.info("Creating network nodes...")
9
10    for element_config in topology['elements']:
11        uid = element_config['uid']
12        element_type = element_config['type']
13
14        logger.debug(f"Processing {uid} (type={element_type})")
15
16        # Get equipment parameters
17        params = _get_equipment_params(
18            element_config, equipment, element_type)
19
20        # Get operational parameters
21        operational = element_config.get('operational', {})
22
23        # Create element object
24        element = _create_element(
25            element_type, uid, params, operational)
26
27        # Add node to graph
28        network.add_node(
29            uid,
30            element=element,
31            type=element_type,
32            metadata=element_config.get('metadata', {})
33        )
34
35        logger.debug(f"Created {uid}: {type(element).__name__}")
36
37    # ===== STEP 2: Create edges =====
38    logger.info("Creating network connections...")
39
40    for connection in topology['connections']:
41        from_node = connection['from_node']
42        to_node = connection['to_node']
43
44        # Validate nodes exist
45        if from_node not in network:
46            raise NetworkTopologyError(
47                f"Connection references unknown node: {from_node}")
48        if to_node not in network:
49            raise NetworkTopologyError(
50                f"Connection references unknown node: {to_node}")
51
52        # Add directed edge
53        network.add_edge(from_node, to_node)
54
55        logger.debug(f"Connected {from_node} -> {to_node}")
56
57    logger.info(f"Network created: {network.number_of_nodes()} nodes, "
58              f"{network.number_of_edges()} edges")
59
60    return network
61
62 def _get_equipment_params(element_config, equipment, element_type):

```

```

63     """Extract equipment parameters for element
64
65     Combines:
66         1. Base params from equipment library (via type_variety)
67         2. Element-specific overrides (params field)
68     """
69     type_variety = element_config.get('type_variety')
70
71     if type_variety:
72         # Look up in equipment library
73         if element_type not in equipment:
74             raise EquipmentConfigError(
75                 f"Unknown element type: {element_type}")
76
77         if type_variety not in equipment[element_type]:
78             raise EquipmentConfigError(
79                 f"Unknown {element_type} type_variety: {type_variety}")
80
81         # Get base parameters
82         base_params = equipment[element_type][type_variety]
83
84         # Convert to dict if object
85         if hasattr(base_params, '__dict__'):
86             params = base_params.__dict__.copy()
87         else:
88             params = base_params.copy()
89     else:
90         # No type_variety - use defaults or element params only
91         params = {}
92
93     # Override with element-specific parameters
94     if 'params' in element_config:
95         params.update(element_config['params'])
96
97     return params
98
99 def _create_element(element_type, uid, params, operational):
100     """Factory function to instantiate element objects
101
102     Maps element types to classes and creates instances
103     """
104     from gnp.py.core.elements import (
105         Fiber, Edfa, Roadm, Transceiver, Fused
106     )
107
108     # Element type mapping
109     element_classes = {
110         'Fiber': Fiber,
111         'Edfa': Edfa,
112         'Roadm': Roadm,
113         'Transceiver': Transceiver,
114         'Fused': Fused
115     }
116
117     if element_type not in element_classes:
118         raise NetworkTopologyError(
119             f"Unknown element type: {element_type}")
120

```

```

121 # Instantiate element
122 ElementClass = element_classes[element_type]
123
124 try:
125     element = ElementClass(
126         uid=uid,
127         params=params,
128         operational=operational
129     )
130 except Exception as e:
131     raise NetworkTopologyError(
132         f"Failed to create {element_type} '{uid}': {str(e)}")
133
134 return element

```

2.3.4 Network Validation

Topological Checks

```

1 def _validate_network_topology(network):
2     """Validate network graph structure and connectivity
3
4     Checks:
5         1. No isolated nodes
6         2. Transceivers at endpoints
7         3. Amplifiers after fibers
8         4. ROADM degree consistency
9         5. Reachability
10
11     """
12     logger.info("Validating network topology...")
13
14     # 1. Check for isolated nodes
15     isolated = list(nx.isolates(network))
16     if isolated:
17         logger.error(f"Isolated nodes detected: {isolated}")
18         raise NetworkTopologyError(
19             f"Network has isolated nodes: {isolated}")
20
21     # 2. Check transceiver placement
22     _validate_transceivers(network)
23
24     # 3. Check fiber-amplifier sequences
25     _validate_fiber_amplifier_sequences(network)
26
27     # 4. Check ROADM degrees
28     _validate_roadm_connectivity(network)
29
30     # 5. Check reachability
31     _validate_reachability(network)
32
33     logger.info("Network topology validation passed")
34
35 def _validate_transceivers(network):
36     """Validate transceiver placement
37
38     Transceivers should be at network endpoints
39     """
40     for node in network.nodes():

```

```

40     element = network.nodes[node]['element']
41
42     if isinstance(element, Transceiver):
43         in_degree = network.in_degree(node)
44         out_degree = network.out_degree(node)
45
46         # Transceiver should have at most 1 input and 1 output
47         if in_degree > 1:
48             logger.warning(
49                 f"Transceiver {node} has multiple inputs "
50                 f"(degree={in_degree})")
51
52         if out_degree > 1:
53             logger.warning(
54                 f"Transceiver {node} has multiple outputs "
55                 f"(degree={out_degree})")
56
57         # Transceiver should not have both input and output
58         # (pure transmitter or receiver)
59         if in_degree > 0 and out_degree > 0:
60             logger.info(
61                 f"Transceiver {node} is bidirectional")
62
63 def _validate_fiber_amplifier_sequences(network):
64     """Check fiber-amplifier relationships
65
66     Warns about:
67     - Fibers without following amplifiers
68     - Very long fiber spans
69     - Unusual loss values
70     """
71     for node in network.nodes():
72         element = network.nodes[node]['element']
73
74         if isinstance(element, Fiber):
75             # Calculate span loss
76             fiber_loss_db = element.params.loss_coef * \
77                 element.params.length / 1000
78             connector_loss = element.params.con_in + \
79                 element.params.con_out
80             total_loss = fiber_loss_db + connector_loss
81
82             # Check if followed by amplifier
83             successors = list(network.successors(node))
84             if successors:
85                 next_node = successors[0]
86                 next_element = network.nodes[next_node]['element']
87
88                 if not isinstance(next_element, (Edfa, Roadm)):
89                     logger.warning(
90                         f"Fiber {node} (loss={total_loss:.1f} dB) "
91                         f"not followed by amplifier or ROADM")
92
93                 if total_loss > 20:
94                     logger.error(
95                         f"Fiber {node} has high loss "
96                         f"({total_loss:.1f} dB) without
97                         amplification")

```

```

97
98     # Check for excessive length
99     if element.params.length > 150000: # 150 km in m
100         logger.warning(
101             f"Fiber {node} is very long "
102             f"({element.params.length/1000:.1f} km)")
103
104 def _validate_roadm_connectivity(network):
105     """Validate ROADM degree and connectivity"""
106     for node in network.nodes():
107         element = network.nodes[node]['element']
108
109         if isinstance(element, Roadm):
110             in_degree = network.in_degree(node)
111             out_degree = network.out_degree(node)
112
113             # ROADM should have at least 2 connections (degree-2)
114             if in_degree + out_degree < 2:
115                 logger.warning(
116                     f"ROADM {node} has low connectivity "
117                     f"(in={in_degree}, out={out_degree})")
118
119             # Check for typical ROADM structure
120             # Input: Fiber or amplifier
121             # Output: Fiber or amplifier
122             predecessors = list(network.predecessors(node))
123             successors = list(network.successors(node))
124
125             if predecessors:
126                 for pred in predecessors:
127                     pred_element = network.nodes[pred]['element']
128                     if not isinstance(pred_element, (Fiber, Edfa)):
129                         logger.warning(
130                             f"ROADM {node} has unusual input "
131                             f"from {pred} ({type(pred_element).__name__})")
132
133 def _validate_reachability(network):
134     """Check network reachability between transceivers"""
135     # Find all transceivers
136     transceivers = [
137         node for node in network.nodes()
138         if isinstance(network.nodes[node]['element'], Transceiver)
139     ]
140
141     if len(transceivers) < 2:
142         logger.warning("Network has fewer than 2 transceivers")
143         return
144
145     # Check if transceivers can reach each other
146     unreachable_pairs = []
147
148     for i, tx in enumerate(transceivers):
149         for rx in transceivers[i+1:]:
150             # Check both directions
151             path_tx_rx = nx.has_path(network, tx, rx)
152             path_rx_tx = nx.has_path(network, rx, tx)
153

```

```

154         if not path_tx_rx and not path_rx_tx:
155             unreachable_pairs.append((tx, rx))
156
157     if unreachable_pairs:
158         logger.warning(
159             f"Found {len(unreachable_pairs)} unreachable "
160             f"transceiver pairs:")
161     for tx, rx in unreachable_pairs:
162         logger.warning(f" {tx} <-> {rx}")

```

2.3.5 NetworkX DiGraph Structure

Node Attributes

Each node in the NetworkX DiGraph stores:

```

1 network.nodes[uid] = {
2     'element': <Element object>, # Fiber, Edfa, Roadm, etc.
3     'type': <element_type>,      # String: 'Fiber', 'Edfa', ...
4     'metadata': {                # Optional metadata
5         'location': {
6             'city': <city>,
7             'latitude': <lat>,
8             'longitude': <lon>
9         },
10    'custom_field': <value>,
11    ...
12 }
13 }

```

Accessing Elements:

```

1 # Get element object
2 element = network.nodes['Paris']['element']
3
4 # Check element type
5 if isinstance(element, Transceiver):
6     print(f"Transceiver at {element.uid}")
7
8 # Access parameters
9 if isinstance(element, Fiber):
10     length = element.params.length
11     loss_coef = element.params.loss_coef

```

Edge Attributes

Currently, edges store minimal information:

```

1 network.edges[from_node, to_node] = {
2     # Currently empty - just connectivity
3 }

```

Future Extensions:

- Physical fiber routing information
- Patch panel connections
- Wavelength assignments
- Protection relationships

2.4. STAGE 3: NETWORK AUTO-DESIGN

2.4.1 auto-design Algorithm Overview

Purpose and Motivation

The `design_network()` function implements the **auto-design** (Local Optimization, Global Optimization) algorithm for automatic network design. The algorithm addresses three key problems:

1. **Amplifier Placement:** Where to place EDFAs along fiber spans
2. **Gain Setting:** What gain to configure for each amplifier
3. **Power Optimization:** What launch power per span to maximize OSNR uniformity

Reference: Augé, J., Curri, V., Le Rouzic, E., "Open Design for Multi-Vendor Optical Networks," OFC 2019, paper M3Z.3

Algorithm Objectives

- **Feasibility:** Ensure all amplifiers operate within limits
- **Uniformity:** Minimize OSNR variation across channels and spans
- **Optimality:** Maximize worst-case OSNR margin
- **Compliance:** Respect equipment constraints and design rules

2.4.2 Function Specification

```

1  # gnpy/core/network.py
2
3  def design_network(reference_channel, network, equipment,
4                     set_connector_losses=True, verbose=True):
5      """Auto-design network algorithm
6
7      Args:
8          reference_channel: SpectralInformation for design
9                          Contains: number of channels, power per channel
10         network: NetworkX DiGraph from load_network()
11         equipment: Equipment dictionary
12         set_connector_losses: Apply connector loss defaults
13         verbose: Enable warning messages
14
15     Returns:
16         network: Modified DiGraph with design complete
17
18     Modifies network in-place:
19         - Adds missing amplifiers
20         - Sets amplifier gains
21         - Optimizes power profile
22         - Configures ROADMs
23         - Sets fiber input powers
24
25     Implements:
26         auto-design algorithm from OFC 2019 paper (Auge et al.)
27         Equation 4:  $G_i = L_{span,i} + \Delta P_i$ 
28     """
29     logger.info("Starting network auto-design...")
30
31     # Step 1: Add missing network elements
32     add_missing_elements_in_network(
33         network, equipment, verbose)
34
35     # Step 2: Set fiber attributes

```

```

36     add_missing_fiber_attributes(
37         network, equipment, set_connector_losses)
38
39     # Step 3: Build network configuration
40     build_network(
41         network, equipment, reference_channel, verbose)
42
43     logger.info("Network auto-design complete")
44
45     return network

```

2.4.3 Step 1: Add Missing Elements

Amplifier Placement Algorithm

Algorithm 1 Amplifier Placement

```

1: Input: Network  $G$ , Equipment library  $E$ , Span config  $S$ 
2: Output: Modified network with amplifiers
3: for each fiber  $f$  in  $G$  do
4:      $L \leftarrow \text{calculate\_span\_loss}(f)$ 
5:      $L_{max} \leftarrow S[\text{max\_loss}]$ 
6:     if  $L > L_{max}$  then
7:         // Span loss exceeds limit
8:          $n \leftarrow \lceil L/L_{max} \rceil$  ▷ Number of segments
9:         call  $\text{split\_fiber}(f, n)$ 
10:    succ  $\leftarrow \text{successors}(f)$ 
11:    if succ  $\neq \emptyset$  then
12:        next  $\leftarrow \text{succ}[0]$ 
13:        if next is not Edfa and not Roadm then
14:            // Need to add amplifier
15:             $G_{target} \leftarrow L$  ▷ Target gain
16:            amp  $\leftarrow \text{select\_amplifier}(G_{target}, E)$ 
17:            call  $\text{insert\_amplifier}(f, \text{next}, \text{amp})$ 

```

```

1 def add_missing_elements_in_network(network, equipment, verbose=True):
2     """Add missing amplifiers to network
3
4     Process:
5         1. Identify fiber spans without amplification
6         2. Calculate span loss
7         3. Split long spans if needed
8         4. Insert amplifiers where required
9     """
10    span_config = equipment['Span']['default']
11    max_span_loss = span_config.max_loss
12    max_span_length = span_config.max_length * 1000 # Convert to m
13
14    # Collect spans needing amplifiers
15    spans_to_modify = []
16    fibers_to_split = []
17
18    for node in list(network.nodes()):
19        element = network.nodes[node]['element']
20

```

```

21     if not isinstance(element, Fiber):
22         continue
23
24     # Calculate span loss
25     fiber_loss = element.params.loss_coef * \
26                 element.params.length / 1000
27     conn_loss = element.params.con_in + element.params.con_out
28     total_loss = fiber_loss + conn_loss + span_config.padding
29
30     # Check if span too long
31     if element.params.length > max_span_length:
32         fibers_to_split.append((node, element))
33         continue
34
35     # Check if amplifier needed
36     successors = list(network.successors(node))
37     if not successors:
38         continue
39
40     next_node = successors[0]
41     next_element = network.nodes[next_node]['element']
42
43     if not isinstance(next_element, (Edfa, Roadm)):
44         # Need amplifier
45         if total_loss > max_span_loss:
46             logger.warning(
47                 f"Span {node} loss ({total_loss:.1f} dB) "
48                 f"exceeds max ({max_span_loss} dB)"
49             )
50             spans_to_modify.append((node, next_node, total_loss))
51
52     # Split long fibers
53     for fiber_node, fiber_element in fibers_to_split:
54         _split_fiber(network, equipment, fiber_node, fiber_element)
55
56     # Add amplifiers
57     for from_node, to_node, loss in spans_to_modify:
58         _insert_amplifier(
59             network, equipment, from_node, to_node, loss)
60
61 def _split_fiber(network, equipment, fiber_node, fiber_element):
62     """Split long fiber into multiple segments with amplifiers"""
63     span_config = equipment['Span']['default']
64     max_length = span_config.max_length * 1000 # m
65
66     total_length = fiber_element.params.length
67     n_segments = int(np.ceil(total_length / max_length))
68     segment_length = total_length / n_segments
69
70     logger.info(
71         f"Splitting fiber {fiber_node} ({total_length/1000:.1f} km) "
72         f"into {n_segments} segments")
73
74     # Get fiber successors
75     successors = list(network.successors(fiber_node))
76     if not successors:
77         logger.warning(f"Fiber {fiber_node} has no successors")
78     return

```

```

79
80     next_node = successors[0]
81
82     # Remove original fiber
83     network.remove_edge(fiber_node, next_node)
84
85     # Create fiber segments
86     prev_node = fiber_node
87     for i in range(n_segments):
88         # Create fiber segment
89         segment_uid = f"{fiber_node}_seg{i+1}"
90         segment_params = fiber_element.params.__dict__.copy()
91         segment_params['length'] = segment_length
92
93         segment_fiber = Fiber(
94             uid=segment_uid,
95             params=segment_params,
96             operational={}
97         )
98
99         network.add_node(segment_uid, element=segment_fiber)
100        network.add_edge(prev_node, segment_uid)
101
102        # Add amplifier after segment (except last)
103        if i < n_segments - 1:
104            amp_uid = f"{fiber_node}_amp{i+1}"
105
106            # Calculate segment loss
107            seg_loss = segment_params['loss_coef'] * \
108                segment_length / 1000
109
110            # Select amplifier
111            amp_variety = _select_amplifier_type(
112                seg_loss, equipment['Edfa'])
113
114            amp_params = equipment['Edfa'][amp_variety].__dict__.copy()
115            amp = Edfa(
116                uid=amp_uid,
117                params=amp_params,
118                operational={'gain_target': seg_loss}
119            )
120
121            network.add_node(amp_uid, element=amp)
122            network.add_edge(segment_uid, amp_uid)
123
124            prev_node = amp_uid
125        else:
126            prev_node = segment_uid
127
128        # Connect last segment to original successor
129        network.add_edge(prev_node, next_node)
130
131    def _insert_amplifier(network, equipment, from_node, to_node,
132                        required_gain):
133        """Insert amplifier between two nodes"""
134        # Generate unique UID
135        amp_uid = f"Edfa_auto_{from_node}_{to_node}"
136

```

```

137 # Select appropriate amplifier type
138 amp_variety = _select_amplifier_type(
139     required_gain, equipment['Edfa'])
140
141 if amp_variety is None:
142     logger.error(
143         f"No suitable amplifier for gain {required_gain:.1f} dB")
144     return
145
146 # Create amplifier
147 amp_params = equipment['Edfa'][amp_variety].__dict__.copy()
148 amp = Edfa(
149     uid=amp_uid,
150     params=amp_params,
151     operational={'gain_target': required_gain}
152 )
153
154 # Insert into network
155 network.add_node(amp_uid, element=amp)
156 network.remove_edge(from_node, to_node)
157 network.add_edge(from_node, amp_uid)
158 network.add_edge(amp_uid, to_node)
159
160 logger.info(
161     f"Added {amp_variety} amplifier at {amp_uid} "
162     f"(gain={required_gain:.1f} dB)")
163
164 def _select_amplifier_type(required_gain, edfa_library):
165     """Select appropriate amplifier for required gain
166
167     Strategy:
168     1. Filter by allowed_for_design=True
169     2. Find amplifier with gain range covering required_gain
170     3. Prefer smallest gain range (most efficient)
171     """
172     # Filter allowed amplifiers
173     allowed_amps = {
174         variety: params
175         for variety, params in edfa_library.items()
176         if params.allowed_for_design
177     }
178
179     if not allowed_amps:
180         logger.error("No amplifiers allowed for design")
181         return None
182
183     # Find amplifiers with suitable gain range
184     suitable_amps = {}
185     for variety, params in allowed_amps.items():
186         if (params.gain_min <= required_gain <=
187             params.gain_flatmax):
188             # Calculate gain range
189             gain_range = params.gain_flatmax - params.gain_min
190             suitable_amps[variety] = {
191                 'params': params,
192                 'range': gain_range,
193                 'margin': min(required_gain - params.gain_min,
194                             params.gain_flatmax - required_gain)

```

```

195         }
196
197     if suitable_amps:
198         # Select amplifier with best margin
199         best_variety = max(suitable_amps.items(),
200                           key=lambda x: x[1]['margin'])[0]
201         return best_variety
202
203     # No exact match - find closest
204     logger.warning(
205         f"No amplifier with gain range covering "
206         f"{required_gain:.1f} dB")
207
208     best_variety = None
209     best_score = float('inf')
210
211     for variety, params in allowed_amps.items():
212         # Score based on how far gain is from range
213         if required_gain < params.gain_min:
214             score = params.gain_min - required_gain
215         else: # required_gain > gain_flatmax
216             score = required_gain - params.gain_flatmax
217
218         if score < best_score:
219             best_score = score
220             best_variety = variety
221
222     logger.warning(
223         f"Using {best_variety} (gain range "
224         f"[{allowed_amps[best_variety].gain_min}, "
225         f"{allowed_amps[best_variety].gain_flatmax}]) "
226         f"for required gain {required_gain:.1f} dB")
227
228     return best_variety

```

2.4.4 Step 2: Set Fiber Attributes

```

1 def add_missing_fiber_attributes(network, equipment,
2                                   set_connector_losses=True):
3     """Set fiber connector losses and padding
4
5     Process:
6     1. Apply default connector losses (if not specified)
7     2. Calculate span loss with padding
8     3. Set design_span_loss attribute
9     """
10    span_config = equipment['Span']['default']
11
12    for node in network.nodes():
13        element = network.nodes[node]['element']
14
15        if not isinstance(element, Fiber):
16            continue
17
18        # Set connector losses if not specified
19        if set_connector_losses:
20            if not hasattr(element.params, 'con_in'):
21                element.params.con_in = 0.5 # dB

```

```

22         if not hasattr(element.params, 'con_out'):
23             element.params.con_out = 0.5 # dB
24
25         # Calculate total span loss
26         fiber_loss = element.params.loss_coef * \
27             element.params.length / 1000
28         conn_loss = element.params.con_in + element.params.con_out
29         padding = span_config.padding
30         eol = span_config.EOL
31
32         design_span_loss = fiber_loss + conn_loss + padding + eol
33
34         # Store as design parameter
35         element.design_span_loss = design_span_loss
36
37         logger.debug(
38             f"Fiber {node}: loss={fiber_loss:.2f} dB, "
39             f"connectors={conn_loss:.2f} dB, "
40             f"padding={padding:.2f} dB, "
41             f"EOL={eol:.2f} dB, "
42             f"total={design_span_loss:.2f} dB")

```

2.4.5 Step 3: Power Optimization (auto-design Algorithm)

Build Network Function

```

1 def build_network(network, equipment, reference_channel, verbose=True):
2     """Configure amplifier gains and powers using auto-design
3
4     Process:
5         1. Set ROADM equalization targets
6         2. Set amplifier gains (auto-design equation 4)
7         3. Optimize power profile (if power_mode enabled)
8         4. Configure output VOAs
9     """
10    logger.info("Building network configuration...")
11
12    # 1. Configure ROADMs
13    _configure_roadm_equalization(
14        network, equipment, reference_channel)
15
16    # 2. Set amplifier gains
17    _set_amplifier_gains(
18        network, equipment, reference_channel, verbose)
19
20    # 3. Optimize power profile (auto-design algorithm)
21    if equipment['Span']['default'].power_mode:
22        _optimize_power_profile(
23            network, equipment, reference_channel)
24
25    # 4. Set egress amplifiers
26    _set_egress_amplifiers(network, equipment)
27
28    logger.info("Network configuration complete")
29
30 def _configure_roadm_equalization(network, equipment, ref_channel):
31     """Configure ROADM channel equalization"""
32     roadm_params = equipment['Roadm']['default']

```

```

33
34     for node in network.nodes():
35         element = network.nodes[node]['element']
36
37         if not isinstance(element, Roadm):
38             continue
39
40         # Set target power per channel
41         element.params.target_pch_out_db = \
42             roadm_params.target_pch_out_db
43
44         # Set reference carrier power for equalization
45         element.ref_carrier_power = ref_channel.signal.mean()
46
47         logger.debug(
48             f"ROADM {node}: target_pch={element.params.
              target_pch_out_db} dBm")

```

auto-design Gain Setting (Equation 4)

```

1  def _set_amplifier_gains(network, equipment, ref_channel, verbose):
2      """Set amplifier gains using auto-design algorithm
3
4      Implements equation 4 from OFC 2019 paper:
5           $G_i = L_{span,i} + \Delta P_i$ 
6
7      Where:
8           $G_i$  = gain of amplifier i
9           $L_{span,i}$  = span loss before amplifier i
10          $\Delta P_i$  = power adjustment for optimization
11      """
12      span_config = equipment['Span']['default']
13      power_mode = span_config.power_mode
14
15      # Find all optical multiplex sections (OMS)
16      # OMS = path from ROADM/Transceiver to next ROADM/Transceiver
17      oms_list = _find_oms_sections(network)
18
19      logger.info(f"Found {len(oms_list)} OMS sections")
20
21      for oms_id, oms_path in enumerate(oms_list):
22          logger.debug(f"Processing OMS {oms_id}: {' -> '.join(oms_path)}")
23
24          # Process each element in OMS
25          for i, node in enumerate(oms_path):
26              element = network.nodes[node]['element']
27
28              if not isinstance(element, Edfa):
29                  continue
30
31              # Find preceding fiber
32              predecessors = list(network.predecessors(node))
33              if not predecessors:
34                  logger.warning(f"Amplifier {node} has no predecessor")
35                  continue
36
37              prev_node = predecessors[0]

```

```

38     prev_element = network.nodes[prev_node]['element']
39
40     if isinstance(prev_element, Fiber):
41         # Calculate required gain
42         span_loss = prev_element.design_span_loss
43
44         # auto-design algorithm equation 4:  $G = L_{span} + \Delta P$ 
45         if power_mode:
46             # Power optimization enabled
47             #  $\Delta P$  determined by optimization loop
48             delta_p = 0 # Initial value
49         else:
50             # Fixed power mode
51             delta_p = 0
52
53         target_gain = span_loss + delta_p
54
55         # Clip to amplifier limits
56         target_gain = np.clip(
57             target_gain,
58             element.params.gain_min,
59             element.params.gain_flatmax
60         )
61
62         # Set gain
63         element.effective_gain = target_gain
64         element.delta_p = delta_p
65
66         logger.debug(
67             f"Amplifier {node}: "
68             f"span_loss={span_loss:.2f} dB, "
69             f"delta_p={delta_p:.2f} dB, "
70             f"gain={target_gain:.2f} dB")
71
72         # Validate against power limit
73         _validate_amplifier_power(
74             element, ref_channel, verbose)
75
76 def _find_oms_sections(network):
77     """Identify optical multiplex sections
78
79     OMS = sequence from ROADM/Transceiver to next ROADM/Transceiver
80
81     Returns:
82         List of OMS paths, each path is list of node UUIDs
83     """
84     oms_list = []
85
86     # Find all ROADM and Transceiver nodes
87     oms_boundaries = [
88         node for node in network.nodes()
89         if isinstance(network.nodes[node]['element'],
90             (Roadm, Transceiver))
91     ]
92
93     # For each boundary, find paths to next boundary
94     for start_node in oms_boundaries:

```

```

95     # Use BFS to find next boundary
96     visited = set()
97     queue = [(start_node, [start_node])]
98
99     while queue:
100         current, path = queue.pop(0)
101
102         if current in visited and current != start_node:
103             continue
104         visited.add(current)
105
106         successors = list(network.successors(current))
107
108         for succ in successors:
109             succ_element = network.nodes[succ]['element']
110             new_path = path + [succ]
111
112             if isinstance(succ_element, (Roadm, Transceiver)):
113                 # Found OMS boundary
114                 if len(new_path) > 2: # Non-trivial path
115                     oms_list.append(new_path)
116             else:
117                 # Continue search
118                 queue.append((succ, new_path))
119
120     return oms_list
121
122 def _validate_amplifier_power(amp, ref_channel, verbose):
123     """Check if amplifier output power within limits"""
124     # Calculate expected output power
125     n_channels = ref_channel.number_of_channels
126     power_per_channel = ref_channel.signal.mean() # W
127
128     # Total output power
129     power_out = power_per_channel * n_channels # W
130     power_out_dbm = 10 * np.log10(power_out * 1000) # dBm
131
132     # Check against p_max
133     if power_out_dbm > amp.params.p_max:
134         if verbose:
135             logger.warning(
136                 f"Amplifier {amp.uid}: output power "
137                 f"({power_out_dbm:.1f} dBm) exceeds p_max "
138                 f"({amp.params.p_max} dBm)")

```

Power Profile Optimization

```

1 def _optimize_power_profile(network, equipment, ref_channel):
2     """Optimize per-span launch power (auto-design global optimization)
3
4     Objective: Minimize OSNR variation across spans
5
6     Method: Iterative adjustment of delta_power per span
7
8     Constraints:
9         - Amplifier power limits (p_max)
10        - Amplifier gain limits [gain_min, gain_flatmax]
11        - Delta power range from Span config

```

```

12 """
13 span_config = equipment['Span']['default']
14 dp_min, dp_max, dp_step = span_config.delta_power_range_db
15
16 logger.info("Starting power profile optimization...")
17
18 # Get all amplifiers
19 amplifiers = [
20     (node, network.nodes[node]['element'])
21     for node in network.nodes()
22     if isinstance(network.nodes[node]['element'], Edfa)
23 ]
24
25 # Initialize delta_powers
26 delta_powers = {node: 0.0 for node, _ in amplifiers}
27
28 # Iterative optimization
29 max_iterations = 10
30 convergence_threshold = 0.5 # dB
31
32 for iteration in range(max_iterations):
33     logger.info(f"Optimization iteration {iteration + 1}")
34
35     # Propagate with current delta_powers
36     osnr_results = _propagate_and_evaluate_osnr(
37         network, equipment, ref_channel, delta_powers)
38
39     # Calculate OSNR variation
40     osnr_mean = np.mean(osnr_results)
41     osnr_std = np.std(osnr_results)
42     osnr_min = np.min(osnr_results)
43     osnr_max = np.max(osnr_results)
44
45     logger.info(
46         f" OSNR: mean={osnr_mean:.2f} dB, "
47         f"std={osnr_std:.2f} dB, "
48         f"range=[{osnr_min:.2f}, {osnr_max:.2f}] dB")
49
50     # Check convergence
51     if osnr_std < convergence_threshold:
52         logger.info("Power optimization converged")
53         break
54
55     # Update delta_powers
56     # Heuristic: increase power for low-OSNR spans
57     for node, amp in amplifiers:
58         span_osnr = osnr_results[node]
59
60         # Adjustment based on OSNR deviation
61         if span_osnr < osnr_mean - osnr_std:
62             # Low OSNR - increase power
63             adjustment = +dp_step
64         elif span_osnr > osnr_mean + osnr_std:
65             # High OSNR - decrease power (or keep)
66             adjustment = -dp_step
67         else:
68             # Near mean - no change
69             adjustment = 0

```

```

70
71     # Apply adjustment
72     new_dp = delta_powers[node] + adjustment
73     delta_powers[node] = np.clip(new_dp, dp_min, dp_max)
74
75     # Update amplifier gains
76     for node, amp in amplifiers:
77         dp = delta_powers[node]
78
79         # Find preceding fiber
80         pred = list(network.predecessors(node))[0]
81         pred_element = network.nodes[pred]['element']
82
83         if isinstance(pred_element, Fiber):
84             span_loss = pred_element.design_span_loss
85             target_gain = span_loss + dp
86
87             # Clip to amplifier limits
88             target_gain = np.clip(
89                 target_gain,
90                 amp.params.gain_min,
91                 amp.params.gain_flatmax
92             )
93
94             amp.effective_gain = target_gain
95             amp.delta_p = dp
96
97     # Log final delta_powers
98     logger.info("Final power adjustments:")
99     for node, dp in delta_powers.items():
100         logger.info(f"    {node}: delta_p = {dp:.2f} dB")
101
102 def _propagate_and_evaluate_osnr(network, equipment, ref_channel,
103                                 delta_powers):
104     """Propagate signal and evaluate OSNR per span
105
106     Simplified propagation for optimization loop
107     """
108     # This is a simplified version
109     # Full implementation would use actual propagation engine
110
111     osnr_per_span = {}
112
113     for node in network.nodes():
114         element = network.nodes[node]['element']
115
116         if isinstance(element, Edfa):
117             # Estimate OSNR contribution
118             # (Simplified - full version uses NLI calculation)
119
120             gain = element.effective_gain
121             nf = element.params.nf_min + \
122                 (element.params.nf_max - element.params.nf_min) * \
123                 (gain / element.params.gain_flatmax)
124
125             # ASE noise
126             h = 6.62607015e-34
127             nu = 193.1e12

```

```

128         delta_f = 12.5e9
129         ase = (10**(gain/10) - 1) * 10**(nf/10) * h * nu * delta_f
130
131         # OSNR estimation (simplified)
132         signal = ref_channel.signal.mean()
133         osnr = 10 * np.log10(signal / ase)
134
135         osnr_per_span[node] = osnr
136
137     return osnr_per_span

```

2.4.6 Output VOA Configuration

```

1 def _set_egress_amplifiers(network, equipment):
2     """Configure egress amplifiers and output VOAs
3
4     Egress amplifier = amplifier before ROADM or receiver
5
6     Strategy:
7     - Maximize gain within p_max limit
8     - Use VOA to attenuate excess
9     """
10    for node in network.nodes():
11        element = network.nodes[node]['element']
12
13        if not isinstance(element, Edfa):
14            continue
15
16        # Check if egress (before ROADM/Transceiver)
17        successors = list(network.successors(node))
18        if not successors:
19            continue
20
21        next_node = successors[0]
22        next_element = network.nodes[next_node]['element']
23
24        if isinstance(next_element, (Roadm, Transceiver)):
25            # This is an egress amplifier
26
27            if element.params.out_voa_auto:
28                # Configure automatic VOA
29
30                # Set gain to maximum
31                element.effective_gain = element.params.gain_flatmax
32
33                # Calculate required VOA
34                # (Simplified - should consider target power)
35                voa = element.params.gain_flatmax - \
36                    element.effective_gain
37
38                # Apply VOA margin
39                if hasattr(element.params, 'voa_margin'):
40                    voa += element.params.voa_margin
41
42                # Quantize to VOA step
43                if hasattr(element.params, 'voa_step'):
44                    voa = np.ceil(voa / element.params.voa_step) * \
45                        element.params.voa_step

```

```

46         element.out_voa = max(0, voa)
47
48
49     logger.debug(
50         f"Egress amplifier {node}: "
51         f"gain={element.effective_gain:.2f} dB, "
52         f"VOA={element.out_voa:.2f} dB")

```

2.5. OPENROADM ALIGNMENT

2.5.1 Equipment Library Mapping

GNPy Equipment	OpenROADM Model
Edfa[type_variety]	ILA/preamp/booster operational modes
Fiber[type_variety]	Span attributes + ITU-T fiber types
Transceiver[mode]	specific-operational-modes
Roadm	ROADM degree impairments
SI['default']	Default transmission parameters
Span	Design constraints (not in OpenROADM)

Table 2.2: Equipment library to OpenROADM mapping

2.5.2 Network Topology Mapping

GNPy Structure	OpenROADM Network Model
DiGraph nodes	Network topology nodes (devices)
DiGraph edges	Express/add/drop links
Node uid	node-id
Node metadata	CLLI codes, location
Element params	Operational parameters

Table 2.3: Network graph to OpenROADM mapping

2.5.3 Design Output Mapping

GNPy Auto-Design	OpenROADM Planning
Amplifier placement	initially-planned-nodes
effective_gain	initially-planned-gain
delta_p	Power offset vs. nominal
out_voa	out-voa-att
target_pch_out_dbm	egress-average-channel-power
design_span_loss	span-loss-transmit/receive

Table 2.4: Auto-design to OpenROADM provisioning mapping

2.6. IMPLEMENTATION CONSIDERATIONS

2.6.1 Performance Optimization

1. Graph operations:

- Use NetworkX built-in algorithms
- Cache node/edge lookups
- Minimize graph modifications during iteration

2. Validation:

- Perform early validation (fail fast)
 - Use logging levels appropriately
 - Provide actionable error messages
3. **Memory management:**
- Avoid deep copies of large structures
 - Use generators for path enumeration
 - Clear temporary data after use

2.6.2 Testing Strategies

```

1 def test_load_equipment():
2     """Unit test for equipment loading"""
3     # Test valid configuration
4     equipment = load_equipment('test_eqpt.json')
5     assert 'Edfa' in equipment
6     assert 'Fiber' in equipment
7
8     # Test invalid configuration
9     with pytest.raises(EquipmentConfigError):
10         load_equipment('invalid_eqpt.json')
11
12 def test_load_network():
13     """Unit test for network loading"""
14     equipment = load_equipment('test_eqpt.json')
15     network = load_network('test_topo.json', equipment)
16
17     # Check graph structure
18     assert network.number_of_nodes() > 0
19     assert network.number_of_edges() > 0
20
21     # Check node attributes
22     for node in network.nodes():
23         assert 'element' in network.nodes[node]
24
25 def test_design_network():
26     """Unit test for network design"""
27     equipment = load_equipment('test_eqpt.json')
28     network = load_network('test_topo.json', equipment)
29
30     # Create reference channel
31     ref_channel = create_test_spectral_info()
32
33     # Design network
34     design_network(ref_channel, network, equipment)
35
36     # Validate design
37     for node in network.nodes():
38         element = network.nodes[node]['element']
39         if isinstance(element, Edfa):
40             assert hasattr(element, 'effective_gain')
41             assert element.params.gain_min <= \
42                 element.effective_gain <= \
43                 element.params.gain_flatmax

```

2.6.3 Error Handling Best Practices

1. Specific exceptions:

```

1 class EquipmentConfigError(Exception):
2     """Equipment configuration is invalid"""
3     pass
4
5 class NetworkTopologyError(Exception):
6     """Network topology is invalid"""
7     pass
8
9 class DesignError(Exception):
10     """Network design failed"""
11     pass

```

2. Context in error messages:

```

1 # Bad
2 raise ValueError("Invalid gain")
3
4 # Good
5 raise EquipmentConfigError(
6     f"Amplifier {variety}: gain_min ({gain_min}) "
7     f"must be < gain_flatmax ({gain_flatmax})")

```

3. Graceful degradation:

```

1 try:
2     design_network(ref_channel, network, equipment)
3 except DesignError as e:
4     logger.error(f"Auto-design failed: {e}")
5     logger.info("Proceeding with manual configuration")
6     # Use network without auto-design

```

2.7. SUMMARY

2.7.1 Key Takeaways

1. **Three-stage pipeline:**
 - Stage 1: Parse and validate equipment library
 - Stage 2: Build network graph with elements
 - Stage 3: Auto-design algorithm
2. **auto-design algorithm:**
 - Equation 4: $G_i = L_{\text{span},i} + \Delta P_i$
 - Iterative power optimization
 - Constraint satisfaction
3. **Validation is critical:**
 - Equipment parameters
 - Network connectivity
 - Design feasibility
4. **OpenROADM compliance:**
 - Equipment catalog mapping
 - Network model alignment
 - Provisioning parameter export

2.7.2 Next Chapter Preview

Chapter 3 will cover:

- Path computation algorithms
- Constraint handling
- Spectrum assignment
- Mode selection strategies

- OSNR-optimized routing

Chapter 3

Path Computation & Spectrum Assignment

CHAPTER OVERVIEW

This chapter analyzes GNPY's path computation and spectrum assignment mechanisms. It covers constrained path finding algorithms, spectrum allocation strategies, mode selection procedures, and the critical distinction between topology-based path selection and quality-of-transmission validation. Understanding these workflows is essential for implementing optical network planning tools.

3.1. OVERVIEW

Path computation and spectrum assignment in optical networks is a multi-stage process that must address:

1. **Topological Constraints:** Finding paths that satisfy user-specified include/exclude node requirements
2. **Spectrum Allocation:** Assigning non-overlapping frequency slots across shared fiber segments
3. **Mode Selection:** Choosing modulation formats and baud rates that meet QoT requirements
4. **QoT Validation:** Verifying GSNR meets minimum thresholds after propagation

Key Implementation Insight: GNPpy separates path finding (topology-based) from QoT evaluation (propagation-based). A single path is selected first using graph algorithms, then validated via physical layer simulation. This differs from joint optimization approaches.

3.2. PATH COMPUTATION

3.2.1 Algorithm Foundation

GNPy uses NetworkX graph algorithms for path computation. The network is represented as a directed graph $G = (V, E)$ where:

- V : Set of network elements (transceivers, ROADMs, fibers, EDFAs)
- E : Set of directed edges representing connectivity
- Edge weights: Determined by hop count or user-defined metrics

3.2.2 Core Function: `compute_constrained_path`

Location: `gnpy/topology/request.py` line 297

Purpose: Find a single path from source to destination satisfying routing constraints

Algorithm Flow

Listing 3.1: Constrained Path Computation

```

1 def compute_constrained_path(network, req):
2     """
3     Computes path satisfying:
4     - Source/destination transceivers
5     - Include nodes (loose/strict)
6     - Exclude nodes
7
8     Returns: Single path (list of network elements)
9     """
10    # 1. Extract transceivers
11    trx = [n for n in network if isinstance(n, Transceiver)]
12    source = next(el for el in trx if el.uid == req.source)
13    destination = next(el for el in trx if el.uid == req.destination)
14
15    # 2. Handle explicit routing if provided
16    if req.nodes_list:
17        nodes_list = []
18        for node in req.nodes_list[:-1]:
19            nodes_list.append(next(el for el in network
20                                if el.uid == node))
21        total_path = explicit_path(nodes_list, source,
22                                destination, network)
23        if total_path is not None:
24            return total_path
25
26    # 3. Constrained shortest path search

```

```

27     try:
28         # LINE 317-318: Key algorithm call
29         path_generator = shortest_simple_paths(network, source,
30                                             destination,
31                                             weight='weight')
32         total_path = next(path for path in path_generator
33                           if ispart(nodes_list, path))
34     except NetworkXNoPath:
35         req.blocking_reason = 'NO_PATH'
36         total_path = []
37     except StopIteration:
38         # No path satisfies constraints
39         if 'STRICT' not in req.loose_list[:-1]:
40             # Try without constraints
41             total_path = dijkstra_path(network, source,
42                                     destination, weight='weight')
43         else:
44             req.blocking_reason = 'NO_PATH_WITH_CONSTRAINT'
45             total_path = []
46
47     return total_path

```

Critical Implementation Details

Line 317-318 Analysis:

```

1 path_generator = shortest_simple_paths(network, source, destination,
2                                     weight='weight')
3 total_path = next(path for path in path_generator
4                   if ispart(nodes_list, path))

```

What This Does:

1. `shortest_simple_paths`: NetworkX generator that yields paths in order of increasing cost
2. `next(...if ispart...)`: Returns the **first** path satisfying node constraints
3. Stops immediately after finding one feasible path

Critical Implication: GSNR is **NOT** considered during path selection. The algorithm selects paths based purely on:

- Edge weights (typically hop count)
- Constraint satisfaction

3.2.3 Path Selection vs. QoT Validation

This is a crucial architectural decision in GNPpy:

Stage	Path Selection	QoT Validation
Location	<code>compute_constrained_path</code>	<code>propagate</code> / <code>propagate_and_optimize</code>
Input	Network topology, constraints	Selected path, equipment
Algorithm	Dijkstra, shortest paths	Physical layer propagation
Criteria	Hop count, constraints	GSNR, penalties
Output	Single path	PASS/FAIL + metrics
Complexity	$O(E + V \log V)$	$O(N_{elements} \times N_{channels})$

Table 3.1: Path Selection vs. QoT Validation

Consequence: Alternative paths with better GSNR are never explored if a shorter path exists.

Example Scenario:

- Path 1: A → B → Z (2 hops, GSNR = 16.5 dB, margin = 1.5 dB)
- Path 2: A → C → D → Z (3 hops, GSNR = 20.0 dB, margin = 5.0 dB)
- **Result:** Path 1 selected despite inferior GSNR

3.2.4 Routing Constraints*Include Nodes*

Specified via `explicit-route-objects` in service request:

Listing 3.2: Include Node Constraint

```

1 "explicit-route-objects": [
2   {
3     "index": 0,
4     "num-unnum-hop": {
5       "node-id": "Lyon",
6       "link-tp-id": "Lyon"
7     },
8     "type": "include",
9     "loose": true  # or false for strict
10  }
11 ]

```

Loose vs. Strict:

- **Loose:** Path may traverse other nodes between constraints
- **Strict:** Path must go directly between specified nodes

Exclude Nodes

```

1 "explicit-route-objects": [
2   {
3     "num-unnum-hop": {"node-id": "BadNode"},
4     "type": "exclude"
5   }
6 ]

```

Implementation removes excluded nodes before path search.

3.2.5 Disjoint Path Computation

For protection/restoration scenarios, multiple disjoint paths are required.

Function: `compute_path_dsjctn` (request.py)

Disjunction Types:

- **Node-disjoint:** No common nodes (except endpoints)
- **Link-disjoint:** No common fiber spans
- **SRLG-disjoint:** No shared risk link groups

Algorithm:

1. Compute primary path
2. Remove disjoint elements from graph
3. Compute secondary path on reduced graph
4. Verify disjunction criteria

3.3. SPECTRUM ASSIGNMENT**3.3.1 Optical Multiplex Section (OMS)**

An OMS represents a fiber segment with uniform spectrum usage:

Definition: Contiguous fiber path between regeneration points where all channels share spectrum

OMS Boundaries:

- Degree boundaries of ROADMs
- Transponder/regenerator interfaces
- Wavelength add/drop points

OMS List Construction

Function: `build_oms_list` (`spectrum_assignment.py`)

Listing 3.3: OMS Construction

```

1 def build_oms_list(network, equipment):
2     """
3     Identify all OMS segments in network
4
5     Returns: List of OMS objects, each containing:
6         - element_list: Network elements in segment
7         - frequency_slots: Available spectrum bitmap
8     """
9     oms_list = []
10
11     for oms_element in find_oms_elements(network):
12         oms = OMS()
13         oms.el_list = []
14
15         # Traverse from OMS start to end
16         current = oms_element
17         while not is_oms_boundary(current):
18             oms.el_list.append(current)
19             current = next_element(current, network)
20
21         # Initialize spectrum bitmap
22         oms.frequency_slots = create_spectrum_bitmap(equipment)
23         oms_list.append(oms)
24
25     return oms_list

```

3.3.2 Spectrum Slot Model**Flexible Grid (ITU-T G.694.1):**

- Central frequency grid: $193.1 \text{ THz} + n \times 6.25 \text{ GHz}$
- Slot width: $m \times 12.5 \text{ GHz}$
- (n, m) notation: n = center slot, m = number of slots

Example:

32 Gbaud channel, 50 GHz spacing $\implies m = 4$ slots

Center at 193.1 THz $\implies n = 0$

3.3.3 Path Spectrum Assignment

Function: `pth_assign_spectrum` (`spectrum_assignment.py`)

Algorithm: First-Fit Strategy

Listing 3.4: First-Fit Spectrum Assignment

```

1 def pth_assign_spectrum(paths, requests, oms_list, reversed_paths):
2     """
3     Assign spectrum to service requests using first-fit
4

```

```

5   For each request:
6   1. Calculate required slots (M)
7   2. Find first available starting slot (N)
8   3. Verify availability across all OMS in path
9   4. Mark slots as occupied
10  """
11  for path, request, reversed_path in zip(paths, requests,
12                                          reversed_paths):
13      if hasattr(request, 'blocking_reason'):
14          request.N = None
15          request.M = None
16          continue
17
18      # Calculate spectrum requirement
19      nb_channels, required_m = compute_spectrum_slot_vs_bandwidth(
20          request.path_bandwidth,
21          request.spacing,
22          request.bit_rate
23      )
24
25      # Find OMS segments traversed by path
26      path_oms_ids = build_path_oms_id_list(path + reversed_path)
27
28      # First-fit search across all OMS
29      if request.M is not None:
30          # User specified slot allocation
31          N, M, remaining = verify_user_spectrum(
32              request.N, request.M, path_oms_ids, oms_list
33          )
34      else:
35          # Automatic assignment
36          N, M, remaining = spectrum_selection(
37              path_oms_ids, oms_list, required_m, nb_channels
38          )
39
40      if remaining > 0:
41          request.blocking_reason = 'NOT_ENOUGH_SPECTRUM'
42          request.N = None
43          request.M = None
44      else:
45          request.N = N
46          request.M = M
47          # Update OMS bitmaps
48          for oms_id in path_oms_ids:
49              oms_list[oms_id].assign_spectrum(N, M)

```

Spectrum Selection Function

Listing 3.5: First-Fit Slot Selection

```

1  def spectrum_selection(oms_list, path_oms_ids, M, policy='first_fit'):
2      """
3      Find first available N for required M slots
4
5      Args:
6          oms_list: All OMS segments
7          path_oms_ids: OMS IDs in current path
8          M: Number of contiguous slots needed
9

```

```

10 Returns:
11     (N, M, feasible): Starting slot, width, feasibility flag
12     """
13     # Aggregate spectrum availability across path
14     composite_bitmap = aggregate_oms_bitmap(path_oms_ids, oms_list)
15
16     # Scan for first available slot
17     for n in range(composite_bitmap.max_slots - M + 1):
18         if is_available(composite_bitmap, n, M):
19             return n, M, True
20
21     # No available spectrum
22     return None, None, False
23
24 def is_available(bitmap, n, m):
25     """Check if slots [n, n+m) are all free"""
26     return all(bitmap.slots[n:n+m] == 0)

```

3.3.4 Bidirectional Spectrum Assignment

For bidirectional services, spectrum must be consistent in both directions:

Listing 3.6: Bidirectional Assignment

```

1 if request.bidirectional:
2     # Find OMS in both directions
3     forward_oms = build_path_oms_id_list(path)
4     reverse_oms = build_path_oms_id_list(reversed_path)
5
6     # Union of OMS IDs
7     all_oms = list(set(forward_oms + reverse_oms))
8
9     # Assign same spectrum in both directions
10    N, M, remaining = spectrum_selection(all_oms, oms_list, required_m)
11
12    for oms_id in all_oms:
13        oms_list[oms_id].assign_spectrum(N, M)

```

3.4. MODE SELECTION

3.4.1 Transponder Modes

Each transponder type supports multiple transmission modes:

Format	Bit Rate	Baud Rate	OSNR _{req}	Spacing _{min}
DP-64QAM	600 Gb/s	69 GBd	28 dB	75 GHz
DP-16QAM	400 Gb/s	69 GBd	23 dB	75 GHz
DP-8QAM	300 Gb/s	69 GBd	19 dB	75 GHz
DP-QPSK	200 Gb/s	69 GBd	15 dB	75 GHz

Table 3.2: Example Transponder Modes

3.4.2 Mode Selection Algorithm

Function: `propagate_and_optimize_mode` (request.py line 392)

Selection Strategy: First-Fit

Listing 3.7: First-Fit Mode Selection

```

1 def propagate_and_optimize_mode(path, request, equipment):
2     """
3     Try modes in order until one meets OSNR requirement
4
5     Key behavior (line 409 comment):
6     "stop and select FIRST mode that passes"
7     """
8     # 1. Get available modes for this transceiver
9     modes = equipment['Transceiver'][request.tsp_type].mode
10
11    # 2. Filter by spacing constraint
12    modes = [m for m in modes
13              if m['min_spacing'] <= request.spacing]
14
15    # 3. Sort modes (lines 407-408)
16    modes = sorted(modes,
17                   key=lambda x: (x['bit_rate'],          # Descending
18                                 x['baud_rate'],          # Descending
19                                 x['equalization_offset_db']), #
20                   reverse=True)
21
22    # 4. Try each mode until one passes
23    last_explored_mode = None
24
25    for mode in modes:
26        # Update spectral information with mode parameters
27        si = create_input_spectral_information(
28            mode['baud_rate'],
29            mode['spacing'],
30            mode['tx_osnr']
31        )
32
33        # Propagate through path
34        for element in path:
35            element(si) # Updates si with impairments
36
37        # Check GSNR at destination (lines 440-442)
38        destination = path[-1]
39        gsnr_effective = destination.snr_01nm - destination.
40            total_penalty
41
42        if round(gsnr_effective, 2) >= mode['OSNR'] + sys_margins:
43            # FIRST MODE THAT PASSES - STOP HERE
44            return path, mode
45
46        last_explored_mode = mode
47
48    # No mode feasible (line 453)
49    request.blocking_reason = 'NO_FEASIBLE_MODE'
50    return path, last_explored_mode

```

*Critical Implementation Note***First-Fit vs. Best-Fit:**

The algorithm stops at the **first** mode that meets OSNR requirements. It does **NOT**:

- Continue checking lower modes for better margin

- Optimize for maximum GSNR margin
- Minimize cost or power consumption

Rationale:

- Prioritizes throughput (higher bit rates checked first)
- Reduces computation time
- Assumes modes are pre-sorted by preference

Example:

- Mode 1 (DP-16QAM 400G): GSNR = 19.5 dB, requires 20 dB → FAIL
- Mode 2 (DP-8QAM 300G): GSNR = 18.2 dB, requires 17 dB → **PASS (selected)**
- Mode 3 (DP-QPSK 200G): GSNR = 22.0 dB, requires 15 dB → **Never tested**
Mode 3 has 7 dB margin vs. Mode 2's 1.2 dB, but is never evaluated.

3.4.3 GSNR Calculation with Penalties

Listing 3.8: Effective GSNR Calculation

```

1 def calculate_effective_gsnr(transceiver, mode):
2     """
3     Apply transmission penalties to OSNR
4
5     From Transceiver class (__call__ method)
6     """
7     # Base GSNR from propagation
8     gsnr_01nm = transceiver.osnr_ase_01nm + transceiver.osnr_nli_01nm
9
10    # Apply penalties from mode configuration
11    penalties = mode['penalties']
12
13    chromatic_dispersion_penalty = calc_cd_penalty(
14        transceiver.cd, # ps/nm
15        mode['baud_rate'],
16        penalties.get('chromatic_dispersion', 0)
17    )
18
19    pmd_penalty = calc_pmd_penalty(
20        transceiver.pmd, # ps
21        mode['baud_rate'],
22        penalties.get('pmd', 0)
23    )
24
25    pdl_penalty = penalties.get('pdl', 0) # dB
26
27    total_penalty = (chromatic_dispersion_penalty +
28                    pmd_penalty +
29                    pdl_penalty)
30
31    # Effective GSNR
32    gsnr_effective = gsnr_01nm - total_penalty
33
34    return gsnr_effective, total_penalty

```

3.5. INTEGRATION: COMPLETE WORKFLOW**3.5.1 Service Request Processing**

Listing 3.9: gnpy-path-request Main Loop

```

1 # Load inputs

```

```

2 equipment = load_equipment(eqpt_config_file)
3 network = load_network(topology_file, equipment)
4 network = design_network(network, equipment) # Auto-design
5 requests = load_requests(services_file)
6 oms_list = build_oms_list(network, equipment)
7
8 # Process each request
9 path_results = []
10
11 for request in requests:
12     # 1. PATH COMPUTATION (topology-based)
13     path = compute_constrained_path(network, request)
14
15     if not path:
16         # No topological path found
17         result = PathResult(request, path=[],
18                             blocking_reason=request.blocking_reason)
19         path_results.append(result)
20         continue
21
22     # 2. SPECTRUM ASSIGNMENT (if not user-specified)
23     reversed_path = find_reversed_path(path)
24     pth_assign_spectrum([path], [request], oms_list, [reversed_path])
25
26     if hasattr(request, 'blocking_reason'):
27         # Spectrum assignment failed
28         result = PathResult(request, path,
29                             blocking_reason=request.blocking_reason)
30         path_results.append(result)
31         continue
32
33     # 3. MODE SELECTION & PROPAGATION (QoT-based)
34     if request.baud_rate is None:
35         # Mode not specified - optimize
36         path, mode = propagate_and_optimize_mode(path, request,
37                                                  equipment)
38     else:
39         # Mode specified - just propagate
40         propagate(path, request, equipment)
41         mode = get_mode_from_request(request, equipment)
42
43     # 4. BIDIRECTIONAL CHECK (if required)
44     if request.bidirectional and not hasattr(request, 'blocking_reason'):
45         propagate(reversed_path, request, equipment)
46         # Check both directions pass
47         if not check_bidirectional_feasibility(path, reversed_path,
48                                                request):
49             request.blocking_reason = 'REVERSE_DIRECTION_INFEASIBLE'
50
51     # 5. RESULT COMPILATION
52     result = PathResult(
53         request_id=request.request_id,
54         path=path,
55         mode=mode,
56         spectrum=(request.N, request.M),
57         metrics=extract_metrics(path, request),
58         feasible=(not hasattr(request, 'blocking_reason'))

```

```

57 )
58 path_results.append(result)
59
60 # 6. OUTPUT GENERATION
61 write_json_response(path_results, output_file)
62 write_csv_report(path_results, csv_file) # Optional

```

3.5.2 Blocking Reasons

Blocking Reason	Cause
NO_PATH	No topological path exists between source and destination
NO_PATH_WITH_CONSTRAINT	Path exists but violates strict routing constraints
NOT_ENOUGH_SPECTRUM	All frequency slots occupied on one or more OMS
NOT_ENOUGH_RESERVED_SPECTRUM	User-specified (N, M) insufficient for bandwidth
NO_FEASIBLE_BAUDRATE_WITH_SPACING	Transponder mode has baud rate $\leq$ spacing
NO_FEASIBLE_MODE	All modes fail GSNR requirement
MODE_NOT_FEASIBLE	User-specified mode fails GSNR requirement
REVERSE_DIRECTION_INFEASIBLE	Bidirectional service: reverse path fails

Table 3.3: Service Blocking Reasons

3.6. PERFORMANCE CONSIDERATIONS

3.6.1 Computational Complexity

Operation	Complexity	Dominant Factor
Path computation	$O(E + V \log V)$	Graph size
Spectrum assignment	$O(R \cdot S)$	Requests $\times$ Slots
Mode selection	$O(M \cdot E \cdot C)$	Modes $\times$ Elements $\times$ Channels
Propagation	$O(E \cdot C)$	Elements $\times$ Channels

Table 3.4: Algorithmic Complexity

Where:

- V : Number of network nodes
- E : Number of network edges
- R : Number of service requests
- S : Number of spectrum slots
- M : Number of transponder modes
- C : Number of optical channels

3.6.2 Optimization Opportunities

Path Computation

Current Limitation: Single path selection without QoT consideration

Improvement Strategies:

1. **K-Shortest Paths with QoT:** Compute k shortest paths, propagate all, select best GSNR
2. **GSNR-Weighted Edges:** Pre-compute approximate GSNR for each edge, use as weights
3. **Machine Learning:** Train models to predict path GSNR from topology

Mode Selection

Current Limitation: First-fit stops at first feasible mode

Improvement Strategies:

1. **Best-Fit:** Evaluate all modes, select optimal for margin or cost
2. **Cost-Aware:** Consider transponder cost in selection
3. **Margin-Aware:** Select mode with specified margin target

Spectrum Assignment

Current Limitation: First-fit may cause fragmentation

Improvement Strategies:

1. **Best-Fit:** Select spectrum slot that minimizes fragmentation
2. **Defragmentation:** Periodically reorganize spectrum allocations
3. **Graph Coloring:** Treat spectrum assignment as graph coloring problem

3.7. PRACTICAL EXAMPLES

3.7.1 Example 1: Simple Path Request

Service Request:

```

1 {
2     "request-id": "req001",
3     "source": "NodeA",
4     "destination": "NodeZ",
5     "path-constraints": {
6         "te-bandwidth": {
7             "trx_type": "Voyager",
8             "spacing": 75e9
9         }
10    }
11 }
```

Processing:

1. **Path Computation:** Dijkstra finds shortest path: $A \rightarrow B \rightarrow C \rightarrow Z$ (3 hops)
2. **Spectrum Assignment:** First-fit finds $(N = 0, M = 6)$ available
3. **Mode Selection:**
 - Try DP-16QAM 400G: $\text{GSNR} = 22.5 \text{ dB} \geq 23 \text{ dB}$ required $\rightarrow$ **FAIL**
 - Try DP-8QAM 300G: $\text{GSNR} = 19.8 \text{ dB} \geq 19 \text{ dB}$ required $\rightarrow$ **PASS**
4. **Result:** Service established with DP-8QAM, $(N = 0, M = 6)$, $\text{GSNR margin} = 0.8 \text{ dB}$

3.7.2 Example 2: Constrained Routing

Service Request:

```

1 {
2     "request-id": "req002",
3     "source": "Paris",
4     "destination": "Marseille",
5     "path-constraints": {
6         "te-bandwidth": {"trx_type": "Voyager"},
7         "explicit-route-objects": [
8             {"num-unnum-hop": {"node-id": "Lyon"},
9              "type": "include", "loose": true},
10            {"num-unnum-hop": {"node-id": "Bordeaux"},
11             "type": "exclude"}
12        ]
13    }
14 }
```

Processing:

1. Remove Bordeaux from graph
2. Search for paths Paris $\rightarrow$... $\rightarrow$ Lyon $\rightarrow$... $\rightarrow$ Marseille
3. If found: continue with spectrum assignment and mode selection
4. If not found: `blocking_reason = 'NO_PATH_WITH_CONSTRAINT'`

3.7.3 Example 3: Spectrum Exhaustion**Scenario:**

- OMS has 80 slots total
- 18 existing services occupy slots 0-71 (72 slots used)
- New request requires 10 slots

Processing:

1. Path computed successfully
2. Spectrum assignment searches for 10 contiguous free slots
3. Finds slots 72-81 (only 8 available)
4. **Result:** `blocking_reason = 'NOT_ENOUGH_SPECTRUM'`

3.7.4 Example 4: Bidirectional Service**Service Request:**

```

1 {
2     "request-id": "bidir001",
3     "source": "NodeA",
4     "destination": "NodeZ",
5     "bidirectional": true,
6     "path-constraints": {
7         "te-bandwidth": {"trx_type": "Voyager", "trx_mode": "DP-QPSK"}
8     }
9 }
```

Processing:

1. Compute forward path: A $\rightarrow$ B $\rightarrow$ C $\rightarrow$ Z
2. Compute reverse path: Z $\rightarrow$ C $\rightarrow$ B $\rightarrow$ A
3. Assign same spectrum in both directions
4. Propagate forward: $\text{GSNR} = 17.5 \text{ dB} \geq 15 \text{ dB} \rightarrow$ **PASS**
5. Propagate reverse: $\text{GSNR} = 16.2 \text{ dB} \geq 15 \text{ dB} \rightarrow$ **PASS**
6. **Result:** Bidirectional service established

3.8. SUMMARY**3.8.1 Key Takeaways**

1. **Separation of Concerns:** Path selection is topology-based; QoT validation is physics-based
2. **Single Path Selection:** Only one path computed per request (not multi-path optimization)
3. **First-Fit Strategies:** Both spectrum and mode selection use first-fit, not best-fit
4. **Sequential Processing:** Path $\rightarrow$ Spectrum $\rightarrow$ Mode $\rightarrow$ Propagation (no backtracking)
5. **Blocking Reasons:** Detailed failure diagnostics for network planning

3.8.2 Implementation Guidance

For developers implementing similar systems:

1. **Graph Library:** Use mature libraries (NetworkX, Boost Graph) for path algorithms
2. **Spectrum Data Structure:** Bitmap representation efficient for first-fit search
3. **Mode Ordering:** Pre-sort modes by preference to enable first-fit logic
4. **Caching:** Cache propagation results for common path segments
5. **Parallel Processing:** Requests are independent - parallelize for large networks

3.8.3 Extensions

To extend beyond GNPpy's current implementation:

1. **Joint Optimization:** Integrate path selection with QoT prediction
2. **Multi-Path:** Support path diversity (primary + backup)
3. **Dynamic Networks:** Handle time-varying topology and spectrum usage
4. **Cost Optimization:** Minimize total network cost (transponders + amplifiers)
5. **Multi-Objective:** Balance throughput, cost, and resilience

Chapter 4

Propagation & QoT Computation

CHAPTER OVERVIEW

This chapter provides a detailed analysis of signal propagation through optical network elements and quality-of-transmission (QoT) computation in GNP_y. It covers the `SpectralInformation` data structure, element-specific propagation models, nonlinear interference calculation, and GSNR computation at receivers. Understanding these mechanisms is essential for implementing accurate physical layer simulation engines.

4.1. OVERVIEW

Signal propagation in GNPpy follows an element-by-element transformation model:

$$SI_{out} = \text{Element}(SI_{in}) \quad (4.1)$$

Where:

- **SI**: SpectralInformation object containing per-channel signal, ASE, NLI, CD, PMD
- **Element**: Network component (Fiber, EDFA, ROADM, Transceiver)
- Each element implements `__call__(spectral_info)` method

Propagation Loop:

Listing 4.1: Core Propagation Pattern

```

1 # From request.py propagate() function
2 spectral_info = create_input_spectral_information(request, equipment)
3
4 for element in path:
5     spectral_info = element(spectral_info) # Transform SI
6
7 # Final element is destination transceiver with GSNR calculation

```

4.2. SPECTRALINFORMATION DATA STRUCTURE

4.2.1 Core Attributes

Location: `gnpy/core/info.py`

Listing 4.2: SpectralInformation Class

```

1 class SpectralInformation:
2     """Container for per-channel optical properties
3
4     All arrays indexed by channel number
5     """
6     def __init__(self, frequency, baud_rate, slot_width,
7                 signal, nli, ase, ...):
8         # Per-channel arrays (W)
9         self.signal = signal # Signal power per channel
10        self.nli = nli # NLI noise per channel
11        self.ase = ase # ASE noise per channel
12        self.tx_power = tx_power # Launch power per channel
13
14        # Frequency grid
15        self.frequency = frequency # Center frequencies (Hz)
16        self.baud_rate = baud_rate # Symbol rates (Bd)
17        self.slot_width = slot_width # Spectrum width (Hz)
18        self.roll_off = roll_off # Nyquist roll-off
19
20        # Accumulated impairments (scalars)
21        self.chromatic_dispersion = 0.0 # ps/nm
22        self.pmd = 0.0 # ps
23        self.pdl = 0.0 # dB
24        self.latency = 0.0 # s
25
26        # Label tracking
27        self.label = label # Service identifiers

```

4.2.2 Key Methods

Listing 4.3: SI Transformation Methods

```

1 def apply_attenuation_db(self, attenuation_db):
2     """Apply dB attenuation to all components"""
3     attenuation_lin = db2lin(-attenuation_db)
4     self.signal *= attenuation_lin
5     self.nli *= attenuation_lin
6     self.ase *= attenuation_lin
7
8 def apply_attenuation_lin(self, attenuation_array):
9     """Apply per-channel linear attenuation"""
10    self.signal *= attenuation_array
11    self.nli *= attenuation_array
12    self.ase *= attenuation_array
13
14 def apply_gain(self, gain_array):
15    """Apply per-channel gain (linear)"""
16    self.signal *= gain_array
17    self.nli *= gain_array
18    self.ase *= gain_array
19
20 def add_ase(self, ase_array):
21    """Add ASE noise (linear power)"""
22    self.ase += ase_array
23
24 def add_nli(self, nli_array):
25    """Add NLI noise (linear power)"""
26    self.nli += nli_array

```

4.2.3 Power Representation

Critical Implementation Detail: All power values stored in linear watts, NOT dBm:

$$\text{SI.signal}[i] = P_{\text{signal},i} \quad [\text{W}] \quad (4.2)$$

Conversions using utility functions:

```

1 from gnpy.core.utils import watt2dbm, dbm2watt
2
3 power_dbm = watt2dbm(spectral_info.signal) # W      dBm
4 power_watt = dbm2watt(power_dbm)          # dBm     W

```

4.3. FIBER PROPAGATION

4.3.1 Fiber Model Overview

Location: gnpy/core/elements.py - Fiber class

Physical Effects Modeled:

1. Attenuation (linear loss)
2. Nonlinear interference (FWM, XPM, SPM)
3. Chromatic dispersion accumulation
4. PMD accumulation
5. Stimulated Raman scattering (optional)

4.3.2 Propagation Algorithm

Listing 4.4: Fiber Propagation Implementation

```

1 def propagate(self, spectral_info: SpectralInformation):
2     """Fiber propagation with all impairments

```

```

3
4     Order matters: NLI computed at fiber input power
5     """
6     # 1. Input connector loss
7     attenuation_in_db = self.params.con_in + self.params.att_in
8     spectral_info.apply_attenuation_db(attenuation_in_db)
9
10    # 2. Stimulated Raman Scattering
11    srs = RamanSolver.calculate_stimulated_raman_scattering(
12        spectral_info, self
13    )
14
15    # 3. Nonlinear Interference (at input power!)
16    spectral_info.nli += NliSolver.compute_nli(
17        spectral_info, srs, self
18    )
19
20    # 4. Chromatic Dispersion accumulation
21    spectral_info.chromatic_dispersion += \
22        self.chromatic_dispersion(spectral_info.frequency)
23
24    # 5. PMD accumulation (RSS)
25    spectral_info.pmd = sqrt(
26        spectral_info.pmd**2 + self.pmd**2
27    )
28
29    # 6. Latency
30    spectral_info.latency += self.params.latency
31
32    # 7. Fiber attenuation (per-channel from Raman)
33    attenuation_fiber = srs.loss_profile[:, -1]
34    spectral_info.apply_attenuation_lin(attenuation_fiber)
35
36    # 8. Output connector loss
37    attenuation_out_db = self.params.con_out
38    spectral_info.apply_attenuation_db(attenuation_out_db)

```

4.3.3 Chromatic Dispersion Model

Listing 4.5: CD Calculation

```

1 def chromatic_dispersion(self, frequency=None):
2     """Accumulated CD [ps/nm]
3
4     Uses Sellmeier equation:
5      $D(\omega) = -(\frac{1}{c}) \frac{d^2 n}{d\omega^2}$ 
6     """
7     freq = self.params.ref_frequency if frequency is None \
8         else frequency
9
10    # and coefficients
11    beta2 = self.beta2(freq)
12    beta3 = self.beta3(freq)
13
14    ref_f = self.params.ref_frequency
15    length = self.params.length
16
17    # Dispersion including slope
18    beta = beta2 + 2*pi*beta3*(freq - ref_f)

```

```

19     dispersion = -beta * 2*pi*ref_f**2 / c
20
21     return dispersion * length # [ps/nm]
22
23 def beta2(self, frequency):
24     """GVD parameter [s /m]"""
25     wavelength = c / frequency
26     D = self.params.dispersion # ps/(nm km) @ 1550nm
27
28     # Convert D to
29     beta2 = -(wavelength**2 * D * 1e-6) / (2*pi*c)
30     return beta2

```

4.3.4 PMD Model

$$\text{PMD}_{\text{total}} = \sqrt{\text{PMD}_{\text{input}}^2 + \text{PMD}_{\text{fiber}}^2} \quad (4.3)$$

```

1 @property
2 def pmd(self):
3     """Differential group delay [s]"""
4     return self.params.pmd_coef * sqrt(self.params.length)
5
6 # In propagate():
7 spectral_info.pmd = sqrt(spectral_info.pmd**2 + self.pmd**2)

```

Note: Root-sum-square (RSS) combination assumes uncorrelated PMD sections.

4.3.5 Nonlinear Interference (NLI)

NLI Models Available

Configuration: `sim_params.json`

Method	Description
<code>gn_model_analytic</code>	Gaussian Noise (GN) model - fast analytical
<code>ggn_spectrally_separated</code>	Enhanced GGN model - accounts for frequency separation
<code>gn_model_numerical</code>	Numerical integration - most accurate, slowest

GN Model Implementation

Location: `gnpy/core/science_utils.py` - `NliSolver`

Equation: Poggiolini GN model (arXiv:1209.0394, eq. 120)

$$P_{\text{NLI},i} = \sum_j \sum_k \eta_{i,j,k} \cdot P_i \cdot P_j \cdot P_k \quad (4.4)$$

Where:

- $P_{\text{NLI},i}$: NLI power on channel i
- $\eta_{i,j,k}$: Nonlinear efficiency coefficient
- P_i, P_j, P_k : Channel powers

Listing 4.6: GN Analytic Implementation

```

1 @staticmethod
2 def _gn_analytic(spectral_info, fiber,
3                 spm_weight=16/27, xpm_weight=32/27):
4     """GN model analytical solution

```

```

5
6 Returns: eta matrix [1/W ]
7 """
8 # Extract parameters
9 frequency = spectral_info.frequency
10 baud_rate = spectral_info.baud_rate
11 beta2 = fiber.beta2(frequency)
12 gamma = fiber.gamma(frequency)
13 alpha = fiber.alpha(frequency)
14 length = fiber.params.length
15
16 # Effective length
17 L_eff = (1 - exp(-alpha*length)) / alpha
18
19 # Asymptotic approximation
20 eta_spm = spm_weight * (gamma * L_eff)**2
21 eta_xpm = xpm_weight * (gamma * L_eff)**2
22
23 # Full eta matrix calculation with dispersion
24 delta_f = outer(ones(N_ch), frequency) - \
25           outer(frequency, ones(N_ch))
26
27 # Dispersion factor
28 rho = pi**2 * abs(beta2) * delta_f**2
29
30 # Phase-matching integral
31 arcsinh_term = arcsinh(rho * L_eff)
32 eta = (gamma**2 / (alpha * rho)) * arcsinh_term
33
34 return eta

```

GGN Model

Enhanced model accounting for spectral separation:

Listing 4.7: GGN Spectrally Separated

```

1 @staticmethod
2 def _ggm_spectrally_separated(cut_indices, spectral_info,
3                               fiber, srs):
4     """GGN model from arXiv:1710.02225
5
6     More accurate for widely-spaced channels
7     """
8     # Compute for subset of channels
9     N_computed = len(cut_indices)
10    eta = zeros((N_computed, N_ch))
11
12    for i, cut_idx in enumerate(cut_indices):
13        f_cut = spectral_info.frequency[cut_idx]
14
15        # For each pump channel
16        for j in range(N_ch):
17            f_pump = spectral_info.frequency[j]
18            delta_f = abs(f_cut - f_pump)
19
20            # Enhanced coherence factor
21            coherence = compute_coherence_factor(
22                delta_f, fiber, spectral_info
23            )

```

```

24         eta[i, j] = base_nli_efficiency * coherence
25
26
27     return eta

```

4.3.6 Raman Scattering

Stimulated Raman Scattering (SRS)

Purpose: Model inter-channel power transfer via Raman effect

Governing Equation:

$$\frac{dP_i}{dz} = -\alpha_i P_i + \sum_j C_R(f_j - f_i) P_i P_j \quad (4.5)$$

Where:

- C_R : Raman gain coefficient [1/(W·m)]
- $f_j - f_i$: Frequency offset

Listing 4.8: Raman Solver Implementation

```

1  @staticmethod
2  def calculate_stimulated_raman_scattering(spectral_info, fiber):
3      """Solve Raman equations along fiber
4
5      Returns: StimulatedRamanScattering object with
6               power_profile[channel, z_position]
7      """
8      if not sim_params.raman_params.flag:
9          # No Raman - simple exponential decay
10         z = array([0, fiber.params.length])
11         frequency = spectral_info.frequency
12         alpha = fiber.alpha(frequency)
13
14         loss_profile = exp(-outer(alpha, z))
15         power_profile = outer(spectral_info.signal,
16                               ones(z.size)) * loss_profile
17
18         return StimulatedRamanScattering(
19             power_profile, loss_profile, frequency, z
20         )
21
22     # Full Raman simulation
23     z_step = sim_params.raman_params.solver_spatial_resolution
24     z = arange(0, fiber.params.length, z_step)
25
26     # Raman coefficient matrix
27     cr = fiber.cr(spectral_info.frequency) # [N_ch  N_ch]
28
29     # Solve ODEs
30     power_profile = solve_raman_odes(
31         spectral_info.signal, # Initial condition
32         z,
33         fiber.alpha(spectral_info.frequency),
34         cr
35     )
36
37     return StimulatedRamanScattering(
38         power_profile,

```

```

39         compute_loss_from_power(power_profile),
40         spectral_info.frequency,
41         z
42     )

```

Spontaneous Raman Scattering

ASE-like noise from Raman amplification:

$$P_{\text{SRS,spont}} = hf \cdot \text{NF}_{\text{Raman}} \cdot G_{\text{Raman}} \cdot B \quad (4.6)$$

```

1  @staticmethod
2  def calculate_spontaneous_raman_scattering(spectral_info, srs, fiber):
3      """Spontaneous Raman ASE noise"""
4      raman_gain = srs.power_profile[:, -1] / spectral_info.signal
5
6      # Raman noise figure (typically 1 dB per 13.2 THz offset)
7      nf_raman = compute_raman_nf(spectral_info.frequency)
8
9      # Spontaneous emission
10     ase_raman = h * spectral_info.frequency * \
11                 (raman_gain - 1) * nf_raman * \
12                 spectral_info.baud_rate
13
14     return ase_raman

```

4.4. EDFA PROPAGATION

4.4.1 EDFA Model Overview

Location: `gnpy/core/elements.py` - `Edfa` class

Primary Effects:

1. Signal amplification (gain)
2. ASE noise addition
3. Gain spectrum shaping
4. Output power limiting

4.4.2 Amplifier Types

Type	Characteristics
<code>variable_gain</code>	Gain varies with input power, fixed NF vs gain curve
<code>fixed_gain</code>	Constant gain regardless of input
<code>openroadm</code>	OpenROADM-compliant with detailed NF model
<code>advanced_model</code>	Multi-stage with VOA control
<code>dual_stage</code>	Explicit pre-amp + booster

4.4.3 Propagation Implementation

Listing 4.9: EDFA Propagation

```

1  def propagate(self, spectral_info: SpectralInformation):
2      """EDFA amplification with ASE addition"""
3
4      # 1. Determine target gain

```

```

5     if self.params.type_def == 'variable_gain':
6         target_gain = self._compute_variable_gain(spectral_info)
7     else:
8         target_gain = self.params.gain_flatmax
9
10    # Apply gain limits
11    actual_gain = self._saturate_gain(target_gain, spectral_info)
12    gain_lin = db2lin(actual_gain)
13
14    # 2. Calculate noise figure
15    nf = self._calc_nf(actual_gain)
16    nf_lin = db2lin(nf)
17
18    # 3. Apply gain to all components
19    spectral_info.signal *= gain_lin
20    spectral_info.nli *= gain_lin
21    spectral_info.ase *= gain_lin
22
23    # 4. Add ASE noise
24    ase_added = self._compute_ase(actual_gain, nf,
25                                spectral_info.baud_rate)
26    spectral_info.ase += ase_added
27
28    # 5. Apply output VOA (if configured)
29    if self.params.out_voa_auto:
30        voa_loss = self._compute_voa()
31        spectral_info.apply_attenuation_db(voa_loss)
32
33    # 6. Dynamic gain tilt (optional)
34    if self.interpol_dgt is not None:
35        tilt = self.interpol_dgt(spectral_info.frequency)
36        spectral_info.apply_gain(db2lin(tilt))

```

4.4.4 ASE Noise Calculation

Fundamental Equation:

$$P_{\text{ASE}} = (G - 1) \cdot \text{NF} \cdot hf \cdot B \quad (4.7)$$

Where:

- G : Linear gain
- NF : Noise figure (linear)
- h : Planck constant
- f : Optical frequency
- B : Noise bandwidth (= baud rate)

Listing 4.10: ASE Computation

```

1 def _compute_ase(self, gain_db, nf_db, baud_rate):
2     """Calculate ASE power per channel
3
4     Returns: array of ASE power [W]
5     """
6     gain_lin = db2lin(gain_db)
7     nf_lin = db2lin(nf_db)
8
9     # h f per channel
10    photon_energy = h * self.frequency # [J]
11

```

```

12 # ASE power
13 ase_power = (gain_lin - 1) * nf_lin * \
14             photon_energy * baud_rate
15
16 return ase_power # [W]

```

4.4.5 Noise Figure Models

Polynomial NF Model

$$\text{NF}(G) = \text{NF}_{\min} + \sum_i a_i \cdot G^i \quad (4.8)$$

```

1 def _calc_nf(self, gain_db):
2     """Calculate NF from gain using polynomial fit"""
3     if hasattr(self.params, 'nf_model'):
4         # Polynomial coefficients
5         coeffs = self.params.nf_model.nf_coef
6         nf = sum(c * gain_db**i
7                 for i, c in enumerate(coeffs))
8     else:
9         # Simple linear interpolation
10        nf = interp(gain_db,
11                  [self.params.gain_min,
12                   self.params.gain_flatmax],
13                  [self.params.nf_max,
14                   self.params.nf_min])
15
16    return nf # [dB]

```

OpenROADM NF Model

$$\text{NF} = \text{NF}_0 + \frac{P_{\text{out}} - P_{\text{out,ref}}}{C} \quad (4.9)$$

Where C is a temperature/degradation coefficient.

4.4.6 Gain Saturation

Listing 4.11: Gain Saturation Model

```

1 def _saturate_gain(self, target_gain, spectral_info):
2     """Enforce gain limits based on output power"""
3
4     # Total input power
5     p_in = sum(spectral_info.signal + spectral_info.ase +
6               spectral_info.nli)
7
8     # Desired output power
9     p_out_target = p_in * db2lin(target_gain)
10
11    # Power limit
12    p_max = dbm2watt(self.params.p_max)
13
14    if p_out_target > p_max:
15        # Reduce gain to meet power limit
16        actual_gain = watt2dbm(p_max) - watt2dbm(p_in)
17        return actual_gain
18
19    # Check minimum gain

```

```

20     if target_gain < self.params.gain_min:
21         return self.params.gain_min
22
23     return target_gain

```

4.5. ROADM PROPAGATION

4.5.1 ROADM Model Overview

Location: gnpy/core/elements.py - Roadm class

Functions:

1. Wavelength switching
2. Channel equalization
3. Add/drop loss
4. Per-degree OSNR degradation

4.5.2 Propagation Implementation

Listing 4.12: ROADM Propagation

```

1 def propagate(self, spectral_info: SpectralInformation,
2               degree=None, from_degree=None):
3     """ROADM passage with equalization and loss"""
4
5     # 1. Determine loss based on path
6     if degree in self.params.add_degrees:
7         # Adding new channels
8         loss_db = self.params.add_drop_osnr
9         osnr_penalty = 0 # No ASE yet
10    elif degree in self.params.drop_degrees:
11        # Dropping channels
12        loss_db = self.params.add_drop_osnr
13        osnr_penalty = 0 # Channels terminated
14    else:
15        # Express path
16        loss_db = self.params.per_degree_loss
17        osnr_penalty = self.params.per_degree_impairments.get(
18            'pmd', 0
19        )
20
21    # 2. Get target power for equalization
22    target_power = self.get_roadm_target_power(degree)
23
24    # 3. Per-channel equalization
25    current_power = spectral_info.signal
26
27    for i, (p_current, p_target) in enumerate(
28        zip(current_power, target_power)
29    ):
30        if p_target > 0:
31            # Calculate required attenuation/gain
32            delta_db = watt2dbm(p_target) - watt2dbm(p_current)
33
34            # Apply with limits
35            delta_db = clip(delta_db, -20, 0) # Max 20 dB atten
36
37            equalization = db2lin(delta_db)
38            spectral_info.signal[i] *= equalization
39            spectral_info.nli[i] *= equalization

```

```

40         spectral_info.ase[i] *= equalization
41
42     # 4. Apply baseline loss
43     spectral_info.apply_attenuation_db(loss_db)
44
45     # 5. Add ROADM ASE (from loss + add/drop OSNR)
46     if self.params.add_drop_osnr is not None:
47         ase_roadm = self._compute_roadm_ase(spectral_info)
48         spectral_info.ase += ase_roadm
49
50     # 6. PMD contribution
51     spectral_info.pmd = sqrt(
52         spectral_info.pmd**2 +
53         self.params.per_degree_impairments['pmd']**2
54     )

```

4.5.3 Channel Equalization Strategy

Listing 4.13: ROADM Equalization

```

1 def get_roadm_target_power(self, degree):
2     """Determine per-channel target power
3
4     Strategies:
5     - FLAT: All channels same power
6     - TILT: Linear power variation across band
7     - RIPPLE: Follow gain ripple compensation
8     """
9     N_ch = len(self.frequency)
10
11     if self.equalization_type == 'FLAT':
12         target = ones(N_ch) * self.target_pch_out_dbm
13
14     elif self.equalization_type == 'TILT':
15         # Linear tilt: higher power at band edges
16         freq_normalized = (self.frequency -
17                             self.frequency[0]) / \
18                             (self.frequency[-1] -
19                             self.frequency[0])
20
21         tilt = self.tilt_target * (freq_normalized - 0.5)
22         target = self.target_pch_out_dbm + tilt
23
24     elif self.equalization_type == 'RIPPLE':
25         # Compensate amplifier gain ripple
26         target = self.target_pch_out_dbm - \
27                 self.upstream_gain_ripple
28
29     return dbm2watt(target)

```

4.5.4 Add/Drop OSNR Model

$$\frac{1}{\text{OSNR}_{\text{out}}} = \frac{1}{\text{OSNR}_{\text{in}}} + \frac{1}{\text{OSNR}_{\text{ROADM}}} \quad (4.10)$$

```

1 def _compute_roadm_ase(self, spectral_info):
2     """ASE from ROADM loss and add/drop OSNR"""
3

```

```

4      # ROADM introduces noise equivalent to:
5      # P_signal / OSNR_ROADM
6
7      osnr_roadm_lin = db2lin(self.params.add_drop_osnr)
8
9      # ASE per channel
10     ase_roadm = spectral_info.signal / osnr_roadm_lin
11
12     return ase_roadm

```

4.6. TRANSCEIVER (RECEIVER) COMPUTATION

4.6.1 Transceiver Model Overview

Location: gnpy/core/elements.py - Transceiver class

Key Functions:

1. OSNR/GSNR calculation
2. Penalty application (CD, PMD, PDL)
3. Feasibility check

4.6.2 SNR Calculation

Listing 4.14: Transceiver SNR Computation

```

1  def _calc_snr(self, spectral_info):
2      """Calculate all SNR metrics
3
4      Computed values:
5      - OSNR_ASE: Signal / ASE
6      - OSNR_NLI: Signal / NLI
7      - GSNR: Signal / (ASE + NLI)
8      """
9      self.propagated_labels = spectral_info.label
10     self.baud_rate = spectral_info.baud_rate
11
12     # Conversion factor for 0.1 nm reference bandwidth
13     ratio_01nm = lin2db(12.5e9 / self.baud_rate)
14
15     # Raw values (no Tx OSNR penalty yet)
16     with errstate(divide='ignore'):
17         # OSNR ASE
18         self.raw_osnr_ase = lin2db(
19             spectral_info.signal / spectral_info.ase
20         )
21         self.raw_osnr_ase_01nm = self.raw_osnr_ase - ratio_01nm
22
23         # OSNR NLI
24         self.raw_osnr_nli = lin2db(
25             spectral_info.signal / spectral_info.nli
26         )
27
28         # GSNR (total)
29         self.raw_snr = lin2db(
30             spectral_info.signal /
31             (spectral_info.ase + spectral_info.nli)
32         )
33         self.raw_snr_01nm = self.raw_snr - ratio_01nm
34
35     # Copy to working values (modified by update_snr)

```

```

36 self.osnr_ase = self.raw_osnr_ase
37 self.osnr_ase_01nm = self.raw_osnr_ase_01nm
38 self.osnr_nli = self.raw_osnr_nli
39 self.snr = self.raw_snr
40 self.snr_01nm = self.raw_snr_01nm

```

4.6.3 Penalty Calculation

Chromatic Dispersion Penalty

Listing 4.15: CD Penalty

```

1 def _calc_cd_penalty(self, mode):
2     """CD penalty from accumulated dispersion
3
4     Uses lookup table from transponder mode definition
5     """
6     penalties = mode.get('penalties', {})
7     cd_boundaries = penalties.get('chromatic_dispersion')
8
9     if cd_boundaries is None:
10         self.penalties['chromatic_dispersion'] = zeros(N_ch)
11         return
12
13     # CD accumulated along path
14     cd_value = self.chromatic_dispersion # ps/nm
15
16     # Interpolate penalty
17     penalty = interp(
18         abs(cd_value),
19         cd_boundaries['up_to_boundary'],
20         cd_boundaries['penalty_value'],
21         left=inf, # Outside range inf penalty
22         right=inf
23     )
24
25     self.penalties['chromatic_dispersion'] = penalty # dB

```

Example Penalty Table:

```

1 "penalties": {
2     "chromatic_dispersion": {
3         "up_to_boundary": [0, 10000, 20000, 30000], # ps/nm
4         "penalty_value": [0, 0.5, 2.0, 10.0] # dB
5     }
6 }

```

PMD Penalty

$$\text{Penalty}_{\text{PMD}} = f(\text{PMD} \cdot B_s) \quad (4.11)$$

Where B_s is the symbol rate.

```

1 def _calc_pmd_penalty(self, mode):
2     """PMD penalty from accumulated PMD"""
3     pmd_boundaries = mode['penalties'].get('pmd')
4
5     if pmd_boundaries is None:
6         return zeros(N_ch)
7
8     # Normalized PMD

```

```

9     pmd_normalized = self.pmd * self.baud_rate # [s] [Bd]
10
11     penalty = interp(
12         pmd_normalized,
13         pmd_boundaries['up_to_boundary'],
14         pmd_boundaries['penalty_value'],
15         left=inf, right=inf
16     )
17
18     return penalty # dB

```

PDL Penalty

PDL (Polarization Dependent Loss) modeled as statistical penalty:

```

1 def _calc_pdl_penalty(self, mode):
2     """PDL penalty - direct from accumulated PDL"""
3     pdl_boundaries = mode['penalties'].get('pdl')
4
5     if pdl_boundaries is None:
6         return zeros(N_ch)
7
8     # PDL accumulated (dB)
9     pdl_value = self.pdl
10
11     penalty = interp(
12         pdl_value,
13         pdl_boundaries['up_to_boundary'],
14         pdl_boundaries['penalty_value'],
15         left=inf, right=inf
16     )
17
18     return penalty # dB

```

4.6.4 Effective GSNR with Penalties

$$\text{GSNR}_{\text{eff}} = \text{GSNR}_{\text{raw}} - \text{Penalty}_{\text{CD}} - \text{Penalty}_{\text{PMD}} - \text{Penalty}_{\text{PDL}} \quad (4.12)$$

Listing 4.16: Total Penalty Application

```

1 def calc_penalties(self, mode):
2     """Calculate all penalties and apply to GSNR"""
3
4     # Individual penalties
5     self._calc_cd_penalty(mode)
6     self._calc_pmd_penalty(mode)
7     self._calc_pdl_penalty(mode)
8
9     # Total penalty (dB)
10    self.total_penalty = (
11        self.penalties['chromatic_dispersion'] +
12        self.penalties['pmd'] +
13        self.penalties['pdl']
14    )
15
16    # Effective GSNR
17    self.gsnr_effective = self.snr_01nm - self.total_penalty
18
19 def __call__(self, spectral_info):
20     """Complete receiver processing"""

```

```

21     self.tx_power = spectral_info.tx_power
22
23     # Calculate raw SNR values
24     self._calc_snr(spectral_info)
25
26     # Store impairments for penalty calculation
27     self._calc_cd(spectral_info)
28     self._calc_pmd(spectral_info)
29     self._calc_pdl(spectral_info)
30     self._calc_latency(spectral_info)
31
32     return spectral_info

```

4.6.5 Transmitter OSNR

Purpose: Account for transmitter imperfections

Listing 4.17: Tx OSNR Update

```

1 def update_snr(self, *args):
2     """Add Tx OSNR and other system penalties
3
4     Called from request.py after propagation
5     Only applied to destination transceiver
6     """
7     # Collect all OSNR penalties (in linear domain)
8     snr_added_lin = 0
9     for osnr_penalty in args:
10         if osnr_penalty is not None:
11             snr_added_lin += db2lin(-osnr_penalty)
12
13     snr_added_db = -lin2db(snr_added_lin)
14
15     # Update SNR values
16     self.osnr_ase = snr_sum(self.raw_osnr_ase,
17                             self.baud_rate,
18                             snr_added_db)
19
20     self.snr = snr_sum(self.raw_snr,
21                        self.baud_rate,
22                        snr_added_db)
23
24     self.osnr_ase_01nm = snr_sum(self.raw_osnr_ase_01nm,
25                                   12.5e9,
26                                   snr_added_db)
27
28     self.snr_01nm = snr_sum(self.raw_snr_01nm,
29                              12.5e9,
30                              snr_added_db)
31
32     # Usage in request.py:
33     destination_trx.update_snr(
34         mode['tx_osnr'], # Transmitter OSNR
35         equipment['SI']['default'].sys_margins # System margins
36     )

```

4.7. COMPLETE PROPAGATION EXAMPLE

4.7.1 End-to-End Signal Flow

Listing 4.18: Complete Propagation Trace

```

1  # Initial state at transmitter
2  si = create_input_spectral_information(request, equipment)
3  # si.signal = [1e-3, 1e-3, ...] W (0 dBm per channel)
4  # si.ase = [0, 0, ...] W
5  # si.nli = [0, 0, ...] W
6
7  # Element 1: Source Transceiver
8  source_trx(si)
9  # No changes (transmitter just launches)
10
11 # Element 2: Fiber (80 km SSMF)
12 fiber_1(si)
13 # Signal attenuated: si.signal *= exp(-0.2 dB/km      80 km)
14 #                      equivalent to 0.0158 of original
15 # NLI added: si.nli += [1e-8, 1e-8, ...] W
16 # CD accumulated: si.chromatic_dispersion += 1360 ps/nm
17 # PMD accumulated: si.pmd = sqrt(0      + (0.1 ps/sqrt(km)      sqrt(80))      )
18 #                      equivalent to 0.89 ps
19
20 # Element 3: EDFA
21 edfa_1(si)
22 # Gain applied: si.signal *= 10^(16/10) equivalent to 40
23 # NLI amplified: si.nli *= 40
24 # ASE added: si.ase += (40-1)      5      h f      32e9
25 #                      equivalent to [3.1e-7, ...] W
26
27 # Element 4: Fiber (80 km)
28 fiber_2(si)
29 # Repeat fiber propagation...
30
31 # Element 5: ROADM
32 roadm(si)
33 # Loss applied: 17 dB
34 # Channels equalized to target power
35 # Additional ASE from ROADM loss
36
37 # ... more fibers, EDFAs, ROADMs ...
38
39 # Final element: Destination Transceiver
40 destination_trx(si)
41 # Calculate GSNR:
42 # GSNR = si.signal / (si.ase + si.nli)
43 #       = 1e-3 W / (5e-6 + 2e-7) W
44 #       = 192 (linear) = 22.8 dB
45 #
46 # Apply penalties:
47 # CD penalty at 1360 ps/nm: 0.5 dB
48 # PMD penalty at 7 ps: 0.2 dB
49 # Total penalty: 0.7 dB
50 #
51 # Effective GSNR = 22.8 - 0.7 = 22.1 dB
52 #
53 # Check feasibility:
54 # Required OSNR for DP-16QAM: 23 dB
55 # Result: FAIL (margin = -0.9 dB)

```

4.7.2 Power Budget Example

Element	Signal	ASE	NLI	OSNR	GSNR
Tx output	0 dBm	-	-	∞	∞
After fiber 1	-16 dBm	0	-80 dBm	∞	64 dB
After EDFA 1	0 dBm	-23 dBm	-64 dBm	23 dB	23.5 dB
After fiber 2	-16 dBm	-23 dBm	-67 dBm	7 dB	6.7 dB
After EDFA 2	0 dBm	-20 dBm	-51 dBm	20 dB	19.8 dB
After ROADM	-17 dBm	-29 dBm	-51 dBm	12 dB	11.5 dB
...	...	...	...	...	...
At Rx	-3 dBm	-20 dBm	-25 dBm	17 dB	15 dB
With penalties (CD: 0.5 dB, PMD: 0.2 dB): Effective					14.3 dB

Table 4.1: Power Evolution Through Network

4.8. PERFORMANCE CONSIDERATIONS

4.8.1 Computational Bottlenecks

Operation	Complexity	% of Runtime
NLI computation (GN)	$O(N_{ch}^2)$	60-70%
Raman solving	$O(N_z \cdot N_{ch}^2)$	20-30% (if enabled)
Element propagation	$O(N_{elem} \cdot N_{ch})$	10-20%
Path computation	$O(E + V \log V)$	<5%

4.8.2 Accuracy vs. Speed Trade-offs

Configuration	NLI Method	Raman	Speed	Accuracy
Fast	gn_analytic	Off	1×	± 1 dB
Standard	ggn_separated	Off	3×	± 0.5 dB
Accurate	ggn_separated	On (1st order)	10×	± 0.3 dB
Research	gn_numerical	On (4th order)	100×	± 0.1 dB

4.8.3 Optimization Strategies

1. **Sparse NLI:** Compute only for representative channels, interpolate

```

1 # In sim_params.json
2 "nli_params": {
3     "method": "ggn_spectrally_separated",
4     "computed_channels": [0, 24, 47, 71, 95] # 5 of 96 channels
5 }

```

2. **Vectorization:** Use NumPy array operations

```

1 # Good: Vectorized
2 signal *= gain_array
3
4 # Bad: Element-wise loop
5 for i in range(len(signal)):
6     signal[i] *= gain_array[i]

```

3. **Caching:** Pre-compute frequency-dependent parameters

```

1 class Fiber:
2     def __init__(self, params):
3         self.params = params
4         # Pre-compute for standard grid
5         self._alpha_cache = {}
6         self._gamma_cache = {}
7
8     def alpha(self, frequency):
9         key = tuple(frequency)
10        if key not in self._alpha_cache:
11            self._alpha_cache[key] = self._compute_alpha(frequency)
12        return self._alpha_cache[key]

```

4. **Parallel Processing:** Independent paths can be computed in parallel

```

1 from multiprocessing import Pool
2
3 def propagate_path(path, request, equipment):
4     # Propagate single path
5     return result
6
7 # Process multiple requests in parallel
8 with Pool(processes=8) as pool:
9     results = pool.starmap(propagate_path,
10                           [(path, req, equipment)
11                            for path, req in zip(paths, requests)])

```

4.9. VALIDATION AND TESTING

4.9.1 Unit Test Strategy

Listing 4.19: Element Testing

```

1 def test_fiber_attenuation():
2     """Verify fiber loss calculation"""
3     fiber = Fiber(length=80e3, loss_coef=0.2e-3)
4
5     si = SpectralInformation(
6         signal=ones(1) * 1e-3, # 1 mW
7         ase=zeros(1),
8         nli=zeros(1),
9         ...
10    )
11
12    fiber(si)
13
14    expected_loss = 0.2 * 80 # 16 dB
15    expected_power = 1e-3 * db2lin(-16)
16
17    assert_allclose(si.signal[0], expected_power, rtol=0.01)
18
19 def test_edfa_ase():
20     """Verify ASE calculation"""
21     edfa = Edfa(gain=20, nf=5)
22
23     si = SpectralInformation(
24         signal=ones(1) * dbm2watt(-16),
25         ase=zeros(1),
26         baud_rate=ones(1) * 32e9,
27         frequency=array([193.1e12]),

```

```

28     ...
29 )
30
31 edfa(si)
32
33 # Check ASE
34 expected_ase = (db2lin(20) - 1) * db2lin(5) * \
35               h * 193.1e12 * 32e9
36
37 assert_allclose(si.ase[0], expected_ase, rtol=0.05)

```

4.9.2 End-to-End Validation

Listing 4.20: System-Level Tests

```

1 def test_link_osnr():
2     """Validate OSNR for known link"""
3
4     # Build simple link: Tx      80km fiber      EDFA      80km fiber
5     Rx
6     network = build_test_network()
7
8     # Known parameters
9     launch_power = 0 # dBm
10    fiber_loss = 16 # dB per span
11    edfa_gain = 16 # dB
12    edfa_nf = 5 # dB
13
14    # Analytical OSNR
15    osnr_analytic = launch_power + edfa_gain - fiber_loss + \
16                  10*log10(12.5e9 / 32e9) - edfa_nf
17
18    # Simulate
19    si = create_si(launch_power)
20    for element in network:
21        element(si)
22
23    # Compare
24    osnr_sim = watt2dbm(si.signal[0]) - watt2dbm(si.ase[0]) + \
25            10*log10(12.5e9 / 32e9)
26
27    assert abs(osnr_sim - osnr_analytic) < 0.5 # Within 0.5 dB

```

4.10. SUMMARY

4.10.1 Key Takeaways

1. **Element-by-Element:** Propagation is sequential transform of `SpectralInformation`
2. **Linear Power Domain:** All calculations in watts, converted to dB for I/O
3. **Order Matters:** NLI computed at fiber input, penalties applied at receiver
4. **Vectorized Operations:** Per-channel arrays enable efficient computation
5. **Physical Fidelity:** Models capture key effects (ASE, NLI, CD, PMD) accurately

4.10.2 Implementation Checklist

For developers implementing similar systems:

1. **Data Structure:** Design efficient per-channel representation
2. **NLI Model:** Start with GN analytic, add GGN for accuracy

3. **ASE Calculation:** Implement $P_{\text{ASE}} = (G - 1) \cdot \text{NF} \cdot hf \cdot B$
4. **Penalty Models:** Use lookup tables with interpolation
5. **Unit Consistency:** Maintain consistent units (linear power internally)
6. **Validation:** Test against analytical formulas for simple cases
7. **Performance:** Vectorize operations, cache frequency-dependent values

4.10.3 Extensions

Advanced features beyond current GNPpy implementation:

1. **Transient Effects:** Model power transients during add/drop
2. **Multi-Band:** Support S+C+L simultaneous transmission
3. **Spatial Multiplexing:** Multi-core/multi-mode fiber
4. **Probabilistic Models:** Statistical distribution of impairments
5. **Machine Learning:** Neural networks for fast NLI approximation

Chapter 5

Output Analysis & Results Interpretation

CHAPTER OVERVIEW

This chapter connects GNPY's theoretical models (Chapters 1-4) to actionable research insights. It explains what GNPY outputs reveal about optical network physics, how to extract different levels of analysis (system/path/element/channel), and what questions the tool can answer. Understanding output interpretation is essential for network research, comparative studies, design validation, and advancing optical transmission theory.

5.1. OVERVIEW: GNPY’S ANALYTICAL CAPABILITIES

5.1.1 GNPY coverage

GNPy provides physics-based predictions for optical network performance. Understanding what questions the tool can answer guides effective output analysis:

Question Type	Example	Output Level
Service feasibility	Can 400G reach 800 km with DP-16QAM?	System
Path comparison	Which route has better GSNR?	Path
Impairment diagnosis	Where does performance degrade?	Element
Spectral effects	How does edge channel GSNR compare to center?	Channel
Design optimization	What launch power maximizes GSNR?	Parametric
Model validation	Does GN model match measurements?	Analytical

5.1.2 Output Hierarchy

GNPy provides four analysis levels, each accessed through different commands and output formats:

1. **System-Level** (all services): CSV from `gnpy-path-request -o results.csv`
2. **Path-Level** (route metrics): JSON from `gnpy-path-request -o response.json`
3. **Element-Level** (per-node diagnostics): Console from `gnpy-transmission-example`
4. **Channel-Level** (spectral analysis): Console from `gnpy-transmission-example -show-channels`

- Selection guidance:
- **Initial assessment:** System-level CSV to identify failing services
 - **Route optimization:** Path-level JSON for alternative comparison
 - **Troubleshooting:** Element-level console to locate impairment sources
 - **Spectral research:** Channel-level output for NLI/ASE per-channel analysis
 - **Bidirectional:** Add `-bi` flag to compute both directions
- Important:** `gnpy-path-request` requires `-o filename` to save results. Without this flag, only console summary is displayed.

5.1.3 Command Reference

Command	Output	Use Case
<code>gnpy-path-request -o file.json</code>	JSON to specified file	API integration, detailed metrics
<code>gnpy-path-request -o results.csv</code>	CSV table	Multi-service batch analysis
<code>gnpy-path-request -bi</code>	Bidirectional analysis	Both A→B and B→A computed
<code>gnpy-transmission-example</code>	Console per-element	Single path diagnosis
<code>-show-channels</code>	Console per-channel table	Spectral OSNR/NLI analysis
<code>-plot</code>	Matplotlib figure	Visual path/power profile
<code>-save-network design.json</code>	Network JSON	Export auto-designed topology
<code>-no-insert-edfas</code>	(modifier)	Manual amplifier placement
<code>-redesign-per-request</code>	(modifier)	Per-service network design
<code>-list-nodes</code>	Console list	Show all transceivers

Table 5.1: GNPY command-line options for output control

Note: Without `-o` flag, `gnpy-path-request` displays console summary only (no file saved).

5.2. CORE METRICS: PHYSICAL INTERPRETATION

5.2.1 Metric Definitions and Physical Meaning

GNPy outputs multiple SNR variants, each revealing different aspects of optical impairments.

Metric	Physical Interpretation
OSNR-0.1nm	ASE-limited regime indicator. Reflects amplifier noise only.
OSNR-bandwidth	ASE noise in signal bandwidth. Decreases with wider channels.
SNR-0.1nm	GSNR in 0.1 nm reference. Combined ASE + NLI effect.
SNR-bandwidth	GSNR in signal bandwidth. Primary feasibility metric.
OSNR _{ASE}	From Chapter 1: $10 \log_{10}(P_{\text{signal}}/P_{\text{ASE}})$
OSNR _{NLI}	From Chapter 1: $10 \log_{10}(P_{\text{signal}}/P_{\text{NLI}})$

GNPy Output Metric Definitions:

- **OSNR-0.1nm:** ASE-limited OSNR in standard 12.5 GHz (0.1 nm) reference bandwidth. Formula: $10 \log_{10}(P_{\text{signal}}/P_{\text{ASE},0.1\text{nm}})$. Reflects amplifier cascade noise only.
- **OSNR-bandwidth:** ASE noise in actual signal bandwidth (e.g., 32 GHz for 32 Gbaud). Formula: $10 \log_{10}(P_{\text{signal}}/P_{\text{ASE},\text{sig}})$. Lower than OSNR-0.1nm due to wider noise integration.
- **SNR-0.1nm:** Generalized SNR (GSNR) including ASE and NLI in 0.1 nm reference. Formula: $10 \log_{10}(P_{\text{signal}}/(P_{\text{ASE}} + P_{\text{NLI}})_{0.1\text{nm}})$.
- **SNR-bandwidth:** Total noise (ASE+NLI) in signal bandwidth. Formula: $10 \log_{10}(P_{\text{signal}}/(P_{\text{ASE}} + P_{\text{NLI}})_{\text{sig}})$. Primary feasibility metric relating to BER.
- **OSNR_{ASE}:** Component metric isolating ASE contribution. Equivalent to OSNR-bandwidth.
- **OSNR_{NLI}:** Component metric isolating NLI contribution. Formula: $10 \log_{10}(P_{\text{signal}}/P_{\text{NLI}})$.
- **Key relationship:** $\frac{1}{\text{SNR}} = \frac{1}{\text{OSNR}_{\text{ASE}}} + \frac{1}{\text{OSNR}_{\text{NLI}}}$ (linear units)

Bandwidth Conversion

All SNR metrics are bandwidth-dependent. GNPy reports both 0.1 nm reference and signal bandwidth:

$$\text{SNR}_{\text{bw}} = \text{SNR}_{0.1\text{nm}} - 10 \log_{10} \left(\frac{B_{\text{signal}}}{12.5 \text{ GHz}} \right) \quad (5.1)$$

Example (32 GBd signal with 1.2 roll-off):

- Signal bandwidth: $32 \times 1.2 = 38.4 \text{ GHz}$
- Conversion: $\Delta_{\text{BW}} = 10 \log_{10}(38.4/12.5) = 4.9 \text{ dB}$
- If $\text{SNR}_{0.1\text{nm}} = 20.0 \text{ dB} \rightarrow \text{SNR}_{\text{bw}} = 15.1 \text{ dB}$

5.2.2 Regime Identification

The relationship between OSNR_{ASE} and OSNR_{NLI} reveals dominant impairment:

Regime	Condition	Limiting Factor
ASE-dominated	$\text{OSNR}_{\text{ASE}} \ll \text{OSNR}_{\text{NLI}}$	Amplifier noise
NLI-dominated	$\text{OSNR}_{\text{NLI}} \ll \text{OSNR}_{\text{ASE}}$	Fiber nonlinearity
Balanced	$ \text{OSNR}_{\text{ASE}} - \text{OSNR}_{\text{NLI}} < 3 \text{ dB}$	Mixed regime

GSNR approximation (from Chapter 1, Section 1.4):

$$\frac{1}{\text{GSNR}} = \frac{1}{\text{OSNR}_{\text{ASE}}} + \frac{1}{\text{OSNR}_{\text{NLI}}} \quad (5.2)$$

Research implication: If OSNR_{NLI} = 15 dB and OSNR_{ASE} = 25 dB, then:

- GSNR $\approx 14.4 \text{ dB}$ (NLI-dominated)
- Improving amplifiers (higher OSNR_{ASE}) yields minimal gain
- Reducing launch power or span count more effective

5.2.3 Decision Thresholds

GSNR Value	Interpretation
$\text{GSNR} > \text{OSNR}_{\text{req}} + 3 \text{ dB}$	Robust operation, margin for aging
$\text{GSNR} \approx \text{OSNR}_{\text{req}} + 1\text{-}2 \text{ dB}$	Marginal, sensitive to variations
$\text{GSNR} < \text{OSNR}_{\text{req}}$	Infeasible, BER exceeds threshold

From Chapter 1: OSNR_{req} depends on modulation format. For DP-QPSK at $\text{BER} = 10^{-3}$, $\text{OSNR}_{\text{req}} \approx 9.5 \text{ dB}$ (theoretical), but transponder datasheets specify penalties (CD, PMD, implementation) adding 2-4 dB.

5.3. OUTPUT FORMATS & ACCESS METHODS

5.3.1 JSON Response Structure

Command: `gnpy-path-request topology.json services.json -e eqpt.json`

Output file: `response.json` (IETF TEAS YANG Path Computation schema)

Research use: API integration, automated analysis, programmatic result extraction

Top-Level Structure

Listing 5.1: Response JSON Structure

```
{
  "response": [
    {
      "response-id": "request-001",
      // Either path-properties OR no-path
      "path-properties": { ... }, // Feasible path
      // OR
      "no-path": { ... }           // Blocked path
    },
    ...
  ]
}
```

5.3.2 Successful Path Response

Listing 5.2: Feasible Path Example

```
{
  "response-id": "Paris-to-Marseille",
  "path-properties": {
    "path-metric": [
      {
        "metric-type": "SNR-0.1nm",
        "accumulative-value": 18.5
      },
      {
        "metric-type": "SNR-bandwidth",
        "accumulative-value": 17.2
      },
      {
        "metric-type": "OSNR-0.1nm",
        "accumulative-value": 20.3
      },
      {

```

```

    "metric-type": "OSNR-bandwidth",
    "accumulative-value": 19.0
  },
  {
    "metric-type": "reference_power",
    "accumulative-value": -3.0
  },
  {
    "metric-type": "path_bandwidth",
    "accumulative-value": 200000000000
  }
],
"z-a-path-metric": [
  // Reverse direction metrics (if bidirectional)
],
"path-route-objects": [
  // Detailed path elements
]
}
}

```

Key Metrics Explained

Metric	Definition
SNR-0.1nm	GSNR in 0.1 nm reference bandwidth (dB)
SNR-bandwidth	GSNR in signal bandwidth (dB)
OSNR-0.1nm	OSNR (ASE only) in 0.1 nm bandwidth (dB)
OSNR-bandwidth	OSNR (ASE only) in signal bandwidth (dB)
reference_power	Signal power at destination (dBm)
path_bandwidth	Total requested bandwidth (Hz)

Conversion:

$$\text{SNR}_{\text{bw}} = \text{SNR}_{0.1\text{nm}} - 10 \log_{10} \left(\frac{B_{\text{signal}}}{12.5 \text{ GHz}} \right) \quad (5.3)$$

Example:

- Signal bandwidth: 32 GBd $\hat{=}$ 50 GHz (with roll-off)
- $\text{SNR}_{0.1\text{nm}} = 18.5 \text{ dB}$
- $\text{SNR}_{\text{bw}} = 18.5 - 10 \log_{10}(50/12.5) = 18.5 - 6.0 = 12.5 \text{ dB}$

5.3.3 Path Route Objects

Listing 5.3: Path Route Objects

```

"path-route-objects": [
  {
    "path-route-object": {
      "index": 0,
      "num-unnum-hop": {
        "node-id": "Paris",
        "link-tp-id": "Paris"
      },
      "transponder": {
        "transponder-type": "Voyager",
        "transponder-mode": "DP-QPSK 200G"
      }
    }
  }
]

```

```

    }
  },
  {
    "path-route-object": {
      "index": 1,
      "num-unnum-hop": {
        "node-id": "fiber_Paris_to_Lyon",
        "link-tp-id": "fiber_Paris_to_Lyon"
      }
    }
  },
  {
    "path-route-object": {
      "index": 2,
      "num-unnum-hop": {
        "node-id": "EDFA_Lyon_in",
        "link-tp-id": "EDFA_Lyon_in"
      }
    }
  },
  {
    "path-route-object": {
      "index": 3,
      "label-hop": [
        {
          "N": 0,
          "M": 4
        }
      ]
    }
  }
]

```

Key Elements:

- num-unnum-hop: Network element identifier
- transponder: Mode selection at endpoints
- label-hop: Spectrum assignment (N, M)

5.3.4 Blocked Path Response

Listing 5.4: Blocked Path Example

```

{
  "response-id": "blocked-request-002",
  "no-path": {
    "no-path": "NO_FEASIBLE_MODE",
    "path-properties": {
      // Partial metrics from last attempted mode
      "path-metric": [
        {
          "metric-type": "SNR-0.1nm",
          "accumulative-value": 14.2
        },
        {
          "metric-type": "OSNR-0.1nm",
          "accumulative-value": 16.5
        }
      ],
      "path-route-objects": [

```

```

        // Path that was attempted
    ]
}
}
}

```

Blocking Reason Interpretation

Reason	Physical Cause	Research Action
NO_PATH	Topology disconnection	Validate network graph
NO_PATH_WITH_CONSTRAINT	Disjoint/include violated	Relax constraints
NOT_ENOUGH_SPECTRUM	Spectrum exhaustion	Study blocking probability
NO_FEASIBLE_MODE	$\text{GSNR} < \text{OSNR}_{\text{req}}$ all modes	Analyze impairment budget
NO_FEASIBLE_BAUDRATE	Spacing too narrow	Study spectral efficiency
MODE_NOT_FEASIBLE	User mode fails GSNR	Compare to auto-select

NO_FEASIBLE_MODE research interpretation:

This blocking reason indicates all transponder modes from the equipment library fail GSNR requirements. From Chapter 3, mode selection iterates from highest to lowest spectral efficiency:

1. DP-64QAM (600G): $\text{OSNR}_{\text{req}} \approx 28$ dB
2. DP-16QAM (400G): $\text{OSNR}_{\text{req}} \approx 23$ dB
3. DP-8QAM (300G): $\text{OSNR}_{\text{req}} \approx 19$ dB
4. DP-QPSK (200G): $\text{OSNR}_{\text{req}} \approx 15$ dB

If even DP-QPSK fails, the path has fundamental impairment issues. Diagnostic steps:

Check element-level diagnostics

```

gnpy-transmission-example topology.json \
  --source NodeA --destination NodeB

```

Verify GSNR at destination

If $\text{GSNR} < 15$ dB, analyze:

- # – Total span count (Chapter 1: NF_{eff} increases with cascaded EDFAs)*
- # – Launch power (too high NLI dominance)*
- # – Fiber type (SSMF vs NZDSF affects nonlinearity)*

Research questions enabled:

- What is maximum transparent reach for each modulation format?
- How does blocking rate scale with network size?
- Which paths are NLI-limited vs ASE-limited?

5.4. CSV OUTPUT FORMAT

5.4.1 Generation

Command:

```

gnpy-path-request topology.json services.json \
  -e eqpt_config.json \
  -o results.csv

```

Research use: Multi-service batch analysis, statistical studies, comparative benchmarking

5.4.2 Console Output: Element-Level Diagnostics

Command:

```
gnpy-transmission-example topology.json \
  -e eqpt_config.json \
  --source Paris --destination Marseille
```

Research use: Impairment localization, model verification, per-element debugging

Element-by-Element Printout

Console output shows cumulative impairments at each network element:

```
Transceiver Paris
  GSNR (0.1nm): 40.00 dB   OSNR ASE: 40.00 dB   Power: 0.0 dBm

Fiber span_Paris_Lyon (80.0 km, 0.2 dB/km)
  GSNR (0.1nm): 35.2 dB   OSNR ASE: 36.5 dB   Power: -16.0 dBm
  Loss: 16.0 dB   CD: 1360 ps/nm

EDFA Lyon_boost
  GSNR (0.1nm): 28.5 dB   OSNR ASE: 28.9 dB   Power: 0.0 dBm
  Gain: 16.0 dB   NF: 5.5 dB   ASE added: 9.4 dB degradation

Fiber span_Lyon_Marseille (120.0 km, 0.2 dB/km)
  GSNR (0.1nm): 22.8 dB   OSNR ASE: 24.1 dB   Power: -24.0 dBm
  NLI accumulated: 1.3 dB degradation

Transceiver Marseille
  Final GSNR (0.1nm): 22.8 dB
```

Analysis workflow:

1. **Locate degradation:** Identify largest GSNR drop (Lyon_boost: 6.7 dB)
2. **Classify impairment:** OSNR ASE vs GSNR gap reveals ASE vs NLI
3. **Validate model:** Compare to analytical NF_{eff} calculation from Chapter 1
4. **Iterate design:** Adjust amplifier gain/NF or reduce span length

Per-Channel Spectral Analysis

Command:

```
gnpy-transmission-example topology.json \
  -e eqpt_config.json --show-channels
```

Output format:

Ch. #	Channel frequency (THz)	Channel power (dBm)	OSNR ASE (signal bw, dB)
1	191.35000	-2.50	18.50
22.30	16.80		
40	193.10000	-2.48	19.20
23.10	17.40		
80	194.85000	-2.52	18.90
21.80	16.90		
96	195.95000	-2.58	18.20
20.50	15.90		

Research insights:

- **Edge channel degradation:** Ch 96 shows 0.9 dB GSNR penalty vs center
- **NLI dominance:** SNR NLI (20.5 dB) limits Ch 96, not ASE (18.2 dB)
- **Power tilt:** Edge channel 0.1 dB higher power indicates ROADM filtering or Raman tilt
- **Spectral uniformity:** <1 dB variation indicates good design

5.4.3 CSV Column Interpretation

Column	Research Application
response-id	Service identifier for batch analysis
source, destination	Network pair for connectivity studies
Pass?	Boolean feasibility or blocking reason
SNR-0.1nm	Comparable metric across different modulations
SNR-bandwidth	Actual GSNR for BER calculation
OSNR-0.1nm	ASE-only contribution for amplifier analysis
transponder-mode	Selected modulation format
baud rate (Gbaud)	Symbol rate for spectral efficiency studies
input power (dBm)	Launch power for optimization studies
spectrum (N,M)	Frequency slot (N, M) from Chapter 3

Research workflow example:

```
# Generate CSV for all services
gnpy-path-request topology.json services.json -e eqpt.json -o results.csv

# Python analysis
import pandas as pd
df = pd.read_csv('results.csv')

# Identify blocked services
blocked = df[df['Pass?'] != True]
print(f"Blocking_rate: {len(blocked)/len(df)*100:.1f}%")

# Margin analysis
df['margin_dB'] = df['SNR-bandwidth'] - df['required_OSNR']
critical = df[df['margin_dB'] < 2.0]

# Distance vs GSNR correlation
df.plot(x='path_length_km', y='SNR-0.1nm', kind='scatter')
```

5.4.4 Example CSV Output

```
response-id,source,destination,path_bandwidth,Pass?,nb of tsp pairs,total cost,...
req-001,Paris,Marseille,200.0,True,1,1,Voyager,DP-QPSK 200G,20.3,18.5,17.2,32,0.0,...
req-002,Lyon,Nice,400.0,NO_FEASIBLE_MODE,2,2,Voyager,DP-16QAM 400G,16.5,14.2,12.9,69,-3.0,...
```

5.5. CONSOLE OUTPUT

5.5.1 Path Request Summary

Example Output:

```
Computing path from Paris to Marseille
Path result for request: Paris-to-Marseille
  GSNR (0.1nm): 18.5 dB
  OSNR ASE (0.1nm): 20.3 dB
  Mode selected: DP-QPSK 200G
  Path: Paris | fiber_1 | EDFA_1 | Lyon | fiber_2 | EDFA_2 | Marseille
  Spectrum: (N=0, M=4)
  Pass: True

Computing path from Lyon to Nice
Path result for request: Lyon-to-Nice
  GSNR (0.1nm): 14.2 dB
  OSNR ASE (0.1nm): 16.5 dB
  Mode selected: DP-16QAM 400G (last explored)
  Path: Lyon | fiber_3 | EDFA_3 | Nice
```

```
Spectrum: (N=8, M=6)
Pass: False
Blocking reason: NO_FEASIBLE_MODE
```

Result summary shows mean GSNR and OSNR (average over all channels)

5.5.2 Transmission Example Output

There are 15 fiber spans over 1200 km between Paris and Marseille

The total OA (optical amplification) is 16 calculated per span:
In Paris : [...]

Propagating in power mode: power reference = 0.00 dBm

```
Transmission result:
Final GSNR (0.1 nm): 18.45 dB
```

The GSNR per channel at the end of the line is:

Ch.#	Channel frequency (THz)	Channel power (dBm)	OSNR ASE (signal bw, dB)
0	193.10000	-2.50	
19.20			
1	193.15000	-2.48	
19.25			
2	193.20000	-2.52	
19.18			
...			

5.6. AUTO-DESIGNED NETWORK OUTPUT

5.6.1 Purpose

After running `design_network()`, GNPY can export the modified topology with:

- Automatically placed EDFAs
- Configured amplifier gains
- Set VOA attenuations
- ROADM power targets

5.6.2 Generation

Command:

```
gnpy-transmission-example topology.json \
  --save-network auto_designed_topology.json
```

5.6.3 Structure

Listing 5.5: Auto-Designed Topology

```
{
  "elements": [
    {
      "uid": "fiber_Paris_Lyon",
      "type": "Fiber",
      "type_variety": "SSMF",
      "params": {
        "length": 80000,
        "loss_coef": 0.0002,
        "length_units": "m"
      }
    }
  ]
}
```

```

    }
  },
  {
    "uid": "Edfa_auto_Paris_Lyon", // Auto-inserted
    "type": "Edfa",
    "type_variety": "std_medium_gain",
    "operational": {
      "gain_target": 16.5, // Auto-calculated
      "delta_p": 0.0,
      "out_voa": 2.3, // Auto-set
      "tilt_target": 0.0
    }
  },
  ...
],
"connections": [ ... ]
}

```

Key Auto-Design Outputs:

- gain_target: Computed from span loss + target ΔP
- out_voa: VOA attenuation for power management
- delta_p: Power variation from reference

5.7. PROGRAMMATIC ACCESS FOR RESEARCH

5.7.1 Python API Integration

Listing 5.6: Parsing JSON Response

```

1 from gnpy.tools.json_io import load_json
2 from numpy import log10
3
4 # Load response
5 response = load_json('response.json')
6
7 for result in response['response']:
8     rid = result['response-id']
9
10    if 'path-properties' in result:
11        # Extract metrics dictionary
12        metrics = {m['metric-type']: m['accumulative-value']
13                   for m in result['path-properties']['path-metric']}
14
15        gsnr = metrics['SNR-bandwidth']
16        osnr_ase = metrics['OSNR-bandwidth']
17
18        # Compute NLI contribution (from Chapter 1, Section 1.4)
19        gsnr_lin = 10**(gsnr/10)
20        ase_lin = 10**(osnr_ase/10)
21        nli_lin = 1 / (1/gsnr_lin - 1/ase_lin)
22        osnr_nli = 10 * log10(nli_lin)
23
24        print(f"{rid}: GSNR={gsnr:.1f} dB, "
25              f"ASE={osnr_ase:.1f} dB, NLI={osnr_nli:.1f} dB")
26    else:
27        # Blocked path
28        reason = result['no-path']['no-path']
29        print(f"{rid}: BLOCKED - {reason}")

```

5.7.2 CSV Batch Analysis

Listing 5.7: Statistical Analysis Example

```

1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Load CSV output
5 df = pd.read_csv('results.csv')
6
7 # Blocking analysis
8 print(f"Total services: {len(df)}")
9 print(f"Blocked: {len(df[df['Pass?'] != True])}")
10 print(f"Blocking rate: {len(df[df['Pass?'] != True])/len(df)*100:.1f}%"
11       )
12
13 # Margin distribution
14 df['margin_dB'] = df['SNR-bandwidth'] - df['required_OSNR']
15 df[df['Pass?'] == True]['margin_dB'].hist(bins=20)
16 plt.xlabel('GSNR Margin (dB)')
17 plt.ylabel('Count')
18 plt.title('Service Margin Distribution')
19
20 # Path length vs GSNR correlation
21 df.plot(x='path_length_km', y='SNR-0.1nm', kind='scatter',
22         alpha=0.6, figsize=(10,6))

```

5.8. METRICS INTERPRETATION

5.8.1 GSNR vs. OSNR

Metric	Physical Meaning
OSNR_{ASE}	Signal-to-noise ratio considering only ASE
OSNR_{NLI}	Signal-to-noise ratio considering only NLI
GSNR	Generalized SNR including both ASE and NLI

Relationships:

$$\frac{1}{\text{GSNR}} = \frac{1}{\text{OSNR}_{\text{ASE}}} + \frac{1}{\text{OSNR}_{\text{NLI}}} \quad (5.4)$$

$$\text{GSNR}_{\text{lin}} = \frac{P_{\text{signal}}}{P_{\text{ASE}} + P_{\text{NLI}}} \quad (5.5)$$

Critical Understanding: In modern high-capacity networks, NLI often dominates over ASE:

- Short-reach (< 500 km): ASE-dominated, $\text{GSNR} \approx \text{OSNR}_{\text{ASE}}$
- Long-reach (> 1000 km): NLI-dominated, $\text{GSNR} < \text{OSNR}_{\text{ASE}}$ significantly

5.8.2 GSNR Margins

$$\text{Margin} = \text{GSNR}_{\text{effective}} - \text{OSNR}_{\text{required}} - \text{System Margins} \quad (5.6)$$

Recommended Margins:

Application	Minimum	Recommended
New deployment	1 dB	3 dB
Aging network	2 dB	4 dB
High availability	3 dB	5 dB

Margin Breakdown:

- **Implementation margin:** Model inaccuracy, real vs. ideal (1 dB)
- **Aging margin:** Fiber degradation, connector loss increase (1 dB)
- **Temperature margin:** Amplifier NF variation (0.5 dB)
- **Repair margin:** Temporary cable repairs (1 dB)

5.8.3 Per-Channel Analysis**Why Per-Channel Matters:**

- Spectrum edges suffer from amplifier gain ripple
- Center channels experience higher XPM
- SRS causes power tilt (lower frequencies lose power)

Extracting Per-Channel Data:

Listing 5.8: Access Per-Channel Metrics

```

1 # From transceiver object
2 destination = path[-1] # Last element = destination transceiver
3
4 # Per-channel arrays
5 gsnr_per_channel = destination.snr_01nm # dB
6 osnr_ase_per_channel = destination.osnr_ase_01nm # dB
7 power_per_channel = destination.tx_power # W
8
9 # Statistics
10 worst_channel_gsnr = min(gsnr_per_channel)
11 best_channel_gsnr = max(gsnr_per_channel)
12 mean_gsnr = mean(gsnr_per_channel)
13
14 print(f"GSNR range: {worst_channel_gsnr:.2f} - "
15       f"{best_channel_gsnr:.2f} dB")
16 print(f"GSNR variation: {best_channel_gsnr - worst_channel_gsnr:.2f} dB
17       ")

```

Acceptable Variation:

- < 2 dB: Excellent equalization
- 2-4 dB: Good (typical for C-band)
- 4-6 dB: Marginal (may need ROADM adjustment)
- > 6 dB: Poor (investigate amplifier gain tilt)

5.9. TROUBLESHOOTING**5.9.1 Low GSNR Issues***Symptom: GSNR < Required***Example:**

Path: Paris Lyon Marseille
GSNR: 10.5 dB
Required OSNR: 12.0 dB (DP-QPSK)
Result: NO_FEASIBLE_MODE

Diagnostic Steps:

1. Check OSNR_{ASE} vs. OSNR_{NLI} :

```

1 osnr_ase = destination.osnr_ase_01nm[0]      # e.g., 15.0 dB
2 osnr_nli = destination.osnr_nli[0]           # e.g., 12.5 dB
3 gsnr = destination.snr_01nm[0]               # e.g., 10.5 dB
4
5 # Determine limiting factor
6 if osnr_ase < osnr_nli:
7     print("ASE-limited: Add gain, reduce NF")
8 else:
9     print("NLI-limited: Reduce power, increase spacing")

```

2. ASE-Limited Solutions:

- Increase launch power (if not saturated)
- Use lower-NF amplifiers
- Reduce number of spans (add regeneration)
- Check amplifier configurations

3. NLI-Limited Solutions:

- Reduce launch power
- Increase channel spacing (50 to 75 GHz)
- Use lower-order modulation
- Check fiber γ (effective area)
- Reduce span count

Example Analysis

Listing 5.9: Detailed Impairment Breakdown

```

1 # Power budget at destination
2 signal_power = watt2dbm(destination.tx_power[0])      # -3.0 dBm
3 ase_power = watt2dbm(destination.ase[0])               # -18.0 dBm
4 nli_power = watt2dbm(destination.nli[0])               # -15.0 dBm
5
6 osnr_ase = signal_power - ase_power      # 15.0 dB
7 osnr_nli = signal_power - nli_power      # 12.0 dB
8
9 # GSNR calculation (in linear domain)
10 gsnr_lin = destination.signal[0] / \
11             (destination.ase[0] + destination.nli[0])
12 gsnr = lin2db(gsnr_lin)      # 10.46 dB
13
14 print(f"Signal: {signal_power:.1f} dBm")
15 print(f"ASE:      {ase_power:.1f} dBm          OSNR_ASE = {osnr_ase:.1f} dB")
16 print(f"NLI:      {nli_power:.1f} dBm          OSNR_NLI = {osnr_nli:.1f} dB")
17 print(f"GSNR:      {gsnr:.1f} dB")
18 print()
19 print("Diagnosis: NLI-limited (NLI 3 dB worse than ASE)")
20 print("Action: Reduce launch power by 2-3 dB")

```

5.9.2 Spectrum Exhaustion

Symptom: NOT_ENOUGH_SPECTRUM

Diagnostic:

Listing 5.10: Spectrum Usage Analysis

```

1 from gnpy.topology.spectrum_assignment import build oms_list

```

```

2
3 # Build OMS list
4 oms_list = build_oms_list(network, equipment)
5
6 for i, oms in enumerate(oms_list):
7     total_slots = len(oms.frequency_slots)
8     used_slots = sum(oms.frequency_slots) # 1 = occupied
9     free_slots = total_slots - used_slots
10
11     utilization = 100 * used_slots / total_slots
12
13     print(f"OMS {i}: {used_slots}/{total_slots} slots used "
14           f"({utilization:.1f}%)")
15
16     if utilization > 90:
17         print(f" WARNING: OMS {i} near capacity!")

```

Solutions:

1. **Wavelength Reuse:** Different OMS can use same spectrum
2. **Defragmentation:** Rearrange existing services to consolidate free spectrum
3. **Add Capacity:** Install additional fiber pairs
4. **Spectrum Expansion:** Use C+L band if supported

5.9.3 Amplifier Saturation

Symptom: Warnings about $p_{\max}$

WARNING: EDFA_3 output power 21.5 dBm exceeds $p_{\max}$ 21.0 dBm
Gain reduced from 18.0 dB to 17.5 dB

Analysis:

```

1 # Check amplifier loading
2 for element in path:
3     if isinstance(element, Edfa):
4         p_in = watt2dbm(element._psig_in)
5         p_out = watt2dbm(sum(spectral_info.signal +
6                             spectral_info.ase +
7                             spectral_info.nli))
8
9         gain_actual = p_out - p_in
10        gain_target = element.params.gain_target
11
12        if abs(gain_actual - gain_target) > 0.5:
13            print(f"{element.uid}: Gain mismatch!")
14            print(f" Target: {gain_target:.1f} dB")
15            print(f" Actual: {gain_actual:.1f} dB")
16            print(f" P_out: {p_out:.1f} dBm "
17                  f"(p_max = {element.params.p_max:.1f} dBm)")

```

Solutions:

- Reduce number of channels
- Reduce per-channel power
- Use higher $p_{\max}$ amplifier model
- Add output VOA before saturation

5.9.4 CD/PMD Penalties

High Chromatic Dispersion

Listing 5.11: Check CD Penalty

```

1 cd_accumulated = destination.chromatic_dispersion # ps/nm
2 cd_penalty = destination.penalties['chromatic_dispersion'] # dB
3
4 print(f"Accumulated CD: {cd_accumulated:.0f} ps/nm")
5 print(f"CD Penalty: {cd_penalty:.2f} dB")
6
7 if cd_penalty > 2.0:
8     print("WARNING: High CD penalty!")
9     print("Actions:")
10    print("  - Check transponder CD tolerance")
11    print("  - Consider DCF (dispersion compensating fiber)")
12    print("  - Use mode with higher CD tolerance")

```

Typical CD Tolerances:

Modulation	CD Tolerance (ps/nm)
DP-QPSK 100G	50,000
DP-8QAM 300G	40,000
DP-16QAM 400G	30,000
DP-64QAM 600G	20,000

High PMD

```

1 pmd_accumulated = destination.pmd # ps
2 pmd_penalty = destination.penalties['pmd'] # dB
3
4 pmd_per_km = pmd_accumulated / sqrt(total_length_km)
5
6 print(f"Accumulated PMD: {pmd_accumulated:.2f} ps")
7 print(f"PMD coefficient: {pmd_per_km:.3f} ps/km ")
8 print(f"PMD Penalty: {pmd_penalty:.2f} dB")
9
10 if pmd_per_km > 0.1:
11     print("WARNING: High PMD fiber!")
12     print("Consider fiber replacement")

```

PMD Guidelines:

- Modern fiber: $< 0.05 \text{ ps}/\sqrt{\text{km}}$
- Legacy fiber: $0.1\text{-}0.5 \text{ ps}/\sqrt{\text{km}}$
- High penalty threshold: $\text{PMD} \times \text{Baud Rate} > 0.3$

5.10. VALIDATION TECHNIQUES

5.10.1 Sanity Checks

Power Conservation

Listing 5.12: Validate Power Budget

```

1 def validate_power_budget(path):
2     """Check power conservation through path"""
3
4     for i, element in enumerate(path[:-1]):
5         # Output power of this element
6         p_out = sum(element.pch_out_dbm) if hasattr(element, '
7             pch_out_dbm') else None

```

```

8      # Input power of next element (should match)
9      next_elem = path[i+1]
10     if hasattr(next_elem, '_psig_in'):
11         p_in_next = watt2dbm(next_elem._psig_in)
12
13     if p_out is not None:
14         delta = abs(p_out - p_in_next)
15         if delta > 0.1: # 0.1 dB tolerance
16             print(f"Power mismatch at {element.uid}      {
17                   next_elem.uid}")
18             print(f"   Output:  {p_out:.2f} dBm")
19             print(f"   Next in: {p_in_next:.2f} dBm")
20             print(f"   Delta:   {delta:.2f} dB")

```

OSNR Budgeting

Listing 5.13: OSNR Budget Validation

```

1  def osnr_budget_check(path, equipment):
2      """Compare analytical OSNR to simulated"""
3
4      # Count spans and amplifiers
5      n_spans = sum(1 for e in path if isinstance(e, Fiber))
6      n_edfas = sum(1 for e in path if isinstance(e, Edfa))
7
8      # Simple OSNR model
9      launch_power = 0 # dBm
10     span_loss = 16    # dB (typical 80 km)
11     edfa_nf = 5       # dB (typical)
12
13     # ASE accumulation
14     osnr_analytical = launch_power + \
15                       10*log10(12.5e9 / 32e9) - \
16                       edfa_nf - \
17                       10*log10(n_edfas)
18
19     # Simulated OSNR
20     destination = path[-1]
21     osnr_simulated = mean(destination.osnr_ase_01nm)
22
23     delta = abs(osnr_analytical - osnr_simulated)
24
25     print(f"OSNR Budget Check:")
26     print(f"   Analytical: {osnr_analytical:.1f} dB")
27     print(f"   Simulated:  {osnr_simulated:.1f} dB")
28     print(f"   Delta:      {delta:.1f} dB")
29
30     if delta > 2.0:
31         print("               Large discrepancy - investigate!")
32     else:
33         print("               Good agreement")

```

5.10.2 Regression Testing

Listing 5.14: Automated Testing

```

1  import pytest
2
3  def test_known_link_osnr():

```

```

4     """Test OSNR for well-characterized link"""
5
6     # Load reference network
7     network = load_network('test_network.json', equipment)
8     path = [network[uid] for uid in ['Tx_A', 'fiber_1',
9                                     'EDFA_1', 'Rx_B']]
10
11    # Propagate
12    si = create_input_spectral_information(...)
13    for element in path:
14        element(si)
15
16    # Check results
17    destination = path[-1]
18    osnr = mean(destination.osnr_ase_01nm)
19
20    # Reference value from measurement/previous sim
21    osnr_reference = 20.5 # dB
22
23    assert abs(osnr - osnr_reference) < 0.5, \
24        f"OSNR {osnr:.2f} dB differs from reference {osnr_reference:.2f} dB"
25
26    def test_nli_scaling():
27        """Verify NLI scales with P """
28
29        powers_dbm = [-3, 0, 3]
30        nli_values = []
31
32        for power in powers_dbm:
33            si = create_si_with_power(power)
34            fiber(si)
35            nli_values.append(mean(si.nli))
36
37        # Check ~9 dB increase for 3 dB power increase
38        nli_ratio = nli_values[2] / nli_values[0] # Linear
39        nli_ratio_db = lin2db(nli_ratio)
40
41        expected_ratio_db = 3 * 3 # 3 dB power          9 dB NLI
42
43        assert abs(nli_ratio_db - expected_ratio_db) < 1.0

```

5.11. PERFORMANCE ANALYSIS

5.11.1 Comparing Alternatives

Mode Comparison

Listing 5.15: Compare Transponder Modes

```

1    def compare_modes(path, modes, equipment):
2        """Evaluate multiple modes for same path"""
3
4        results = []
5
6        for mode in modes:
7            # Create SI for this mode
8            si = create_input_spectral_information(
9                baud_rate=mode['baud_rate'],

```

```

10         tx_osnr=mode['tx_osnr'],
11         ...
12     )
13
14     # Propagate
15     path_copy = deepcopy(path)
16     for element in path_copy:
17         element(si)
18
19     # Collect metrics
20     dest = path_copy[-1]
21     gsnr = mean(dest.snr_01nm)
22     margin = gsnr - mode['OSNR']
23
24     results.append({
25         'mode': mode['format'],
26         'bit_rate': mode['bit_rate'] / 1e9, # Gb/s
27         'gsnr': gsnr,
28         'required': mode['OSNR'],
29         'margin': margin,
30         'feasible': margin > 0
31     })
32
33     # Display comparison
34     print(f"{'Mode':<20} {'Rate':>8} {'GSNR':>8} {'Req':>8} "
35           f"{'Margin':>8} {'Pass':>6}")
36     print("-" * 70)
37     for r in sorted(results, key=lambda x: x['bit_rate'], reverse=True):
38         print(f"{r['mode']:<20} {r['bit_rate']:>8.0f} "
39               f"{r['gsnr']:>8.2f} {r['required']:>8.2f} "
40               f"{r['margin']:>8.2f} {r['feasible']:>6}")
41
42     return results

```

Example Output:

Mode	Rate	GSNR	Req	Margin	Pass
DP-64QAM 600G	600	14.50	28.00	-13.50	False
DP-16QAM 400G	400	18.20	23.00	-4.80	False
DP-8QAM 300G	300	19.80	19.00	0.80	True
DP-QPSK 200G	200	22.00	15.00	7.00	True

Path Comparison

```

1 def compare_paths(network, source, destination, k=3):
2     """Compare top-k shortest paths"""
3
4     from networkx import shortest_simple_paths
5
6     paths_gen = shortest_simple_paths(network, source,
7                                       destination, weight='weight')
8
9     results = []
10    for i, path_nodes in enumerate(islice(paths_gen, k)):
11        # Convert to path elements
12        path = [network[uid] for uid in path_nodes]
13

```

```

14     # Propagate
15     si = create_input_spectral_information(...)
16     for elem in path:
17         elem(si)
18
19     # Metrics
20     hops = len([e for e in path if isinstance(e, Fiber)])
21     gsnr = mean(path[-1].snr_01nm)
22
23     results.append({
24         'path_id': i+1,
25         'hops': hops,
26         'gsnr': gsnr,
27         'path': ' '.join([e.uid for e in path if
28                             isinstance(e, (Fiber, Roadm))])
29     })
30
31     # Display
32     for r in results:
33         print(f"Path {r['path_id']}: {r['hops']} hops, "
34               f"GSNR = {r['gsnr']:.2f} dB")
35         print(f"    {r['path']}")

```

5.11.2 Sensitivity Analysis

Listing 5.16: Parameter Sensitivity

```

1 def power_sweep(path, power_range_dbm):
2     """Sweep launch power and measure GSNR"""
3
4     gsnr_results = []
5
6     for power_dbm in power_range_dbm:
7         si = create_si_with_power(power_dbm)
8
9         path_copy = deepcopy(path)
10        for element in path_copy:
11            element(si)
12
13        dest = path_copy[-1]
14        gsnr = mean(dest.snr_01nm)
15        osnr_ase = mean(dest.osnr_ase_01nm)
16        osnr_nli = mean(dest.osnr_nli)
17
18        gsnr_results.append({
19            'power': power_dbm,
20            'gsnr': gsnr,
21            'osnr_ase': osnr_ase,
22            'osnr_nli': osnr_nli
23        })
24
25    return gsnr_results
26
27 # Usage
28 powers = linspace(-5, 5, 21) # -5 to +5 dBm, 0.5 dB steps
29 results = power_sweep(path, powers)
30
31 # Find optimal power
32 optimal = max(results, key=lambda x: x['gsnr'])

```

```

33 print(f"Optimal power: {optimal['power']:.1f} dBm")
34 print(f"GSR at optimal: {optimal['gsnr']:.2f} dB")

```

5.12. PRACTICAL WORKFLOWS

5.12.1 Network Planning

Workflow:

1. Initial Assessment:

```

# Quick feasibility check
gnpy-transmission-example topology.json -e eqpt_config.json

```

2. Service Simulation:

```

# Test all services
gnpy-path-request topology.json services.json \
-e eqpt_config.json \
-o results.csv

```

3. Analysis:

- Open CSV in Excel/Python
- Filter blocked services
- Sort by margin
- Identify problem paths

4. Optimization Iteration:

- Adjust amplifier placements
- Modify ROADM configurations
- Change modulation formats
- Re-run simulation

5. Export Final Design:

```

gnpy-transmission-example topology.json \
--save-network final_design.json

```

5.12.2 Troubleshooting Workflow

1. **Identify Problem:** CSV shows blocking reason
2. **Isolate Path:** Run single service to focus analysis
3. **Enable Detailed Output:**

```

gnpy-transmission-example topology.json \
-e eqpt_config.json \
--source NodeA --destination NodeB \
--show-channels

```

4. **Check Per-Element:** Print each element's output
5. **Validate Models:** Compare to analytical formulas
6. **Implement Fix:** Modify configuration
7. **Verify:** Re-run and confirm improvement

5.13. BEST PRACTICES

5.13.1 Simulation Setup

1. **Start Simple:** Single span before full network
2. **Validate Equipment:** Check amplifier NF, fiber loss match datasheets
3. **Use Realistic Margins:** System margins ≥ 2 dB
4. **Consider Aging:** OSNR degrades 0.5-1 dB over years

5. **Document Assumptions:** Record launch powers, spacing, etc.

5.13.2 Output Analysis

1. **Check Worst Case:** Minimum GSNR across all channels
2. **Validate Extremes:** Edge channels often problematic
3. **Compare Directions:** Bidirectional services should be balanced
4. **Track Margins:** Low margins indicate fragility
5. **Archive Results:** Keep simulation outputs for reference

5.13.3 Common Pitfalls

Pitfall	Solution
Over-optimistic margins	Add system margins (≈ 2 dB) explicitly
Ignoring penalties	Enable CD/PMD penalties in transponder modes
Wrong bandwidth	Use signal BW for BER, 0.1nm for comparison
Single-channel tests	Always test full loading for NLI accuracy
Static config	Re-run after topology changes

5.14. SUMMARY

5.14.1 Key Metrics Hierarchy

1. **Feasibility:** Pass/Fail or blocking reason
2. **GSNR_{effective}:** After penalties, primary metric
3. **Margin:** $\text{GSNR}_{\text{eff}} - \text{OSNR}_{\text{req}} - \text{sys_margins}$
4. **OSNR_{ASE} vs OSNR_{NLI}:** Diagnostic
5. **Per-channel variation:** Quality indicator

5.14.2 Analysis Checklist

- ☐ All services pass or have documented blocking reasons
- ☐ Margins ≈ 2 dB for all passing services
- ☐ Per-channel GSNR variation < 4 dB
- ☐ No amplifier saturation warnings
- ☐ Power budget validated
- ☐ Penalties reasonable (< 2 dB total)
- ☐ Results compared to analytical estimates
- ☐ Sensitivity analysis performed for critical paths

5.14.3 Next Steps

After mastering output analysis:

1. **API Integration:** Use GNPpy programmatically in network controllers
2. **Optimization:** Automate parameter tuning
3. **Monitoring:** Compare simulation to field measurements
4. **Prediction:** Use models for failure scenario analysis

Chapter 6

Practical Guide to Network Simulation

CHAPTER OVERVIEW

This chapter provides hands-on guidance for engineers simulating optical networks with GNPpy. Unlike previous chapters that explain *how GNPpy works internally*, this chapter shows *how to use GNPpy* to simulate real networks.

Target Audience:

- Optical network engineers new to GNPpy
- Network planners evaluating path feasibility
- System designers optimizing network configurations
- Anyone who needs to simulate an optical network quickly

What You'll Learn:

- Install and verify GNPpy
- Run your first successful simulation in 30 minutes
- Create models of your actual network
- Interpret results and make engineering decisions
- Troubleshoot common issues
- Find realistic parameter values

Prerequisites:

- Basic understanding of WDM optical networks
- Familiarity with command line (bash/terminal)
- Python 3.9+ installed
- Text editor or IDE

6.1. GETTING STARTED

6.1.1 Installation

Using prebuilt Docker images

This is the most feasible way to use GNPpy for personal use. **Step 1: Install Docker** **Step 2:**

```
# linux/mac
docker run -it --rm --volume $(pwd):/shared telecominfrastructure/oopt-gnpy

# windows
PS C:\> docker run -it --rm --volume ${PWD}:/shared telecominfrastructure/oopt-gnpy
```

This will start a Docker environment and it is directly usable via CLI. The `C : \Users\username\example-data` is a shared folder that can be editable from PC and used in the Docker environment.

Python Quick Install

This should be the way of install for a real case usage.

Step 1: Install GNPpy

```
# Install from PyPI
pip install gnpyp

# Verify installation
gnpyp-transmission-example --help
```

Step 2: Download Example Files

```
# Clone GNPpy repository for examples
git clone https://github.com/Telecom-Infra-Project/oopt-gnpyp.git
cd oopt-gnpyp/examples

# Or download specific files
wget https://raw.githubusercontent.com/Telecom-Infra-Project/\
oopt-gnpyp/master/examples/edfa_example_network.json
wget https://raw.githubusercontent.com/Telecom-Infra-Project/\
oopt-gnpyp/master/examples/eqpt_config.json
```

Step 3: Verify Installation

```
# Run example simulation
gnpyp-transmission-example edfa_example_network.json \
-e eqpt_config.json

# Expected output (abbreviated):
# There are 8 fiber spans over 608 km between Site_A and Site_C
# Now propagating between Site_A and Site_C
# Transceiver Site_A
#   GSNR (0.1 nm, dB):          23.00
# Transceiver Site_C
#   GSNR (0.1 nm, dB):          17.85
```

Troubleshooting Installation

Issue: command not found: gnpyp-transmission-example

Solution:

```
# Check if installed
pip show gnpyp

# If not in PATH, use python -m
python -m gnpyp.cli.gnpyp_transmission_example --help

# Add to PATH (Linux/Mac)
export PATH="$HOME/.local/bin:$PATH"
# Add to ~/.bashrc for permanent
```

Issue: ImportError: No module named 'numpy'

Solution:

```
# Install dependencies
pip install numpy scipy networkx matplotlib xlrd

# Or reinstall gnpyp with dependencies
pip install --upgrade gnpyp
```

Issue: Permission denied

Solution:

```
# Install for current user only
pip install --user gnpyp

# Or use virtual environment (recommended)
python -m venv gnpyp-env
source gnpyp-env/bin/activate # Linux/Mac
gnpyp-env\Scripts\activate    # Windows
pip install gnpyp
```

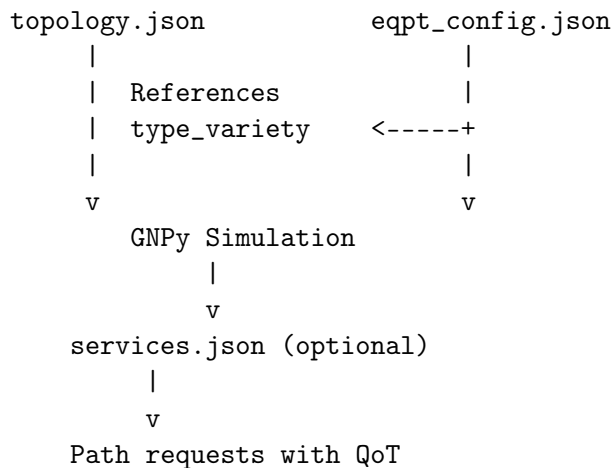
6.1.2 GNPpy Files

GNPpy requires 2-4 input files depending on the use case:

File	Purpose	Required?
topology.json	Network topology (nodes, fibers, connections)	Yes
eqpt_config.json	Equipment library (amplifiers, transceivers, fibers)	Yes
services.json	Service requests (source, destination, bit rate)	For path requests
sim_params.json	Simulation parameters (NLI model, spectrum)	Optional

Table 6.1: GNPpy Input Files

File Relationships:



6.1.3 Command Line Tools

GNPpy provides two main command-line tools:

gnpyp-transmission-example

Purpose: Simulate full spectrum propagation through network

Usage:

```
gnpyp-transmission-example [OPTIONS] TOPOLOGY EQUIPMENT

# Basic usage
gnpyp-transmission-example topology.json -e eqpt_config.json

# Common options
-e, --equipment FILE          # Equipment library (required)
-o FILE                      # Save results to CSV
--output-json FILE           # Save detailed JSON response
--sim-params FILE            # Custom simulation parameters
--verbose                    # Detailed logging output, more depth with -v
-v
```

```
--no-kopaths          # Compute only one path per request
```

When to use:

- Evaluate overall network capacity
- Optimize amplifier placement and gains
- Assess worst-case OSNR across all channels
- Design new network from scratch

gnpy-path-request

Purpose: Compute specific paths between nodes with QoT validation

Usage:

```
gnpy-path-request TOPOLOGY SERVICES [OPTIONS] EQUIPMENT

# Basic usage
gnpy-path-request topology.json services.json \
  -e eqpt_config.json

# Common options
-o FILE          # Save results to CSV
--output-json FILE # Save detailed JSON response
```

When to use:

- Check if specific paths are feasible
- Select transponder modes for services
- Validate service requests with constraints
- Compare multiple path options

6.1.4 Quick Decision Tree

Which tool should I use?

START

```
|
+--> Do you have specific services to simulate?
|
| YES --> Use gnpy-path-request
|         (Requires services.json)
|
| NO ---> Do you want to evaluate full network?
|
|         YES --> Use gnpy-transmission-example
|
|         NO ---> Do you want to design amplifier placement?
|
|                 YES --> Use gnpy-transmission-example
|                         |
|                         | with --save-network
|                         |
|                 NO ---> Start with transmission-example
|                         |
|                         | to explore capabilities
```

6.2. A FIRST SIMULATION

This is a 3-node network simulation to understand the workflow, part of one of the examples GNPpy provides.

6.2.1 Step 1: Create Topology

Create a file named `simple_network.json`:

```
{
  "elements": [
    {
      "uid": "Stockholm",
      "type": "Transceiver",
      "metadata": {
        "location": {
          "city": "Stockholm",
          "region": "Sweden",
          "latitude": 59.33,
          "longitude": 18.06
        }
      }
    },
    {
      "uid": "fiber_1",
      "type": "Fiber",
      "type_variety": "SSMF",
      "params": {
        "length": 80.0,
        "length_units": "km"
      }
    },
    {
      "uid": "Gothenburg",
      "type": "Transceiver",
      "metadata": {
        "location": {
          "city": "Gothenburg",
          "region": "Sweden",
          "latitude": 57.71,
          "longitude": 11.97
        }
      }
    }
  ],
  "connections": [
    {
      "from_node": "Stockholm",
      "to_node": "fiber_1"
    },
    {
      "from_node": "fiber_1",
      "to_node": "Gothenburg"
    }
  ]
}
```

Key Points:

- uid: Unique identifier
- type: Must match GNPpy element types exactly

- `type_variety`: References equipment library
 - `connections`: Define network topology
 - `metadata`: Optional but useful for documentation
- Every type or variety needs to be defined in the equipment file referenced.

6.2.2 Step 2: Run Simulation

`-e` calls the equipment file to reference.

```
# transmission example + network + -e equipment
gnpy-transmission-example simple_network.json \
    -e eqpt_config.json

# Or if you have the examples directory:
gnpy-transmission-example simple_network.json \
    -e examples/eqpt_config.json
```

6.2.3 Step 3: Output

Expected Console Output:

```
INFO: Auto-design: adding missing elements
INFO: Auto-design: placed 1 amplifier
INFO: Auto-design: setting fiber spans
INFO: Auto-design: running auto-design algorithm
INFO: There are 1 fiber spans over 80 km
        between Stockholm and Gothenburg
```

Now propagating between Stockholm and Gothenburg

Transceiver Stockholm

```
Mode: default mode
GSMR (0.1 nm, dB):      23.45
OSNR ASE (0.1 nm, dB):  24.12
OSNR NLI (0.1 nm, dB):  31.87
CD (ps/nm):             1336.0
PMD (ps):               8.94
PDL (dB):               0.0
Latency (ms):           0.40
```

Transceiver Gothenburg

```
Mode: default mode
GSMR (0.1 nm, dB):      23.45
OSNR ASE (0.1 nm, dB):  24.12
OSNR NLI (0.1 nm, dB):  31.87
CD (ps/nm):             1336.0
PMD (ps):               8.94
PDL (dB):               0.0
Latency (ms):           0.40
Penalty (dB):           0.32
```

Summary:

```
Path feasibility: PASS
GSMR margin:      8.45 dB (requirement: 15.0 dB)
```

See [6.2](#) for reference.

Metric	Meaning	Typical Value	Good
GSNR	Signal quality including all impairments	> 15-20 dB	
OSNR ASE	Signal-to-noise from amplifier noise	> 20 dB	
OSNR NLI	Signal-to-noise from nonlinear effects	> 25 dB	
CD	Accumulated chromatic dispersion	< 20,000 ps/nm	
PMD	Polarization mode dispersion	< 20 ps	
Penalty	OSNR reduction from impairments	< 2 dB	
Margin	GSNR above requirement	> 3 dB	

Table 6.2: Output Metrics Interpretation

This example:

- **GSNR = 23.45 dB:** Excellent signal quality
- **Margin = 8.45 dB:** Very healthy margin (> 3 dB target)
- **CD = 1336 ps/nm:** Moderate, well within transceiver limits
- **Penalty = 0.32 dB:** Low penalty, good path

Conclusion: This path is **feasible** with excellent performance.

What Defines Feasibility?

Path feasibility is determined by comparing the results against the transceiver mode requirements defined in `eqpt_config.json`:

```
{
  "Transceiver": [
    {
      "type_variety": "vendorA_transponder",
      "mode": [
        {
          "format": "DP-QPSK",
          "baud_rate": 32e9,
          "OSNR": 15.0,           // Minimum required GSNR (dB)
          "bit_rate": 100e9,
          "tx_osnr": 40.0,
          "min_spacing": 50e9,
          "cost": 1,
          "penalties": {
            "chromatic_dispersion": 80000, // CD tolerance (ps/nm)
            "pmd": 40,                     // PMD tolerance (ps)
            "pdl": 5.5                     // PDL tolerance (dB)
          }
        },
        {
          "format": "DP-16QAM",
          "baud_rate": 32e9,
          "OSNR": 20.5,           // Higher modulation needs more GSNR
          "bit_rate": 200e9,
          "tx_osnr": 40.0,
          "min_spacing": 50e9,
          "cost": 1,
          "penalties": {
            "chromatic_dispersion": 60000, // Tighter CD tolerance
            "pmd": 20,                     // Tighter PMD tolerance
            "pdl": 3.0
          }
        }
      ]
    }
  ]
}
```

```

    ]
  }
]
}

```

Feasibility Rules:

- **GSNR:** Path GSNR $\geq$ Mode OSNR requirement
- **CD:** Accumulated CD $\leq$ Transceiver CD tolerance
- **PMD:** Accumulated PMD $\leq$ Transceiver PMD tolerance
- **Spectrum:** Sufficient contiguous spectrum available
- **Route:** Path respects explicit route constraints
- **Margin:** GSNR - OSNR > 3 dB (recommended)

In our example:

- Path GSNR = 23.45 dB vs. requirement 15.0 dB $\rightarrow$ **PASS** (Margin: 8.45 dB)
- Path CD = 1336 ps/nm vs. tolerance 80000 ps/nm $\rightarrow$ **PASS**
- Path PMD = 8.94 ps vs. tolerance 40 ps $\rightarrow$ **PASS**
- Penalty = 0.32 dB $\rightarrow$ Low impairment

Result: All constraints satisfied $\rightarrow$ **FEASIBLE** with excellent margin

Higher-order modulations (16-QAM, 64-QAM) require higher OSNR values but deliver greater bit rates. GNPpy can automatically select the best mode that meets GSNR requirements.

6.2.4 Step 5: Save Auto-Designed Network

GNPpy automatically adds equipment [subsection 2.4.1](#). To save the complete design after auto-design phase:

```

gnpy-transmission-example simple_network.json \
-e eqpt_config.json \
--save-network designed_simple_network.json

```

Use `-no-insert-edfas` to disable insertion of EDFAs after ROADMs and fibers as well as splitting of fibers by auto-design. (default: False)

Open `designed_simple_network.json` to see:

- Amplifier placement (GNPpy added EDFAs automatically)
- Configured gains and powers
- Complete network ready for deployment planning

6.2.5 Step 6: Experiment

Try modifying parameters to see effects:

Experiment 1: Increase distance

```

{
  "uid": "fiber_1",
  "type": "Fiber",
  "params": {
    "length": 200.0 // Changed from 80 to 200 km
  }
}

```

Re-run and observe:

- More amplifiers added
- Lower GSNR
- Higher accumulated CD and PMD

Experiment 2: Change fiber type

```

{
  "uid": "fiber_1",
  "type": "Fiber",

```

```
"type_variety": "NZDSF" // Changed from SSMF
}
```

Observe:

- Lower CD accumulation
- Different nonlinear behavior

6.3. CREATING A NETWORK MODEL

Now let's model a realistic network step-by-step.

6.3.1 Network Information Gathering

Before creating GNPY files, collect this information:

Category	Information Needed
Topology	Node names and locations Fiber routes and lengths Amplifier sites ROADM locations
Equipment	Fiber type (SSMF, NZDSF, etc.) Amplifier models and specs ROADM models Transceiver types and modes
Parameters	Fiber loss coefficient (dB/km) Amplifier gain and noise figure Launch power per channel Channel spacing and count

Table 6.3: Network Information Checklist

It does worth the time checking the examples provided by GNPY.

6.3.2 Step-by-Step Topology Creation

Step 1: Define Nodes

Start with endpoints (transceivers):

```
{
  "elements": [
    {
      "uid": "Site_A",
      "type": "Transceiver",
      "metadata": {
        "location": {
          "city": "City A",
          "latitude": 59.33,
          "longitude": 18.06
        }
      }
    },
    {
      "uid": "Site_B",
      "type": "Transceiver",
      "metadata": {
```

```

        "location": {
            "city": "City B",
            "latitude": 59.86,
            "longitude": 17.64
        }
    }
}
]
}

```

Step 2: Add Fiber Spans

Add fiber segments between nodes:

```

{
  "uid": "fiber_A_to_B",
  "type": "Fiber",
  "type_variety": "SSMF",
  "params": {
    "length": 65.0,
    "length_units": "km",
    "loss_coef": 0.2,           // dB/km (optional, from library if
                               // omitted)
    "con_in": 0.5,             // Input connector loss (dB)
    "con_out": 0.5,            // Output connector loss (dB)
    "att_in": 0,                // Additional input attenuation (dB)
    "pmd_coef": 0.1             // ps/sqrt(km) (optional)
  }
}

```

Parameter Guidelines:

- **length:** Physical fiber length in km
- **loss_coef:** Typical SSMF = 0.2 dB/km at 1550 nm
- **con_in/out:** Connector losses, typically 0.3-0.5 dB each
- Leave other params blank for auto-fill from equipment library

Step 3: Add ROADMs (If Applicable)

For networks with optical switching:

```

{
  "uid": "ROADM_Stockholm",
  "type": "Roadm",
  "type_variety": "default",
  "params": {
    "target_pch_out_db": -20 // Target per-channel power
  }
}

```

Step 4: Add Amplifiers (Optional)

Let GNPpy auto-place, OR specify manually:

```

{
  "uid": "amp_1",
  "type": "Edfa",
  "type_variety": "std_medium_gain",
  "operational": {
    "gain_target": 16.0,      // dB
    "tilt_target": 0.0        // dB
  }
}

```

When to specify amplifiers manually:

- Matching existing deployed network
- Specific amplifier models required
- Non-uniform span lengths ($> 30\%$ variation)
- Multi-band systems (C+L)

Auto-placement: Omit amplifiers from topology, use `-save-network` to generate optimized design.

Step 5: Define Connections

Link elements in order:

```
{
  "connections": [
    {
      "from_node": "Site_A",
      "to_node": "fiber_A_to_amp1"
    },
    {
      "from_node": "fiber_A_to_amp1",
      "to_node": "amp_1"
    },
    {
      "from_node": "amp_1",
      "to_node": "fiber_amp1_to_B"
    },
    {
      "from_node": "fiber_amp1_to_B",
      "to_node": "Site_B"
    }
  ]
}
```

Some Rules:

- Every `from_node` and `to_node` must match a `uid`
- Connections define signal flow direction (specify both links for bidirectional connection)
- Valid element sequences:
 - Transceiver $\rightarrow$ Fiber $\rightarrow$ Transceiver (minimal)
 - Transceiver $\rightarrow$ Fiber $\rightarrow$ Edfa $\rightarrow$ Fiber $\rightarrow$ Transceiver (with amplifier)
 - Transceiver $\rightarrow$ Roadm $\rightarrow$ Fiber $\rightarrow$ Edfa $\rightarrow$ Fiber $\rightarrow$ Roadm $\rightarrow$ Transceiver (with switching)
- Cannot connect Transceiver directly to Edfa (needs Fiber between)
- Cannot connect Fiber directly to Fiber (needs element between)
- ROADMs can connect to Transceiver, Fiber, or other ROADMs

Common connection errors:

```
# WRONG: Missing fiber between elements
{"from_node": "Site_A", "to_node": "amp_1"} # Error!

# CORRECT: Fiber between transceiver and amplifier
{"from_node": "Site_A", "to_node": "fiber_1"}
{"from_node": "fiber_1", "to_node": "amp_1"}
```

6.3.3 Common Topology Patterns*Point-to-Point Link (< 80 km)*

```
{
  "elements": [
    {"uid": "Trx_A", "type": "Transceiver"},
  ]
}
```

```

    {"uid": "fiber_1", "type": "Fiber", "type_variety": "SSMF",
     "params": {"length": 60.0, "length_units": "km"}},
    {"uid": "Trx_B", "type": "Transceiver"}
  ],
  "connections": [
    {"from_node": "Trx_A", "to_node": "fiber_1"},
    {"from_node": "fiber_1", "to_node": "Trx_B"}
  ]
}

```

Amplified Link (80-120 km)

```

{
  "elements": [
    {"uid": "Trx_A", "type": "Transceiver"},
    {"uid": "fiber_1", "type": "Fiber", "params": {"length": 80.0}},
    {"uid": "EDFA_1", "type": "Edfa", "type_variety": "std_medium_gain"},
    {"uid": "fiber_2", "type": "Fiber", "params": {"length": 40.0}},
    {"uid": "Trx_B", "type": "Transceiver"}
  ],
  "connections": [
    {"from_node": "Trx_A", "to_node": "fiber_1"},
    {"from_node": "fiber_1", "to_node": "EDFA_1"},
    {"from_node": "EDFA_1", "to_node": "fiber_2"},
    {"from_node": "fiber_2", "to_node": "Trx_B"}
  ]
}

```

Multi-Span Long-Haul (> 200 km)

Pattern: $\text{Trx} \rightarrow (\text{Fiber} \rightarrow \text{EDFA})_N \rightarrow \text{Fiber} \rightarrow \text{Trx}$

Typical span length: 80-100 km per fiber segment

ROADM Node (Add/Drop with Routing)

```

{
  "elements": [
    {"uid": "Trx_A", "type": "Transceiver"},
    {"uid": "fiber_in", "type": "Fiber", "params": {"length": 5.0}},
    {"uid": "ROADM_1", "type": "Roadm", "type_variety": "default"},
    {"uid": "fiber_to_drop", "type": "Fiber", "params": {"length": 2.0}},
    {"uid": "Trx_Local", "type": "Transceiver"},
    {"uid": "fiber_out", "type": "Fiber", "params": {"length": 80.0}},
    {"uid": "Trx_B", "type": "Transceiver"}
  ],
  "connections": [
    // Express path through ROADM
    {"from_node": "Trx_A", "to_node": "fiber_in"},
    {"from_node": "fiber_in", "to_node": "ROADM_1"},
    {"from_node": "ROADM_1", "to_node": "fiber_out"},
    {"from_node": "fiber_out", "to_node": "Trx_B"},
    // Add/drop at ROADM
    {"from_node": "ROADM_1", "to_node": "fiber_to_drop"},
    {"from_node": "fiber_to_drop", "to_node": "Trx_Local"}
  ]
}

```

Use case: Metro/regional networks with wavelength add/drop and flexible routing

6.3.4 Equipment Configuration

Using Default Equipment Library

GNPy includes comprehensive defaults in `eqpt_config.json`:

```
# Use GNPy's default library
gnpy-transmission-example topology.json \
-e $(python -c "import gnpy; print(gnpy.__path__[0])")/\
  example-data/eqpt_config.json
```

Customizing Equipment

Create custom `my_eqpt_config.json`:

```
{
  "Fiber": {
    "SSMF": {
      "type_variety": "SSMF",
      "dispersion": 16.7,           // ps/(nm km) at 1550nm
      "gamma": 1.3e-3,             // 1/(W m)
      "pmd_coef": 0.1              // ps/sqrt(km)
    },
    "NZDSF": {
      "type_variety": "NZDSF",
      "dispersion": 4.0,
      "gamma": 1.8e-3,
      "pmd_coef": 0.05
    }
  },
  "Edfa": {
    "std_medium_gain": {
      "type_variety": "std_medium_gain",
      "type_def": "variable_gain",
      "gain_flatmax": 25,          // dB
      "gain_min": 15,              // dB
      "p_max": 21,                 // dBm total output
      "nf_min": 5.5,               // dB minimum noise figure
      "nf_max": 7.0,               // dB maximum noise figure
      "out_voa_auto": true,
      "allowed_for_design": false  // If true, can be auto-placed
    }
  },
  "Transceiver": {
    "default": {
      "type_variety": "default",
      "frequency": {...},
      "mode": [
        {
          "format": "DP-QPSK",
          "baud_rate": 32e9,
          "OSNR": 11,               // Required OSNR (dB)
          "bit_rate": 100e9,
          "roll_off": 0.15,
          "tx_osnr": 40,
          "min_spacing": 50e9,
          "cost": 1,                // For mode selection
          "penalties": {
            "chromatic_dispersion": 80000, // ps/nm tolerance

```

```

        "pmd": 40,                // ps tolerance
        "pdl": 5.5                // dB tolerance
    }
}
]
}
}
"SI": [
{
    "f_min": 191.35e12,           // Hz - spectrum start
    "f_max": 196.10e12,           // Hz - spectrum end
    "baud_rate": 32e9,            // Hz - channel baud rate
    "spacing": 50e9,              // Hz - channel spacing
    "power_dbm": 0,               // dBm - per channel power
    "delta_pdb_per_channel": 0,   // dB - power ripple
    "tx_osnr": 40,                // dB - transmitter OSNR
    "sys_margins": 0              // dB - system margin
}
]
}

```

Where to Find Real Parameters

Parameter	Source
Fiber properties	Vendor datasheet (Corning, Prysmian, etc.) ITU-T G.652/G.655 standards
Amplifier specs	Equipment datasheet (gain, NF, output power) Site survey data if deployed
ROADM specs	Vendor specs (loss, add/drop OSNR) Typical: 17 dB loss, 25 dB add/drop OSNR
Transceiver modes	Module datasheet (OSNR requirement) OIF/IEEE standards for common formats
Channel plan	Network design documents ITU-T G.694.1 grid specifications

Table 6.4: Parameter Source Reference

Typical Values Quick Reference:

Parameter	Typical Value	Unit
SSMF loss	0.18-0.22	dB/km
SSMF dispersion	16-17	ps/(nm·km)
SSMF PMD coef	0.04-0.1	ps/ $\sqrt{km}$
EDFA gain	15-25	dB
EDFA noise figure	4.5-6.5	dB
EDFA output power	17-23	dBm
ROADM loss	15-20	dB
ROADM add/drop OSNR	23-28	dB
Launch power	-2 to +2	dBm/ch
Channel spacing	50-100	GHz

Table 6.5: Typical Parameter Values

6.3.5 Validation Checklist

Before running simulation, verify:

GNPy Implementation Analysis

- ☐ All node uids are unique
- ☐ All from_node/to_node reference existing uids
- ☐ Every transceiver is connected to network
- ☐ Fiber exists between all amplifiers/transceivers
- ☐ All type_variety values exist in equipment library
- ☐ Fiber lengths are realistic (> 0 km)
- ☐ No isolated nodes or disconnected segments
- ☐ JSON syntax is valid (use online validator)
- ☐ Connections form valid paths (no dead ends)

Quick Validation:

```
# Test JSON syntax
python -m json.tool topology.json

# Validate with GNPY
gnpy-transmission-example topology.json \
  -e eqpt_config.json --verbose
# Check for ERROR messages
```

6.4. PRACTICAL WORKFLOWS**6.4.1 Workflow 1: Network Feasibility Study**

Scenario: Evaluate if existing network can support new services

Steps:**1. Model existing network**

```
# Create topology from network documentation
vim my_network.json

# Include actual fiber lengths, amplifier locations
```

2. Run full spectrum simulation

```
gnpy-transmission-example my_network.json \
  -e eqpt_config.json \
  --show-channels \
  -o results.csv
```

3. Analyze results

```
# Check for any failing channels
grep "PASS\|FAIL" results.csv

# Identify worst-case GSNR
awk -F',' '{print $1, $5}' results.csv | sort -k2 -n
```

4. Make decision

- All channels GSNR > 15 dB with > 3 dB margin → **Feasible**
- Some channels marginal (15-17 dB) → **Feasible with risk**
- Channels < 15 dB → **Not feasible, needs upgrades**

5. Interpret bottlenecks

```
# If GSNR is low, check if ASE or NLI limited
python << EOF
import pandas as pd
df = pd.read_csv('results.csv')
for idx, row in df[df['GSNR'] < 17].iterrows():
    osnr_ase = row['OSNR_ASE']
    osnr_nli = row['OSNR_NLI']
```

```

if osnr_ase < osnr_nli:
    print(f"Channel {idx}: ASE limited (add amplifiers)")
else:
    print(f"Channel {idx}: NLI limited (reduce power)")
EOF

```

Possible actions:

- ASE limited → Add amplifiers, reduce span lengths
- NLI limited → Reduce launch power, increase channel spacing
- CD/PMD issues → Upgrade transceivers with higher tolerance

6.4.2 Workflow 2: Path Request Validation

Scenario: Check if specific service can be provisioned

Path requests validate whether specific services are feasible given network topology and constraints. Unlike transmission-example (which evaluates entire spectrum), path-request checks individual services.

Steps:

1. Understand service file structure

A service request file contains one or more path requests:

```

{
  "path-request": [
    {
      // Each service request object
    }
  ]
}

```

Required fields per request:

- request-id: Unique service identifier
- source: Source node UID from topology
- destination: Destination node UID from topology
- src-tp-id: Source termination point (usually same as source)
- dst-tp-id: Destination termination point (usually same as destination)
- path-constraints: Service requirements (bandwidth, mode, spectrum)

2. Create basic service request

Minimal 100G service:

```

{
  "path-request": [
    {
      "request-id": "Service_100G_A_to_B",
      "source": "Site_A",
      "destination": "Site_B",
      "src-tp-id": "Site_A",
      "dst-tp-id": "Site_B",
      "bidirectional": false,
      "path-constraints": {
        "te-bandwidth": {
          "technology": "flexi-grid",
          "trx_type": "vendorA_transponder",
          "trx_mode": null, // Auto-select best mode
          "spacing": 50000000000.0, // 50 GHz
          "path_bandwidth": 100000000000.0 // 100 Gb/s
        }
      }
    }
  ]
}

```

```
]
}
```

Key parameters:

- **bidirectional**: If true, validates both $A \rightarrow B$ and $B \rightarrow A$
- **trx_type**: Must match transceiver in equipment library
- **trx_mode**: Specific mode or null for auto-selection
- **spacing**: Channel spacing in Hz (50 GHz = 50e9)
- **path_bandwidth**: Required bit rate in bit/s

3. Add spectrum constraints

Control wavelength assignment using **effective-freq-slot**:

```
{
  "path-constraints": {
    "te-bandwidth": {
      "technology": "flexi-grid",
      "trx_type": "vendorA_transponder",
      "trx_mode": null,
      "effective-freq-slot": [
        {
          "N": 0,          // Central frequency index
          "M": 8          // Slot width (M      6.25 GHz)
        }
      ],
      "spacing": 50000000000.0,
      "path_bandwidth": 200000000000.0
    }
  }
}
```

Frequency slot calculation:

- ITU-T flex grid: frequency = 193.1 THz + $N \times 6.25$ GHz
- Bandwidth = $M \times 6.25$ GHz
- Example: $N=0, M=8 \rightarrow 193.1$ THz center, 50 GHz bandwidth

4. Add route constraints

Force or exclude specific nodes/paths:

```
{
  "explicit-route-objects": {
    "route-object-include-exclude": [
      {
        "explicit-route-usage": "route-exclude-object",
        "index": 0,
        "num-unnum-hop": {
          "node-id": "Problematic_Node",
          "hop-type": "STRICT"
        }
      },
      {
        "explicit-route-usage": "route-include-object",
        "index": 1,
        "num-unnum-hop": {
          "node-id": "Preferred_Node",
          "hop-type": "LOOSE"
        }
      }
    ]
  }
}
```

Constraint types:

- route-exclude-object: Path must NOT traverse this node
- route-include-object: Path must traverse this node
- STRICT: Exact match required
- LOOSE: Allow flexible routing through node

5. Complete service file example**Multiple services with different requirements:**

```
{
  "path-request": [
    {
      "request-id": "Low_Latency_100G",
      "source": "Site_A",
      "destination": "Site_C",
      "src-tp-id": "Site_A",
      "dst-tp-id": "Site_C",
      "bidirectional": true,
      "path-constraints": {
        "te-bandwidth": {
          "technology": "flexi-grid",
          "trx_type": "vendorA_transponder",
          "trx_mode": "DP-QPSK",          // Force specific mode
          "spacing": 50000000000.0,
          "path_bandwidth": 100000000000.0
        }
      },
      "explicit-route-objects": {
        "route-object-include-exclude": [
          {
            "explicit-route-usage": "route-exclude-object",
            "index": 0,
            "num-unnum-hop": {
              "node-id": "Site_B",          // Avoid Site_B
              "hop-type": "STRICT"
            }
          }
        ]
      }
    },
    {
      "request-id": "High_Capacity_400G",
      "source": "Site_A",
      "destination": "Site_C",
      "src-tp-id": "Site_A",
      "dst-tp-id": "Site_C",
      "bidirectional": false,
      "path-constraints": {
        "te-bandwidth": {
          "technology": "flexi-grid",
          "trx_type": "vendorB_transponder",
          "trx_mode": null,                // Auto-select
          "effective-freq-slot": [
            {"N": 10, "M": 16}           // 100 GHz slot
          ],
          "spacing": 100000000000.0,
          "path_bandwidth": 400000000000.0
        }
      }
    }
  ]
}
```

```

    }
  ]
}

```

6. Run path request

```

gnpy-path-request my_network.json my_services.json \
-e eqpt_config.json \
--output-json response.json \
-o results.csv

```

7. Interpret response

Check feasibility:

```

python << EOF
import json
with open('response.json') as f:
    resp = json.load(f)

for req in resp['response']:
    rid = req.get('request-id', 'Unknown')
    if 'blocking-reason' in req:
        print(f"BLOCKED {rid}: {req['blocking-reason']}")
    else:
        path = req['path-properties']
        gsnr = path['z-end']['GSNR-0.1nm']
        mode = path.get('modulation-format', 'N/A')
        print(f"FEASIBLE {rid}: GSNR={gsnr:.2f} dB, Mode={mode}")
EOF

```

Successful response structure:

```

{
  "response": [
    {
      "request-id": "Service_100G_A_to_B",
      "path-properties": {
        "path-route-object": [
          {"path-route-object-uid": "Site_A"},
          {"path-route-object-uid": "fiber_1"},
          {"path-route-object-uid": "Site_B"}
        ],
        "z-end": {
          "GSNR-0.1nm": 21.3,
          "OSNR-ASE-0.1nm": 23.1,
          "OSNR-NLI-0.1nm": 28.4,
          "CD-ps-per-nm": 1200,
          "PMD-ps": 7.5
        },
        "modulation-format": "DP-QPSK",
        "OSNR-required": 15.0,
        "bit-rate": 100000000000
      }
    }
  ]
}

```

8. Understand blocking reasons

Blocking Reason	Meaning	Action
NO_PATH	No route exists	Check topology
NO_SPECTRUM	All wavelengths occupied	Increase capacity
NO_FEASIBLE_MODE	GSNR too low	Add amplifiers
NO_PATH_WITH_CONSTRAINT	Constraints too restrictive	Relax constraints
UNKNOWN_TRANSCIVER	Invalid <code>trx_type</code>	Check equipment

Table 6.6: Service Request Blocking Reasons

6.4.3 Workflow 3: Network Design Optimization

Scenario: Design new network or optimize existing

Steps:

1. Create minimal topology

```
{
  "elements": [
    {"uid": "A", "type": "Transceiver"},
    {"uid": "fiber1", "type": "Fiber",
     "params": {"length": 120}},
    {"uid": "B", "type": "Transceiver"}
  ],
  "connections": [
    {"from_node": "A", "to_node": "fiber1"},
    {"from_node": "fiber1", "to_node": "B"}
  ]
}
```

2. Run auto-design

```
gnpy-transmission-example minimal_topology.json \
-e eqpt_config.json \
--save-network designed_network.json
```

3. Review design

```
# GNPY added amplifiers automatically
# Check amplifier placement
cat designed_network.json | grep -A5 '"type": "Edfa"'

# Check configured gains
cat designed_network.json | grep "gain_target"
```

4. Iterate

- Adjust span lengths
- Specify amplifier types
- Fine-tune power levels
- Re-run and compare

6.4.4 Workflow 4: Troubleshooting Low GSNR

Scenario: Simulation shows $\text{GSNR} < 15$ dB

Diagnostic Steps:

1. Identify bottleneck

```
# Run with verbose output
gnpy-transmission-example topology.json \
-e eqpt_config.json \
--show-channels --verbose
```

2. Check OSNR components

GSNR: 12.3 dB <- Too low
 OSNR ASE: 14.1 dB <- Amplifier noise dominant
 OSNR NLI: 28.5 dB <- Nonlinear OK

Analysis: ASE noise is the problem.

3. Solutions by root cause

Root Cause	Solutions
Too many amplifiers	Reduce span count (consolidate spans) Use lower-loss fiber
High amplifier NF	Specify better amplifiers in equipment config Check actual vs. configured NF
Long spans	Add intermediate amplifier Increase launch power (within limits)
Low launch power	Increase Tx power Optimize amplifier output power

Table 6.7: OSNR Troubleshooting Guide

Example fix:

```
// Change amplifier to lower noise figure
{
  "uid": "amp_1",
  "type": "Edfa",
  "type_variety": "high_performance", // Was: std_medium_gain
  "operational": {
    "gain_target": 18.0 // Adjust for span loss
  }
}
```

6.4.5 Workflow 5: Comparing Design Alternatives

Scenario: Evaluate multiple design options

Steps:

1. Create variants

```
# Option A: SSMF fiber
cp topology.json topology_ssmf.json

# Option B: NZDSF fiber
cp topology.json topology_nzdsf.json
# Edit: change type_variety to "NZDSF"

# Option C: Tighter span length
cp topology.json topology_short_spans.json
# Edit: reduce fiber lengths
```

2. Run all scenarios

```
for topo in topology_*.json; do
  echo "Simulating $topo"
  gnpys-transmission-example $topo \
    -e eqpt_config.json \
    -o results_${topo%.json}.csv
done
```

3. Compare results

```
# Extract key metrics
for csv in results_*.csv; do
    echo "=== $csv ==="
    # Worst GSNR
    awk -F',' 'NR>1 {print $5}' $csv | \
        sort -n | head -1
    # Average GSNR
    awk -F',' 'NR>1 {sum+=$5; count++;} \
        END {print sum/count}' $csv
done
```

4. Decision matrix

Option	Worst GSNR	Avg GSNR	Amps	Cost
SSMF	16.2 dB	18.5 dB	8	\$
NZDSF	17.5 dB	19.8 dB	8	\$\$
Short spans	18.1 dB	20.2 dB	12	\$\$\$

Table 6.8: Design Comparison Example

6.5. COMMON ISSUES AND SOLUTIONS

6.5.1 File and Configuration Issues

JSON Syntax Errors

Error:

json.decoder.JSONDecodeError: Expecting ',' delimiter: line 45

Solutions:

```
# Validate JSON syntax
python -m json.tool topology.json

# Use JSON linter
jsonlint topology.json

# Common mistakes:
# - Missing comma between elements
# - Extra comma after last element
# - Unquoted strings
# - Unescaped quotes in strings
```

Missing type_variety

Error:

KeyError: 'my_custom_fiber' not found in equipment library

Solution:

```
// In topology.json, use type_variety from equipment library
{
    "uid": "fiber_1",
    "type": "Fiber",
    "type_variety": "SSMF" // Must exist in eqpt_config.json
}

// Or add to equipment library:
// In eqpt_config.json:
```

```
{
  "Fiber": {
    "my_custom_fiber": {
      "type_variety": "my_custom_fiber",
      "dispersion": 16.7,
      // ... other parameters
    }
  }
}
```

Disconnected Network

Error:

NetworkXNoPath: No path between Site_A and Site_B

Solution:

```
# Verify connections form complete path
# Check that every node in path has:
#   - "from_node" connection (except start)
#   - "to_node" connection (except end)

# Debug: visualize network
python << EOF
import json
import networkx as nx
import matplotlib.pyplot as plt

with open('topology.json') as f:
    data = json.load(f)

G = nx.DiGraph()
for elem in data['elements']:
    G.add_node(elem['uid'])
for conn in data['connections']:
    G.add_edge(conn['from_node'], conn['to_node'])

# Check if path exists
print(nx.has_path(G, 'Site_A', 'Site_B'))

# Visualize
nx.draw(G, with_labels=True)
plt.savefig('network_graph.png')
EOF
```

6.5.2 Physical Layer Issues

Amplifier Saturation

Warning:

WARNING: Amplifier amp_1 output power (23.5 dBm) exceeds
p_max (21.0 dBm)

Solutions:

Option 1: Reduce channel count

```
// In spectrum configuration
{
  "power_dbm": 0,
```

```
"nb_channel": 80 // Reduce from 96
}
```

Option 2: Use higher-power amplifier

```
{
  "Edfa": {
    "high_power": {
      "type_variety": "high_power",
      "gain_flatmax": 25,
      "p_max": 24 // Higher than 21 dBm
    }
  }
}
```

Option 3: Adjust delta_p

```
{
  "uid": "amp_1",
  "operational": {
    "delta_p": -1.0 // Reduce by 1 dB
  }
}
```

Excessive Chromatic Dispersion

Issue:

Transceiver Site_B:

CD (ps/nm): 45000 <- Exceeds limit

CD Penalty: 5.2 dB <- Too high

Solutions:

Option 1: Use DCF (Dispersion Compensating Fiber)

```
// Add DCF element
{
  "uid": "dcf_module_1",
  "type": "Fiber",
  "type_variety": "DCF",
  "params": {
    "length": 10, // km
    "dispersion": -80 // ps/(nm km) - negative!
  }
}

// Insert in topology before receiver
```

Option 2: Change transceiver mode

```
// Use mode with higher CD tolerance
{
  "trx_mode": "mode 2", // Higher tolerance
  "baud_rate": 32e9 // Lower baud rate = more tolerance
}
```

Option 3: Use NZDSF fiber

```
// Replace SSMF with NZDSF
{
  "type_variety": "NZDSF" // ~4 ps/(nm km) vs 17
}
```

*High PMD Penalty***Issue:**

PMD (ps): 45

PMD Penalty: 3.1 dB <- Too high

Solutions:**Option 1: Reduce PMD coefficient**

```
// Use better fiber specification
{
  "Fiber": {
    "low_PMD_fiber": {
      "pmd_coef": 0.04 // Instead of 0.1 ps/sqrt(km)
    }
  }
}
```

Option 2: Shorter path

- PMD scales as $\sqrt{length}$
- Consider alternative routing
- Add regeneration point

6.5.3 Service Request Issues

NO_FEASIBLE_MODE

Error:

Service_1: blocking_reason = NO_FEASIBLE_MODE

Meaning: GSNR too low for all available transceiver modes

Solutions:**1. Check GSNR vs requirements**

```
# Run transmission example first
gnpy-transmission-example topology.json -e eqpt_config.json

# Compare GSNR to mode requirements:
# Mode 1 (DP-QPSK): requires 11 dB
# Mode 2 (DP-16QAM): requires 18 dB
# Mode 3 (DP-64QAM): requires 25 dB
```

2. Improve GSNR

- Add amplifiers
- Use better equipment (lower NF)
- Reduce span lengths
- Increase launch power

3. Use lower-order modulation

```
// Add DP-QPSK mode to transceiver
{
  "mode": [
    {
      "format": "DP-QPSK",
      "OSNR": 11, // Lower requirement
      "bit_rate": 100e9
    }
  ]
}
```

*NO_SPECTRUM***Error:**

```
Service_2: blocking_reason = NOT_ENOUGH_SPECTRUM
```

Solutions:**Option 1: Wider spectrum**

```
{
  "f_min": 191.35e12,
  "f_max": 196.10e12  // Expand C-band
}
```

Option 2: Narrower channels

```
{
  "spacing": 37.5e9  // Instead of 50 GHz
}
```

Option 3: Review existing allocations

```
# Check spectrum usage
gnpy-path-request topology.json services.json \
-e eqpt_config.json --verbose | grep "Spectrum"
```

6.5.4 Performance Issues*Slow Simulation*

Issue: Simulation takes > 5 minutes

Solutions:**1. Reduce channel count**

```
{
  "nb_channel": 40  // Instead of 96
}
```

2. Use simpler NLI model

```
// In sim_params.json
{
  "nli_params": {
    "method": "gn_model_analytic"  // Faster than
    ggn_spectrally_separated
  }
}
```

3. Disable Raman if not needed

```
{
  "raman_params": {
    "flag": false
  }
}
```

Memory Issues

Issue: MemoryError for large networks

Solutions:**1. Process in segments**

```
# Simulate portions separately
gnpy-transmission-example segment1.json ...
gnpy-transmission-example segment2.json ...
```

2. Reduce computed channels

```
{
  "nli_params": {
    "computed_channels": [1, 20, 40, 60, 80, 96]  // Subset
  }
}
```

6.6. PARAMETER REFERENCE GUIDE

6.6.1 Finding Realistic Values

Fiber Parameters

From Vendor Datasheets:

Parameter	Datasheet Location	Typical Values
Loss coefficient	"Attenuation" section	0.18-0.22 dB/km @1550nm
Dispersion	"Chromatic dispersion"	15-18 ps/(nm·km) @1550nm
PMD coefficient	"Polarization effects"	0.04-0.1 ps/km
Effective area	"Mode field diameter"	80-90 μm^2

Table 6.9: Fiber Datasheet Reference

ITU-T Standards:

Fiber Type	ITU-T	Loss	Dispersion	Use Case
SSMF	G.652	0.2	16.7	Standard metro/long-haul
NZDSF	G.655	0.22	2-6	DWDM systems
DSF	G.653	0.2	0	Legacy networks
Low-loss	G.652.D	0.16	16.7	Modern long-haul

Table 6.10: Fiber Types by ITU-T Standard

Amplifier Parameters

Key Specifications from Datasheet:

Amplifier: Acme Model XYZ-1000

Gain Range: 15 – 26 dB
 Noise Figure: 4.5 dB (typical), 5.5 dB (max)
 Output Power: +23 dBm (max total)
 Input Power Range: -25 to 0 dBm
 Operating Band: C-band (1530–1565 nm)
 Gain Flatness: 0 .5 dB (over C-band)

Map to GNPY:

```
{
  "Edfa": {
    "Acme_XYZ1000": {
      "type_variety": "Acme_XYZ1000",
      "gain_min": 15,
      "gain_flatmax": 26,
      "nf_min": 4.5,
      "nf_max": 5.5,
      "p_max": 23,
      "f_min": 191.35e12,  // 1565 nm
    }
  }
}
```

```

        "f_max": 195.90e12    // 1530 nm
    }
}
}

```

Transceiver Parameters

From Module Specifications:

Format	Bit Rate	Baud Rate	OSNR	Source
DP-QPSK	100G	32 GBd	11 dB	OIF specs
DP-8QAM	150G	32 GBd	17 dB	Vendor data
DP-16QAM	200G	32 GBd	20 dB	Vendor data
DP-64QAM	300G	48 GBd	28 dB	Vendor data

Table 6.11: Transceiver Mode Specifications

OSNR Requirements - Common Approximations:

- **QPSK**: OSNR $\approx$ 9-12 dB (0.1 nm)
 - **8-QAM**: OSNR $\approx$ 15-18 dB (0.1 nm)
 - **16-QAM**: OSNR $\approx$ 18-21 dB (0.1 nm)
 - **64-QAM**: OSNR $\approx$ 25-28 dB (0.1 nm)
- Add 2-3 dB margin for real deployment.

6.6.2 When You Don't Have Exact Values

Conservative Estimates

Use worst-case values for planning:

```

{
  "Fiber": {
    "conservative_SSMF": {
      "loss_coef": 0.22,      // High end of range
      "pmd_coef": 0.1,       // Conservative PMD
      "dispersion": 17,       // High end
      "con_in": 0.5,          // Account for splices
      "con_out": 0.5
    }
  },
  "Edfa": {
    "conservative_amp": {
      "gain_flatmax": 20,     // Lower than max
      "nf_max": 6.5,          // High NF
      "p_max": 20              // Conservative power
    }
  }
}

```

Default Values Strategy

For initial studies when data is limited:

Component	Parameter	Default Value
Fiber	All	Use GNPpy defaults (<code>type_variety: SSMF</code>)
EDFA	Gain	80% of span loss
EDFA	NF	5.5 dB
ROADM	Loss	17 dB per degree
ROADM	Add/drop OSNR	25 dB
Launch power	Per channel	0 dBm

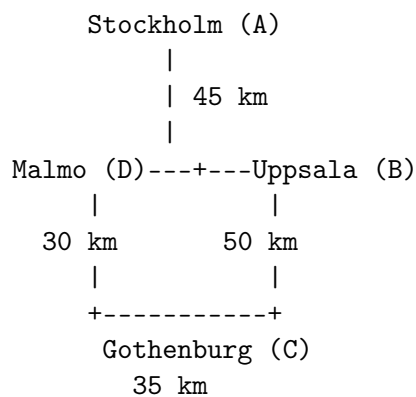
Table 6.12: Recommended Defaults

6.7. COMPLETE WORKED EXAMPLES

6.7.1 Example 1: Metro Ring Network

Scenario: 4-node metro ring, 30-50 km between nodes, 40-channel system

Network Diagram



Complete topology.json

```

{
  "elements": [
    {
      "uid": "Stockholm",
      "type": "Roadm"
    },
    {
      "uid": "fiber_A_B",
      "type": "Fiber",
      "params": {"length": 45.0}
    },
    {
      "uid": "Uppsala",
      "type": "Roadm"
    },
    {
      "uid": "fiber_B_C",
      "type": "Fiber",
      "params": {"length": 50.0}
    },
    {
      "uid": "Gothenburg",
      "type": "Roadm"
    },
    {
      "uid": "fiber_C_D",

```

```

    "type": "Fiber",
    "params": {"length": 35.0}
  },
  {
    "uid": "Malmo",
    "type": "Roadm"
  },
  {
    "uid": "fiber_D_A",
    "type": "Fiber",
    "params": {"length": 30.0}
  },
  {
    "uid": "trx_Stockholm",
    "type": "Transceiver"
  },
  {
    "uid": "trx_Uppsala",
    "type": "Transceiver"
  },
  {
    "uid": "trx_Gothenburg",
    "type": "Transceiver"
  },
  {
    "uid": "trx_Malmo",
    "type": "Transceiver"
  }
],
"connections": [
  {"from_node": "Stockholm", "to_node": "fiber_A_B"},
  {"from_node": "fiber_A_B", "to_node": "Uppsala"},
  {"from_node": "Uppsala", "to_node": "fiber_B_C"},
  {"from_node": "fiber_B_C", "to_node": "Gothenburg"},
  {"from_node": "Gothenburg", "to_node": "fiber_C_D"},
  {"from_node": "fiber_C_D", "to_node": "Malmo"},
  {"from_node": "Malmo", "to_node": "fiber_D_A"},
  {"from_node": "fiber_D_A", "to_node": "Stockholm"},
  {"from_node": "trx_Stockholm", "to_node": "Stockholm"},
  {"from_node": "trx_Uppsala", "to_node": "Uppsala"},
  {"from_node": "trx_Gothenburg", "to_node": "Gothenburg"},
  {"from_node": "trx_Malmo", "to_node": "Malmo"}
]
}

```

Run Simulation

```

# Let GNPpy auto-design amplifiers
gnpy-transmission-example metro_ring.json \
-e eqpt_config.json \
--save-network metro_ring_designed.json \
-o metro_ring_results.csv

```

Expected Results

```

INFO: Auto-design added 8 amplifiers (2 per ROADM)
INFO: auto-design algorithm optimization converged

```

Path: trx_Stockholm to trx_Gothenburg (worst case)

Spans: 2
 Distance: 95 km
 GSNR: 19.2 dB
 Margin: 4.2 dB
 Status: PASS

All 40 channels feasible with > 3 dB margin

6.7.2 Example 2: Long-Haul Point-to-Point

Scenario: 800 km link, 80 km spans, 96 channels

Minimal Topology

```
{
  "elements": [
    {"uid": "Site_A", "type": "Transceiver"},
    {"uid": "Site_Z", "type": "Transceiver"}
  ],
  "connections": []
}
```

Note: GNPpy will create fiber spans and amplifiers automatically.

Custom Equipment Config

```
{
  "Span": {
    "default": {
      "power_mode": true,
      "delta_power_range_db": [-2, 3, 0.5],
      "max_length": 100,
      "length_units": "km",
      "max_loss": 25,
      "padding": 10
    }
  },
  "Edfa": {
    "long_haul": {
      "type_variety": "long_haul",
      "type_def": "variable_gain",
      "gain_flatmax": 23,
      "gain_min": 15,
      "p_max": 22,
      "nf_min": 4.5,
      "nf_max": 6.0,
      "out_voa_auto": true,
      "allowed_for_design": true
    }
  }
}
```

Simulation with Custom Spectrum

```
# Create spectrum definition
cat > spectrum.json << EOF
{
  "spectrum": {
```

```

    "f_min": 191.35e12,
    "f_max": 196.10e12,
    "roll_off": 0.15,
    "baud_rate": 32e9,
    "spacing": 50e9,
    "power_dbm": 0,
    "nb_channel": 96,
    "tx_osnr": 40
  }
}
EOF

# Run simulation
gnpy-transmission-example longhaul.json \
  -e custom_eqpt.json \
  --sim-params spectrum.json \
  --save-network longhaul_designed.json

```

Analysis

```

# Check amplifier count
grep '"type": "Edfa"' longhaul_designed.json | wc -l
# Expected: ~10 amplifiers (800 km / 80 km)

# Verify all channels pass
gnpy-transmission-example longhaul_designed.json \
  -e custom_eqpt.json \
  --show-channels | grep "GSNR" | \
  awk '{if ($3 < 15) print "FAIL: " $0; else print "PASS"}'

```

6.7.3 Example 3: Service Request with Constraints

Scenario: Request 200G service avoiding specific node

Service Definition

```

{
  "path-request": [
    {
      "request-id": "Service_200G_A_to_Z",
      "source": "Site_A",
      "destination": "Site_Z",
      "src-tp-id": "Site_A",
      "dst-tp-id": "Site_Z",
      "bidirectional": false,
      "path-constraints": {
        "te-bandwidth": {
          "technology": "flexi-grid",
          "trx_type": "Voyager",
          "trx_mode": null,
          "effective-freq-slot": [
            {"N": 0, "M": 8}
          ],
          "spacing": 50000000000.0,
          "path_bandwidth": 200000000000.0
        }
      },
      "explicit-route-objects": {
        "route-object-include-exclude": [

```

```

        {
            "explicit-route-usage": "route-exclude-object",
            "index": 0,
            "num-unnum-hop": {
                "node-id": "Problematic_Node",
                "hop-type": "STRICT"
            }
        }
    ]
}
]
}
]
}

```

6. Run path request

```

gnpy-path-request network.json service_200g.json \
-e eqpt_config.json \
--output-json response_200g.json \
-o results_200g.csv

```

7. Interpret response

```

# Check if feasible
python << EOF
import json
with open('response_200g.json') as f:
    resp = json.load(f)

for req in resp['response']:
    if 'blocking-reason' in req:
        print(f"BLOCKED: {req['blocking-reason']}")
    else:
        path = req['path-properties']
        print(f"FEASIBLE:")
        print(f"  Path: {path['path-route-object']}")
        print(f"  GSNR: {path['z-end']['GSNR-0.1nm']:.2f} dB")
        print(f"  Mode: {path['modulation-format']}")
        print(f"  Bit rate: {path['bit-rate']/1e9:.0f} Gb/s")
EOF

```

6.8. BEST PRACTICES SUMMARY

6.8.1 Modeling Checklist

- ☐ **Start Simple:** Begin with minimal topology, add complexity
- ☐ **Use Defaults:** Let GNPpy auto-design initially
- ☐ **Validate Incrementally:** Test after each change
- ☐ **Check Units:** km vs m, GHz vs Hz, dBm vs W
- ☐ **Document Assumptions:** Comment parameter sources
- ☐ **Version Control:** Track topology changes with git
- ☐ **Sanity Check:** Compare results to analytical estimates
- ☐ **Cross-Validate:** Use multiple tools when critical

6.8.2 Performance Guidelines

Metric	Target for Healthy Network
GSNR margin	> 3 dB above mode requirement
Worst-case GSNR	> 15 dB (DP-QPSK), > 20 dB (16-QAM)
Per-channel variation	< 3 dB across all channels
CD accumulation	< 50% of transceiver tolerance
PMD	< 10% of symbol period
Amplifier output	< 90% of p_max
Span loss	18-24 dB (optimal for standard EDFA)

Table 6.13: Network Health Targets

6.8.3 Common Pitfalls to Avoid

1. Wrong Units

- Fiber length in meters when expecting km
- Frequency in GHz when expecting Hz
- Power in watts when expecting dBm

2. Incomplete Connections

- Missing fiber between elements
- Disconnected network segments
- Bidirectional paths without reverse links

3. Unrealistic Parameters

- Amplifier gain > span loss (causes oscillation)
- Fiber loss coefficient < 0 (physical impossibility)
- PMD coefficient > 0.2 ps/km (suspiciously high)

4. Over-Optimistic Design

- Using min NF instead of typical/max
- Ignoring connector losses
- Not including margin for aging

5. Mismatched References

- `type_variety` not in equipment library
- Node UIDs not matching in connections
- Service requests referencing non-existent nodes

6.8.4 When to Use What

Use Case	Recommended Approach
Quick feasibility check	<code>transmission-example</code> with defaults
Detailed service validation	<code>path-request</code> with <code>services.json</code>
Network design from scratch	<code>transmission-example -save-network</code>
Comparing alternatives	Multiple runs with different configs
Optimizing existing network	Manual amplifier placement + <code>auto-design</code>
Spectrum planning	<code>path-request</code> with spectrum constraints
Worst-case analysis	Conservative parameters + full channel count

Table 6.14: Tool Selection Guide

6.9. NEXT STEPS

6.9.1 From Here

After mastering this practical guide:

GNPy Implementation Analysis

1. Deepen Understanding

- Read Chapters 1-5 for internal implementation details
- Study NLI models (GN vs GGN) for accuracy vs speed trade-offs
- Learn auto-design algorithm for power optimization

2. Advanced Topics

- Multi-band systems (C+L band)
- Raman amplification
- Mesh network routing
- Disjoint path computation
- OpenROADM integration

3. Integration

- Use GNPpy as Python library (not just CLI)
- Integrate with network controllers (ONOS, OpenDaylight)
- Automate network planning workflows
- Build optimization algorithms on top of GNPpy

4. Contribute

- Report issues on GitHub
- Share equipment configurations
- Contribute validation results
- Propose new features

6.9.2 Additional Resources**Official Documentation**

- GitHub: <https://github.com/Telecom-Infra-Project/oopt-gnpy>
- Read the Docs: <https://gnpy.readthedocs.io>
- Examples: `gnpy/examples/` directory

Academic References

- GN Model: Poggiolini, P., "The GN Model of Non-Linear Propagation," JLT 2012
- GGN Model: Poggiolini, P. et al., arXiv:1710.02225
- auto-design Algorithm: Auge, J. et al., "Physical-Layer-Aware Path Computation," OFC 2019

Standards

- ITU-T G.652: SSMF specifications
- ITU-T G.655: NZDSF specifications
- ITU-T G.694.1: Flexible DWDM grid
- OpenROADM MSA: Network, device, service models

6.9.3 Getting Help**Problem-Solving Flow:**

Issue?

```

|
+--> Check this chapter (Section 6.5)
|
+--> Search GitHub issues
|   https://github.com/Telecom-Infra-Project/oopt-gnpy/issues
|
+--> Post new issue with:
    - GNPpy version
    - Complete error message
    - Minimal reproducible example
    - What you expected vs what happened

```

6.10. SUMMARY

This chapter provided practical guidance for simulating optical networks with GNPpy:

Key Takeaways:

- GNPpy requires 2 core files: `topology.json` and `eqpt_config.json`
- Start with simple examples, then build complexity
- Let GNPpy auto-design amplifier placement initially
- $\text{GSNR} > 15 \text{ dB}$ with $> 3 \text{ dB}$ margin indicates healthy path
- Use `transmission-example` for network evaluation
- Use `path-request` for service-specific validation
- Check OSNR ASE vs NLI to diagnose issues
- Conservative parameters safer for initial planning

You should now be able to:

- ✓ Install and run GNPpy
- ✓ Create topology files for your network
- ✓ Interpret simulation results
- ✓ Troubleshoot common issues
- ✓ Make engineering decisions from GNPpy output
- ✓ Find and use realistic parameter values

Remember: GNPpy is a powerful planning tool, but always validate critical decisions with:

- Cross-checks using multiple methods
- Field measurements when available
- Vendor specifications
- Safety margins for deployment

With this practical foundation, you're ready to effectively use GNPpy for network planning, design, and optimization.

Appendix A

Quick Reference

A.1. COMMON PARAMETERS

Parameter	Typical Value	Units
Launch power	0	dBm/channel
EDFA gain	15-20	dB
EDFA noise figure	4-6	dB
Fiber loss	0.2	dB/km
Fiber dispersion	16.7	ps/(nm·km)
Fiber gamma	1.3	1/(W·km)
ROADM loss	17	dB
Channel spacing	50-100	GHz

A.2. UNIT CONVERSIONS

$$P_{\text{dBm}} = 10 \log_{10}(P_{\text{mW}})$$

$$P_{\text{dBm}} = 30 + 10 \log_{10}(P_{\text{W}})$$

$$\text{OSNR}_{\text{BW}} = \text{OSNR}_{0.1\text{nm}} - 10 \log_{10} \left(\frac{B_{\text{signal}}}{12.5 \text{ GHz}} \right)$$

$$f_{\text{THz}} = \frac{299792.458}{\lambda_{\text{nm}}}$$

A.3. COMMAND REFERENCE

```
# Transmission example (full spectrum)
gnpy-transmission-example topology.json -e eqpt_config.json

# Path request (service-specific)
gnpy-path-request topology.json services.json -e eqpt_config.json

# With output files
gnpy-path-request topology.json services.json \
  -e eqpt_config.json \
  -o results.csv

# Save auto-designed network
gnpy-transmission-example topology.json \
  -e eqpt_config.json \
  --save-network designed_network.json
```

```
# Show per-channel results
gnpy-transmission-example topology.json \
  -e eqpt_config.json \
  --show-channels
```

Appendix B

Glossary

ASE Amplified Spontaneous Emission - noise from optical amplifiers
CD Chromatic Dispersion - frequency-dependent propagation delay
EDFA Erbium-Doped Fiber Amplifier
FWM Four-Wave Mixing - nonlinear interaction between channels
GGN Gaussian-Gaussian-Noise model - enhanced NLI model
GN Gaussian Noise model - analytical NLI estimation
GSNR Generalized Signal-to-Noise Ratio - includes ASE and NLI
auto-design Local Optimization, Global Optimization - auto-design algorithm
NLI Nonlinear Interference - signal distortion from Kerr effect
OMS Optical Multiplex Section - spectrum management unit
OSNR Optical Signal-to-Noise Ratio - typically ASE-only
PDL Polarization Dependent Loss
PMD Polarization Mode Dispersion
QoT Quality of Transmission - overall link performance
ROADM Reconfigurable Optical Add-Drop Multiplexer
SPM Self-Phase Modulation
SRS Stimulated Raman Scattering - power transfer between channels
XPM Cross-Phase Modulation

Appendix C

Bibliography

Bibliography

Telecom Infra Project, *GNPy: Gaussian Noise Python*, <https://github.com/Telecom-Infra-Project/oopt-gnpy>

P. Poggiolini, *The GN Model of Non-Linear Propagation in Uncompensated Coherent Optical Systems*, IEEE/OSA Journal of Lightwave Technology, vol. 30, no. 24, 2012. arXiv:1209.0394

P. Poggiolini et al., *The GGN Model of Non-Linear Propagation*, arXiv:1710.02225

J. Auge, E. Le Rouzic, M. Curri, *Experimental Demonstration of Flexible Optical Network Planning with Physical-Layer-Aware Path Computation*, OFC 2019.

OpenROADM MSA, *Device, Network, and Service Models*, <http://www.openroadm.org/>

IETF TEAS Working Group, *Yang Data Model for Requesting Path Computation*, draft-ietf-teas-yang-path-computation

Index

For compilation instructions and to generate a full index, use the `makeidx` package with:

```
pdflatex main.tex  
makeindex main  
pdflatex main.tex  
pdflatex main.tex
```